

آموزش صفر تا صد – نسخه ۲۰۲۴

WordPress

صفر تا سایت واقعی و Next.js با Headless

وردپرس کامل: فروشگاه، مهاجرت، ایران‌سازی – و به عنوان بک‌اند؛
Next.js به عنوان فرانت تو

WooCommerce

SEO

Elementor

WordPress 6.9

Next.js App Router

WPGraphQL

REST API

CPT + ACF

Deploy Vercel

۲۰ فصل در ۴ بخش · پروژه: سایت شرکتی + فروشگاه + Headless · تمرین با جواب · چیت‌شیت و گلاساری

فهرست مطالب

نقشه دوره و مسیر چهار بخشی	شروع دوره – نقشه ۴ بخشی	1
CMS، معماری، اکوسیستم، Headless چیست	فصل ۱ – وردپرس چیست	2
دامنه، ایرنیک، DNS، هاست، سی‌پنل، SSL	فصل ۲ – زیرساخت وب: دامنه، هاست، DNS، SSL	3
برنامه Local، اولین سایت، WP-CLI	فصل ۳ – نصب لوکال با Local	4
پست/صفحه، بلوک‌ها، رسانه	فصل ۴ – داشبورد و محتوا	5
Block Themes، Site Editor، تم فرزند	فصل ۵ – تم‌ها و Site Editor	6
هدر/فوتر، منو/مگامنو، اسلایدر، Elementor	فصل ۶ – صفحه‌سازی عملی و Elementor	7
نصب، ضروری‌ها، امنیت، Yoast	فصل ۷ – پلاگین‌ها	8
Yoast/Rank Math، سرچ کنسول، نشان/بld	فصل ۸ – سئو عملی	9
نقش‌ها، Permalinks، تنظیمات	فصل ۹ – کاربران و تنظیمات	10
Headless پایه – Custom Post Type + ACF	فصل ۱۰ – ACF و CPT ★	11
PHP سریع، The Loop، Hooks، اولین پلاگین	فصل ۱۱ – PHP برای دولوپر JS	12
محصولات، سبد، ارسال، درگاه ایرانی، WooCommerce	فصل ۱۲ – ووکامرس 🛒	13
SMTP، Wordfence، بکاپ ۱-۲-۳، بازیابی، کش	فصل ۱۳ – امنیت، بکاپ و نگهداری	14
Application Password، فیلترها، endpoint	فصل ۱۴ – WP REST API	15
GraphQL در ACF، کوئری‌های دقیق، GraphQL	فصل ۱۵ – WPGraphQL ★	16
لیست، [slug]، ISR، پروژه CPT	فصل ۱۶ – Next.js + وردپرس 🚀	17

SEO، Vercel، فرم‌ها، Preview

فصل ۱۷ – Headless پیشرفته

18

خرید هاست، مهاجرت، انماد، درگاه، SMS

فصل ۱۸ – مهاجرت به هاست و ایران‌سازی

19

سه بسته خدماتی، قیمت‌گذاری، قرارداد

فصل ۱۹ – فریلنسری و درآمد

20

مرور سریع + گیرهای کلاسیک

فصل ۲۰ – چیت‌شیت و گلاساری

21

دوره جامع WordPress – از صفر مطلق تا سایت واقعی، فروشگاه و Headless با Next.js

نسخه دوره: ۲۰۲۶ – سازگار با WordPress 6.9.x (با اشاره به 7.0 در راه)، Block Themes/Site Editor،
ووکامرس کامل، زیرساخت ایران (انماد/درگاه/پیامک) و استک Headless مدرن: REST API + WPGraphQL +
ACF مخاطب: دولوپر Next.js که از وردپرس هیچی بلد نیست – و در پایان می‌خواهد هم سایت و فروشگاه تحویل
مشتری بده و هم Next.js رو فرانت و وردپرس رو بک‌اند کنه (Headless)

چرا وردپرس – و چرا برای تو جالبه؟

سه دلیل که این دوره از نظر شغلی برای تو ارزش داره:

- ۱. بازار کار عظیم:** ۴۰٪+ کل وب‌سایت‌های دنیا وردپرسی‌ان. مشتری‌ها و شرکت‌های ایرانی هم عمدتاً وردپرس می‌خوان – و اکثر دولوپرهای وردپرس، Next.js بلد نیستن. ترکیب دوتاش یعنی مزیتی که خیلی‌ها ندارن.
- ۲. Headless = جمع دورهمی مهارت‌ها:** وردپرس فقط پنل مدیریت محتوا می‌شه (رابط کاربری برای مشتری که خودش محتوایش رو وارد کنه) و فرانت رو با همون Next.js می‌سازی که بلدی. مشتری خوشحاله (پنل راحت داره)، تو خوشحالی (React می‌نویسی).
- ۳. مستقل از ابزار، آماده بازار:** سایت شرکتی، فروشگاه ووکامرس با درگاه ایرانی، مهاجرت به هاست، انماد و پیامک – یعنی از همین دوره می‌تونی سه بسته خدماتی بفروشی (فصل ۱۹).

مسیر دوره چهار بخش داره:

- بخش ۱ (فصل ۱-۱۱): وردپرس استاندارد – زیرساخت وب، نصب، محتوا، تم، صفحه‌سازی، پلاگین، سئو، CPT/ACF و کمی PHP
- بخش ۲ (فصل ۱۲-۱۳): فروشگاه و نگهداری – ووکامرس کامل + امنیت/بک‌آپ
- بخش ۳ (فصل ۱۴-۱۷): Headless – REST API، WPGraphQL و اتصال کامل به Next.js با پروژه واقعی
- بخش ۴ (فصل ۱۸-۲۰): آنلاین شدن و بازار کار – مهاجرت به هاست، ایران‌سازی، فریلنسری

بخش ۱ – پایه‌ها و وردپرس استاندارد

فصل	موضوع	چی یاد می‌گیری
فصل ۱	وردپرس چیست؟	CMS، معماری، اکوسیستم، wp.org vs wp.com
فصل ۲	زیرساخت وب 	دامنه، ایرنیک، DNS، هاست، سی‌پنل، SSL
فصل ۳	نصب لوکال	برنامه Local، اولین سایت، ورود به پنل
فصل ۴	داشبورد و محتوا	پست/صفحه، دسته‌بندی، رسانه، ادیتور بلوکی
فصل ۵	تم‌ها	Block Themes، Site Editor، تم فرزند
فصل ۶	صفحه‌سازی عملی 	هدر/فوتر، منو و مگامنو، اسلایدر، فرم، Elementor
فصل ۷	پلاگین‌ها	اکوسیستم، پلاگین‌های ضروری، آناتومی
فصل ۸	سئو عملی 	Yoast/Rank Math، sitemap، سرچ کنسول، نشان/بld
فصل ۹	کاربران و تنظیمات	نقش‌ها، Permalinks (مهم برای API!)، تنظیمات کلیدی
فصل ۱۰	Custom Fields و CPT 	محتوای ساخت‌یافته با ACF – پایه Headless
فصل ۱۱	PHP برای دولوپر JS	سینتکس سریع، Hooks، The Loop، اولین پلاگین خودت

بخش ۲ – فروشگاه و نگهداری

فصل	موضوع	چی یاد می‌گیری
فصل ۱۲	 ووکامرس 	محصولات ساده/متغیر/دانلودی، سبد، ارسال، درگاه ایرانی، WooGraphQL
فصل ۱۳	امنیت و نگهداری 	Wordfence، 2FA، بکاپ ۱-۲-۳، بازیابی، کش، SMTP

فصل	موضوع	چی یاد می‌گیری
فصل ۱۴	WP REST API	curl، Application Passwords، endpoint ها
فصل ۱۵	WPGraphQL ⭐	GraphQL سرور، کوئری‌های دقیق، ACF در GraphQL
فصل ۱۶	Next.js + وردپرس 🚀	پروژه کامل: لیست و صفحه پست با Server Components
فصل ۱۷	Headless پیشرفته	Preview، فرم‌ها، SEO، تصاویر، دپلوی Vercel

بخش ۴ – آنلاین شدن و بازار کار

فصل	موضوع	چی یاد می‌گیری
فصل ۱۸	مهاجرت و ایران‌سازی NEW	خرید هاست، مهاجرت لوکال → هاست، انماد، درگاه، پنل SMS
فصل ۱۹	فریلنسری و درآمد NEW	سه بسته خدماتی، قیمت‌گذاری، قرارداد، نگهداری ماهانه
فصل ۲۰	چیت‌شیت و گلاساری	مرور سریع + گیرهای کلاسیک + نقشه راه

زمین تمرین 🏋️

همه‌چیز روی وردپرس **لوکال** (روی سیستم خودت) انجام می‌شه – هیچ چیز خراب شدنش هزینه نداره:

- سایت تست `wp-course.local` با برنامه **Local** (فصل ۳)
 - سه پروژه دوره‌ای: **سایت شرکتی «برق‌آسا»** (فصل ۶)، **فروشگاه ابزار با ووکامرس** (فصل ۱۲)، **اپ -wp- Headless** **headless** (فصل ۱۶)
 - مهاجرت به هاست واقعی **اول داخل لوکال** شبیه‌سازی می‌شه (فصل ۱۸) – بعد در اولین پروژه واقعی اجرا می‌کنی
- قانون طلایی دوره:** وردپرس رو مثل یک محصول آماده / استفاده کن (فصل ۳-۹) و مثل یک فریم‌ورک توسعه بده (فصل ۱۰-۱۷) – دولوپری که هردو رو بلده، در تیم‌های Headless می‌درخشه.

ابزار	وضعیت تو	چک کردن
ویندوز 10/11	✓	—
Next.js 15+ (App Router)	✓ از پروژه‌هایت	<code>npx next --version</code>
Git/GitHub، CLI، داکر	✓ سه دوره قبلی	کمک‌کار نوی مسیر
PHP / MySQL / هاست	✗ لازم نیست بلد باشی	فصل ۲، ۳ و ۱۱ فقط به اندازه لازم

تبدیل به PDF

نسخه PDF رنگی راست‌به‌چپ (فونت وزیرمتن) در همین پوشه آماده است: «دوره کامل WordPress و Headless با Next.js.pdf». برای بازسازی: `python build/build_pdf.py` + رندر Chrome headless (دستور در پرامپت آماده پوشه والد).

فصل ۱ – وردپرس چیست و چطور فکرش کنیم؟

🎯 هدف این فصل: مدل ذهنی درست: CMS یعنی چی، وردپرس از چه چیزی ساخته شده، اکوسیستمش چیه، و مهم‌تر از همه – اینکه چرا برای تو به عنوان دولوپر Next.js این ابزار جذابه. هنوز چیزی نصب نمی‌کنیم.

۱.۱ – CMS: وقتی مشتری نمی‌تونه کد بزنه

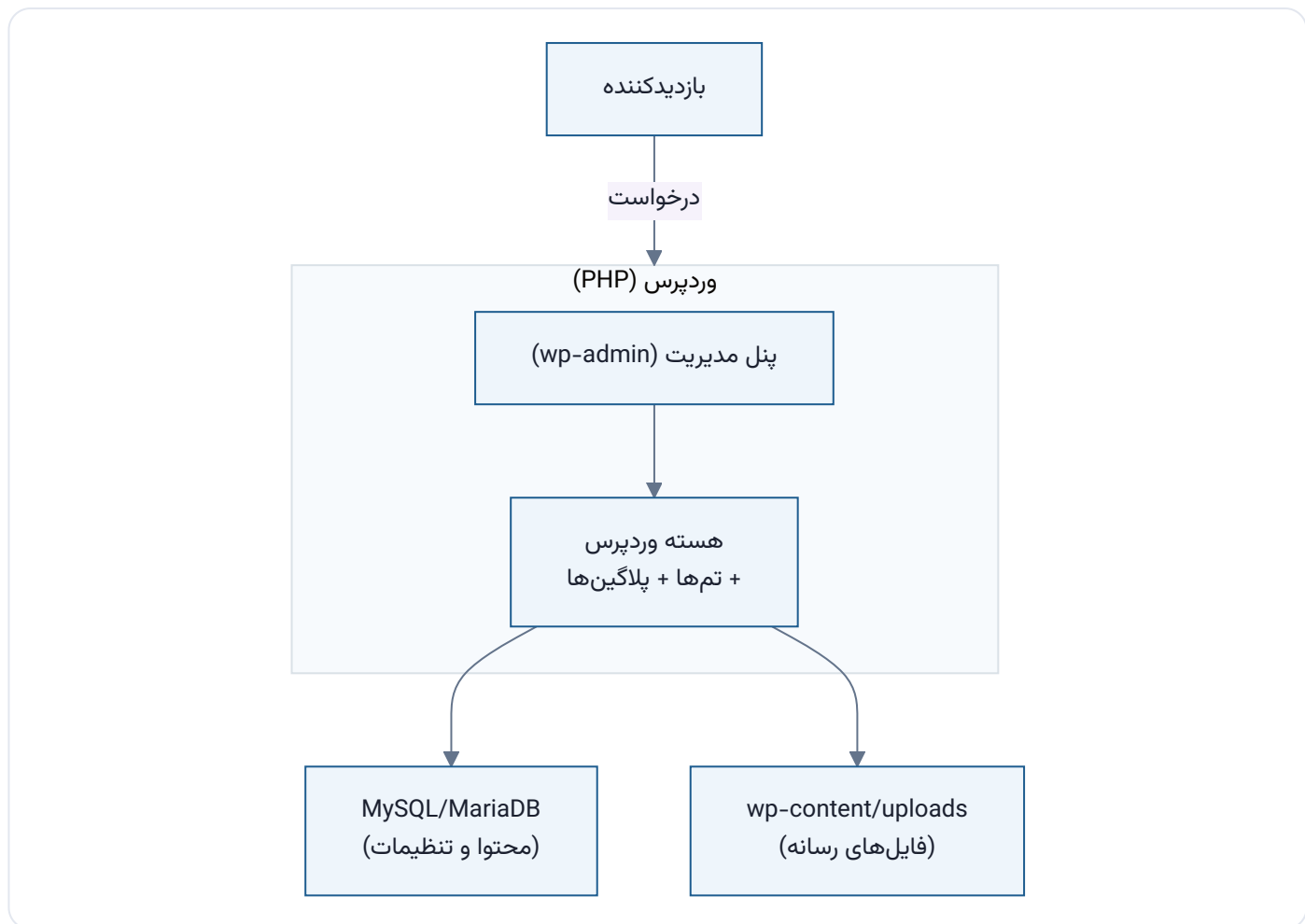
سناریوی آشنا: مشتری یک وب‌سایت می‌خواهد که خودش بتونه اخبارش رو وارد کنه. اگه با Next.js خالص بسازی، هر بار مشتری بخواد یک پست بذاره باید بیاد بهت بگه یک commit بزنی! 🤖

CMS (Content Management System) یعنی: یک پنل مدیریت آماده که محتوا رو مشتری اونجا وارد می‌کنه (عنوان، متن، عکس) و سایت ازش می‌خونه. وردپرس = محبوب‌ترین CMS دنیا (نیروی محرکه حدود ۴۰٪+ کل وب؛ بیش از ۶۰٪ سهم بازار CMS).

سایت نمایش می‌دهد → MySQL مشتری → پنل مدیریت وردپرس → محتوا در

۱.۲ – وردپرس زیر hood چی داره؟ (برای تو که مهندسی)

سه لایه:



جزء	چیه	برای تو چه معنایی داره
PHP	زبان سرور وردپرس (۷.۲+)	فقط به اندازه لازم یاد می‌گیری (فصل ۱۱) – از TS ساده‌تره
MySQL	دیتابیس محتوا	تو SQL بلدی! جدول‌هاش رو فصل ۱۰ نگاه می‌کنیم
wp-content	جایی که تم‌ها، پلاگین‌ها و آپلودها زندگی می‌کنن	★ تنها جایی که تغییر می‌دی؛ هسته رو هرگز دست نمی‌زنی

۱.۳ – اکوسیستم: تم‌ها و پلاگین‌ها

قدرت واقعی وردپرس، اکوسیستمش است (ده‌ها هزار پلاگین رایگان):

مفهوم	چی می‌کنه	شبیه به
تم (Theme)	ظاهر و قالب سایت – نحوه نمایش	design system + layout
پلاگین (Plugin)	افزودن قابلیت: فروشگاه، فرم، SEO، مولتی‌زبان	npm package
ابزارک/بلوک (Block)	واحدهای UI در ادیتور (پاراگراف، گالری...)	کامپوننت‌های React!

💡 ادیتور وردپرس (گوتنبرگ) از داخل با React نوشته شده! یعنی مفهوم «بلوک» که فصل ۴ می‌بینی، دقیقاً همون کامپوننت‌محوریه که تو می‌فهمی.

۱.۴ – wp.org در برابر wp.com (سوال همیشگی)

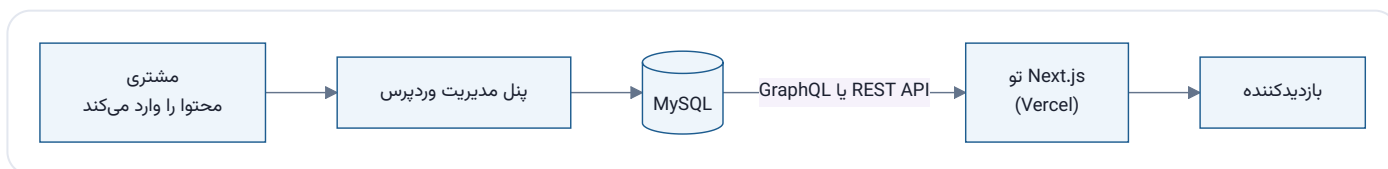
WordPress.com	WordPress.org	
سرویس میزبانی تجاری بر پایه همون	نرم‌افزار متن‌باز – خودت نصب می‌کنی	چییه؟
✗ محدود	✓ کامل	کنترل کامل (تم/پلاگین دلخواه)
✗	✓	نصب لوکال و توسعه
نه	★ این یکی	ما کدوم؟

از این به بعد منظورمون همیشه WordPress.org (خودِ نرم‌افزار) است.

۱.۵ – چرا این دوره برای تو طلایه: Headless

دو راه استفاده از وردپرس:

Headless (بی‌سر!)	Classic (سنتی)	
★ Next.js تو!	تم PHP خود وردپرس	فرانت چییه؟
فقط بک‌اند و پنل محتوا	همه‌چیز: فرانت + بک + پنل	وردپرس چه نقشی داره؟
React/Next (که بلدی!)	PHP	مهارت لازم
★ مدرن SSG/ISR	محدود به PHP	سرعت و UX
همون پنل وردپرس!	پنل وردپرس	برای مشتری



یعنی: مشتری پنل آشنای وردپرس رو می‌گیره، تو فرانت رو با Next.js می‌سازی. بهترین دو دنیا – و دقیقاً همون چیزیه که بخش سوم دوره می‌سازیش.

نکته واقع‌بینانه: وردپرس Headless برای هر پروژه‌ای نیست (سایت‌های خیلی ساده با تم آماده سریع‌تر ردی می‌شن). ولی برای پروژه‌های جدی با AI سفارشی و پرفورمنس بالا، ترکیب برنده است.

جمع‌بندی فصل ✓

- CMS = پنل محتوا برای مشتری؛ وردپرس = پادشاهش (۴۰٪ وب)
- زیر `wp-content` + PHP + MySQL hood: (تنها جای تغییرات تو)
- تم = ظاهر، پلاگین = قابلیت، بلوک = کامپوننت (با React نوشته شده!)
- WordPress.org (نرم‌افزار) نه. com. (سرویس)
- **Headless**: وردپرس فقط بک‌اند و پنل؛ فرانت = Next.js تو

تمرین فصل ۱ 📄

1. با یک جمله: چرا مشتری‌ها CMS می‌خوان ولی دولوپرها همیشه با فریم‌ورک دوستشون ندارن؟
2. در مدل Headless، وردپرس چه لایه‌هایی رو برای تو حل می‌کنه؟ (سه چیز بنویس)
3. فرق wp.org و wp.com رو به یک دوست غیرفنی توضیح بده با analogi مناسب. (مثلاً: اندروید خام در مقابل گوشی با لایسنس...)
4. یک پروژه واقعی فکر کن (مثلاً سایت یک رستوران): کدوم بخش‌هاش محتوا است که مشتری باید خودش ویرایش کنه؟ (منو، آدرس، قیمت‌ها...)

▼ نکته تمرین ۲

پنل مدیریت محتوا (برای مشتری)، REST/GraphQL API (لایه داده استاندارد)، مدیریت کاربران و نقش‌ها و رسانه — یعنی کل بک‌اند CMS. تو فقط UI می‌سازی.

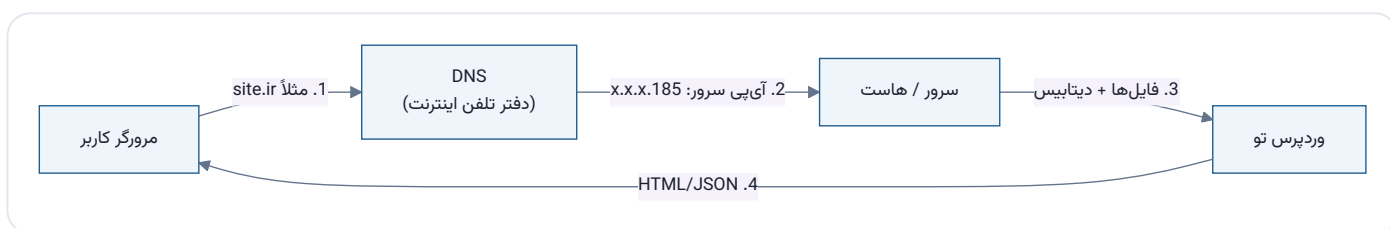
➔ **فصل بعد:** زیرساخت وب — دامنه، هاست، DNS و SSL؛ کلماتی که مشتری و هاستینگ‌ها مدام می‌گویند.

فصل ۲ – زیرساخت وب: دامنه، هاست، DNS و SSL

🎯 هدف: قبل از نصب وردپرس، زبان این دنیا را یاد بگیر: دامنه چیست، هاست کجاست، DNS چطور «آدرس» را به «سرور» وصل می‌کند و SSL چرا حیاتیه. این فصل یعنی: هر وقت مشتری یا هاستینگ گفت «نیم‌اس، سی‌پنل، رکورد MX»، تو دقیقاً بدانی چه می‌گویند. هنوز چیزی نمی‌خریم و نصب نمی‌کنیم.

۲.۱ – نقشه بزرگ: سایت تو کجا زندگی می‌کند؟

وقتی یک URL را باز می‌کنی، این زنجیره اتفاق می‌افتد:



سه موجودیت اصلی:

موجودیت	چیست	آنالوژی از دنیای تو
دامنه (Domain)	اسم سایت: <code>site.ir</code>	اسم ریپو در GitHub
هاست (Hosting)	کامپیوتری روشن که فایل‌ها و دیتابیس تو را نگه می‌دارد	سرور/VPS که اپ Next.js را رویش deploy می‌کنی
DNS	سرویسی که دامنه را به آی‌پی سرور ترجمه می‌کند	routing لایه شبکه

💡 برای تو که با Vercel کار کرده‌ای: وردپرس = اپ تو، هاست = Vercel (ولی خودت نگهش می‌داری)، دامنه = custom domain، DNS = همان تنظیمات `A / CNAME` که موقع وصل کردن دامنه به Vercel زده‌ای! هیچ چیز جدیدی نیست – فقط GUI اش سی‌پنل است.

۲.۲ – دامنه (Domain)

ساختار: `sub.domain.tld` – مثل `blog.mysite.ir`

بخش	مثال	توضیح
TLD (پسوند)	.com , .ir , .net , .shop	com جهانی؛ ir داخلی (با تفاوت مهم – پایین)
نام دامنه	mysite	همان چیزی که می‌خری
زیردامنه	blog. shop.	رایگان – روی هر هاستی می‌سازی

قواعد خرید دامنه:

- کوتاه و خوانا – shop-royal-iran1.com نه؛ royalshop.ir آری
- .ir در برابر .com :

.com	.ir	
گران‌تر (ارزی)	ارزان‌تر (از رجیستری ایران)	ثبت و تمدید
هر رجیستری جهانی	ایرنیک (irnic.ir) ★ – با هویت ملی	ثبت‌کننده رسمی
وابسته به وضعیت بین‌المللی	مقاوم‌تر برای بازار داخلی	تحریم/قطعی
بالا (جهانی)	بالا (بازدیدکننده به دامنه ایرانی اعتماد دارد)	تراست کاربر ایرانی

🔑 ایرنیک (IRNIC) = سازمان ثبت دامنه‌های ایران. برای دامنه .ir باید در irnic.ir حساب با هویت تأییدشده بسازی. پنلش قدیمی است ولی کار می‌کند – در فصل ۱۸ باهاش کار می‌کنیم.

Whois = جستجوی اطلاعات مالکیت دامنه (به whois.ir یا whois.com برو، دامنه را بنویس: مالک، تاریخ ثبت و انقضا). کاربردها: 1. قبل از خرید یک دامنه، ببین آزاد است یا نه 2. ببین دامنه‌ای که مشتری می‌گوید «مال من است» واقعاً به چه کسی ثبت شده (بارها پیش آمده که دامنه به نام سازنده قبلی سایت است، نه مشتری! ⚠️)

۲.۳ – DNS: رکوردهایی که باید بشناسی

DNS مثل دفتر تلفن است: اسم → آدرس. رکوردهای پرکاربرد:

رکورد	کارش	نمونه
A	دامنه → آی پی سرور	site.ir → 185.55.x.x
CNAME	اسم → اسم دیگر	www.site.ir → site.ir
MX	ایمیل های دامنه کجا بروند	mail.site.ir
NS	سرورهای DNS مسئول	ns1.hosting.ir
TXT	احراز مالکیت/امضای ایمیل	رکورد SPF و DKIM

- TTL (زمان کش شدن رکورد) را موقع انتقال کم بگذار (مثلاً ۳۰۰ ثانیه) – وگرنه تغییر DNS تا ۲۴ ساعت کش می ماند
- **مقایسه دولوپری:** وقتی دامنه را به Vercel وصل می کنی و می گوید «رکورد A را به 76.76.21.21 تغییر بده» – دقیقاً همین کار است، فقط اینجا با پنل هاستینگ

۲.۴ – هاست: خانه فایل های تو

انواع هاست (از ارزان به گران):

نوع	چیست	مناسب	معادل ذهنی تو
اشتراکی (Shared) ★	یک سرور، صدها سایت باهم	سایت شرکتی/بلاگ/فروشگاه کوچک	پلن رایگان Vercel
VPS	سرور مجازی خصوصی – منابع تضمینی	فروشگاه شلوغ، چند سایت	VPS که داکر را رویش بالا می آوری
ابری (Cloud)	مقیاس پذیر، پرداخت به مصرف	پروژه های جدی با ترافیک بالا	Vercel Pro / AWS

برای وردپرس به این ها نیاز داری: **PHP + MySQL + وب سرور (Apache/Nginx)** – همان استکی که Local خودش نصب می کند (فصل ۳).

پنل مدیریت هاست – GUI که همه کار با آن است:

- **cPanel** ★ رایج ترین (ایران و جهان)
 - **DirectAdmin** – سبک تر، در هاست های ایرانی زیاد است
 - هر دوتا یک کار می کنند؛ در این دوره از تصاویر سی پنل استفاده می کنیم
- ابزارهای داخل پنل که در فصل ۱۸ از آن ها استفاده می کنیم:

ابزار	کار
File Manager	فایل سیستم هاست در مرورگر (آپلود/اکسپورت zip وردپرس)
MySQL Databases	ساخت دیتابیس + کاربر و رمز
phpMyAdmin	مدیریت دیتابیس در مرورگر (همان که در Local دیدی)
Softaculous / WordPress Installer	نصب وردپرس یک‌کلیکی
SSL/TLS Status	صدور گواهی رایگان Let's Encrypt

⚠ خرید هاست: حتماً هاست وردپرسی/لینوکسی بگیر نه ویندوزی؛ حداقل PHP 8.1 + و MySQL 8؛ و پشتیبانی پاسخ‌گو – در بازار ایران پشتیبانی = نصف قیمت هاست.

۲.۵ – دیتابیس: همان MySQL که می‌شناسی

وردپرس محتوا را در MySQL/MariaDB نگه می‌دارد – همان SQL که با Prisma/Postgres کار کردی، اینجا با این تفاوت‌ها:

- هر وردپرس یک دیتابیس با پیشوند جدول‌ها `wp_` (فصل ۴ با جدول `wp_posts` کار کردیم)
- اطلاعات اتصال در فایل `wp-config.php` است (معادل `.env` تو): `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`
- در هاست، دیتابیس و کاربر را جدا می‌سازی و به هم وصل می‌کنی (بخش MySQL Databases سی‌پنل) – دقت کن: کاربر دیتابیس ≠ کاربر ورود به پنل هاست

💡 چرا این توضیح مهم است؟ چون در فصل ۱۸ موقع نصب روی هاست، پر کردن همین چهار مقدار در `wp-config.php` کل مرحله «اتصال به دیتابیس» است – و ۸۰٪ خطاهای نصب روی هاست از همین‌جاست (کاربر با دیتابیس وصل نشده یا localhost نباشد).

۲.۶ – SSL و HTTPS: قفل سبز کنار آدرس

SSL/TLS = رمزنگاری مسیر مرورگر تا سرور. بدون آن: `http://` (با آیکون هشدار!) و با آن: `https://` (قفل). چرا حیاتی است:

۱. اعتماد کاربر – مرورگرها سایت‌های بدون SSL را «ناامن» برچسب می‌زنند

۲. سئو – گوگل HTTPS را عامل رتبه‌بندی می‌داند (فصل ۸)

3. پرداخت آنلاین – درگاه‌های ایرانی بدون SSL اصلاً کار نمی‌کنند (فصل ۱۲ و ۱۸)

4. API تو – در Headless، Next.js روی Vercel باید با `https` به وردپرس وصل شود (فصل ۱۷)

خبر خوب: امروز SSL ارزان نیست – رایگان است:

- در هاست: **Let's Encrypt** از سی‌پنل (SSL/TLS Status → Run AutoSSL) – تمدید خودکار
- در Local: برنامه خودش گواهی محلی می‌سازد (`https` با اخطار مرورگر که طبیعی است)

🔑 در وردپرس بعد از فعال‌سازی SSL، آدرس سایت در **Settings → General** به `https://` تغییر می‌کند و با یک جست‌وجو-جایگزین لینک‌های قدیمی `http` اصلاح می‌شوند (فصل ۱۸ گام‌به‌گام).

۲.۷ – سه اصطلاح دیگر که خواهی شنید

اصطلاح	یعنی	کجا می‌شنویش
Nameserver (NS)	«DNS سایتت را کدام شرکت نگه می‌دارد» – موقع خرید هاست، دو آدرس <code>ns1...</code> و <code>ns2...</code> می‌دهند که در پنل دامنه وارد کنی	اتصال دامنه به هاست (فصل ۱۸)
Propagation	پخش شدن تغییرات DNS در دنیا (چند دقیقه تا ۲۴ ساعت)	«سایت‌م باز نمی‌شود!» – صبر، نه پنیک
کنترل پنل دامنه	پنل ایرنیک یا رجیستری جهانی – جای تغییر NS	مدیریت دامنه

جمع‌بندی فصل ✓

- زنجیره: دامنه → DNS → هاست (PHP+MySQL) → سایت؛ هر URL باز شده همین مسیر است
- دامنه `.ir` با ایرنیک (ارزان و داخلی) یا `.com` جهانی – whois برای چک مالکیت
- رکوردهای DNS: A و CNAME و MX و NS – همان کارهایی که با دامنه در Vercel کرده‌ای
- هاست اشتراکی برای شروع کافی است؛ سی‌پنل = GUI برای MySQL + File Manager + نصب وردپرس
- **SSL رایگان** (Let's Encrypt) – ضروری برای سئو، اعتماد و درگاه پرداخت

تمرین فصل ۲ 📝

1. برای یک پروژه فرضی (مثلاً سایت فروش شیرینی‌پزی) ۵ نام دامنه پیشنهاد بده؛ یکی `.ir` و یکی `.com` – قیمت تقریبی هر دو را چک کن (`irnic.ir` و یک رجیستری جهانی).
2. آدرس سایت هر شرکتی را در whois بزن: مالک ثبت‌شده کیست؟ تاریخ انقضای دامنه کی است؟

3. سه رکورد DNS را از حافظه توضیح بده: A چه می‌کند؟ MX چه؟ CNAME چه؟

4. فرق هاست اشتراکی و VPS را با آنالوژی Vercel رایگان در برابر VPS شخصی توضیح بده – یک سایت فروشگاهی با ۱۰۰۰ بازدید روزانه کدام؟

5. (کشف) در مرورگر کنار قفل آدرس سایت‌های مختلف را ببین؛ یک سایت `http` بدون SSL پیدا کن – چه هشدار می‌دهد؟

▼ نکته تمرین ۴

۱۰۰۰ بازدید روزانه برای وردپرس خیلی نیست – هاست اشتراکی خوب جواب می‌دهد (وردپرس با کش، فصل ۱۳، صدها بازدید همزمان را مدیریت می‌کند). VPS جایی لازم است که ترافیک سنگین شود یا چند سایت/سرویس داشته باشی – همان جایی که به docker compose فکر می‌کنی.

→ **فصل بعد:** نصب وردپرس روی سیستم خودت با برنامه Local – پنج دقیقه‌ای!

فصل ۳ – نصب لوکال: سایت وردپرسی روی سیستم خودت در ۵ دقیقه

🎯 هدف: وردپرس برای اجرا به سه چیز نیاز داره: PHP + MySQL + وب سرور (همون استک LAMP معروف). به جای نصب دستی، با برنامه Local (محبوبترین ابزار دولوپرهای وردپرس) همه رو یکجا نصب می‌کنیم. آخر فصل، پنل مدیریت تو بالاست!

۳.۱ – راه‌های نصب لوکال (و اینکه چرا Local)

روش	چیه	توصیه
Local (localwp.com)	اپ اختصاصی: PHP+MySQL+سرور یکجا، چند سایت همزمان، CLI و mail catcher	★ استاندارد دولوپرها
Laragon	سبک و سریع ویندوزی	جایگزین خوب
XAMPP	قدیمی‌تر، دستی‌تر	توصیه نمی‌کنم
Docker (وردپرس رسمی)	با دانش دوره داکترت می‌شده!	برای دوره‌های بعدی – Local راحت‌تره

🔗 دانلود: <https://localwp.com> → نسخه Windows. (رایگان؛ ساختاً حساب می‌خواد که فقط ایمیل می‌خواد).
💡 ارتباط با دوره داکر: اگه دقت کنی، Local دقیقاً همون کاری رو می‌کنه که `docker compose up` برای یک استک PHP+MySQL – فقط با GUI. مفهوم «محیط ایزوله هر پروژه» رو حالا از دو زاویه دیدی.

۳.۲ – ساخت اولین سایت، گام به گام

1. Local رو باز کن → دکمه **Create a new site**

2. اسم سایت: `wp-course` → (ادامه)

3. Environment: پیش‌فرض (Preferred) بمون – PHP 8.x + MySQL

4. حساب کاربری وردپرس رو بساز – ★ این اطلاعات رو جایی یادداشت کن: **Username:** `admin` - **Password:** [خودت]
رمز (لوکاله، ساده اشکال نداره) - **Email:** ایمیلی از خودت

5. صبر کن – Local خودش وردپرس رو نصب می‌کنه (~۱ دقیقه)

۳.۳ – دو دکمه طلایی Local

بعد از اتمام، بالای پنل Local سه دکمه می‌بینی:

دکمه	کار
★ WP Admin	باز کردن پنل مدیریت (wp-course.local/wp-admin) – اینجا خونه‌ی اصلی‌ه
Open Site	باز کردن خود سایت (فرانت)
Start/Stop	روشن/خاموش کردن استک سایت

و دو جاهای مفید پایین پنل:

- **Database tab:** اطلاعات اتصال MySQL (Host/Port/User/Pass) – فصل ۱۴ برای اتصال از بیرون لازم می‌شه
- **Go to Site Folder / Open Site Shell:** ★ شل داخل محیط سایت – اینجا wp (WP-CLI)، خط فرمان رسمی وردپرس (نصبه! امتحان کن):

```
$ wp core version
6.9.x
```

(WP-CLI = npm برای وردپرس – مدیریت کامل سایت با CLI. در طول دوره بهش برمی‌گردیم.)

۳.۴ – پنل مدیریت: اولین گشت

WP Admin رو بزنی → با user/pass خودت لاگین کن. خوش اومدی به wp-admin – یک نوار مشکی کنار، منوهای سمت راست (چون وردپرس RTL رو هم پشتیبانی می‌کنه، اینجا فارسی/راست‌چین می‌بینی):

منو	چی توشه	فصل مربوط
Posts	پست‌های وبلاگ (محتوای زمان‌دار)	۳
Pages	صفحات ثابت (درباره ما، تماس...)	۳
Media	کتابخانه تصاویر و فایل‌ها	۳
Appearance	تم‌ها و Site Editor	۴
Plugins	پلاگین‌ها	۵
Users	کاربران و نقش‌ها	۶
Settings	تنظیمات کلیدی	۶

اولین کار حرفه‌ای: **Settings → General → Site Language** فارسی → وردپرس کامل فارسی و RTL می‌شه (ترجمه رسمی خودش داره). Timezone هم بذار Tehran.

۳.۵ – سلامت‌های پایه (عادت از همین اول)

- بعد از هر تغییر مهم: در Local یک **Snapshot** بگیر (دکمه بالا کنار اسم سایت) – مثل کامیت گیت! هر وقت خراب شد، Restore.
- در فصل‌های بعد هر وقت وردپرس آپدیت خواست (نوار بالا)، بذار آپدیت شه – نسخه‌های x.6.9 امنیتی‌ان.

✓ جمع‌بندی فصل

- استک وردپرس = MySQL + PHP + وب‌سرور؛ **Local** همه را با GUI نصب می‌کند
- دو دنیا: **WP Admin** (پنل) و خود سایت؛ WP-CLI = Open Site Shell
- فارسی‌سازی از Settings → Site Language
- Snapshot = کامیت محیطی؛ آپدیت‌های امنیتی را نگیر نده

📝 تمرین فصل ۳

1. Local و سایت wp-course را نصب کن و به پنل لاگین کن.
2. زبان را فارسی کن – منوها را دوباره نگاه کن (همه RTL شدند).
3. در Open Site Shell → Local بزن و wp core version و wp plugin list را بزن – چه پلاگین‌هایی از قبل نصب‌اند؟
4. یک Snapshot به اسم «after-install» بگیر.
5. (کشف آزاد) در Database tab → Local، اطلاعات اتصال را ببین و در یک فایل یادداشت ذخیره کن (DB_HOST، DB_USER، DB_PASSWORD، DB_NAME) – فصل ۱۴ با این‌ها سرور GraphQL/API رو از بیرون تست می‌کنیم.

▼ نکته تمرین ۳

```
wp core version      # 6.9.x
wp plugin list       # فعال/غیرفعال status با - (پلاگین‌های پیش‌فرض) Hello Dolly و Akismet معمولاً
```

WP-CLI از اینجا به بعد دوست خوبته: wp plugin list --status=active و ...

➔ **فصل بعد:** داشبورد و محتوا – پست، صفحه، دسته‌بندی و ادیتور بلوکی گوتنبرگ.

فصل ۴ – داشبورد و محتوا: پست، صفحه، رسانه و ادیتور بلوکی

🎯 هدف: هسته کار روزمره وردپرس: تولید محتوا. فرق Post و Page، دسته/برچسب، کتابخانه رسانه و ادیتور بلوکی گوتنبرگ – از نگاه کاربری و از نگاه دولوپری (چون همین محتواها بعداً از API می‌خوانیم).

۴.۱ – Posts در برابر Pages: فرق بنیادی

Page (برگه)	Post (نوشته)	
محتوای ثابت (درباره ما، تماس)	محتوای زمان‌دار (وبلاگ، خبر، مقاله)	ماهیت
اهمیتی ندارد	دارد ★	تاریخ انتشار
ندارد	دارد	دسته/برچسب
نمی‌آید	می‌آید	در فید/آرشیو
«منوی رستوران»	«تخفیف پاییزه اومد!»	مثال

در API هم جدا میان: `/wp/v2/posts` و `/wp/v2/pages` – وقتی به فصل ۱۴ برسی این تقسیم رو عیناً می‌بینی.

۴.۲ – اولین پست: گردش کامل

Posts → Add New و آشنایی با ادیتور بلوکی (Gutenberg):

- عنوان بالا، متن پایین – ولی متن از **بلوک‌ها** ساخته می‌شه
- Enter = بلوک جدید؛ / = منوی جستجوی بلوک (مثلاً `/code`، `/table`، `/image`)
- بلوک انتخاب‌شده = تنظیماتش در نوار راست (Sidebar) – رنگ، اندازه، align

پرکاربردترین بلوک‌ها:

بلوک	کار	معادل در ذهن تو
Paragraph / Heading	متن و تیتر	<code><p></code> / <code><h2></code>
Image / Gallery	تصویر / چندتایی	<code></code> / grid
List / Table / Quote	لیست / جدول / نقل قول	<code></code> / <code><table></code> / <code><blockquote></code>
Code / Embed	کد / ویدیو یوتیوب	<code><pre></code> / <code><iframe></code>
Columns	چند ستونه	flex/grid

دسته‌بندی و برچسب (پنل راست پایین): - **Categories** = ساختار درختی اصلی (مثلاً «آموزش» < «وردپرس») - یکی اجباری - **Tags** = برچسب‌های عرضی آزاد («تخفیف»، «جدید»)

Featured Image (تصویر شاخص): عکس اصلی پست - در API به صورت `_links` و با GraphQL عیناً میاد. برای لیست‌های Next.js حیاتیه؛ همیشه بذار.

بعد **Publish** - و در سایت (Open Site) ببینش.

۴.۳ - صفحات و اولین صفحه ثابت

یک Page بساز: «درباره ما» + چند بلوک. بعد یک صفحه «خانه» (Home) با چند بلوک معرفی.

حالا مهم‌ترین تنظیم این فصل:

Settings → Reading → Your homepage displays: بذار روی «Your latest posts» بمون (پیش‌فرض) - در Headless فرانت خودت صفحه اصلی رو می‌سازی، پس وردپرس فقط داده‌ها رو می‌ده.

۴.۴ - کتابخانه رسانه (Media Library)

Media → Add New: همه تصاویر/فایل‌ها اینجا جمع می‌شن، با:

- **Alt Text** ★ (متن جایگزین) - سئو و دسترس‌پذیری؛ همیشه پر کن (در API میاد: `alt_text`)
- Title/Caption/Description
- در Headless: URL عکس‌ها رو از API می‌گیری و در `next/image` می‌ذاری (فصل ۱۷) - پس اینجا فقط کیفیت و `alt` مهمه

⚠ نکته چاقی سایت: وردپرس از هر عکس چند سایز می‌سازه. عکس‌های سنگین (۵ مگ +) آپلود نکن؛ قبلش فشرده کن.

۴.۵ – آنچه پشت صحنه ذخیره شد (نگاه مهندسی)

نگاه سریع به دیتابیس (phpMyAdmin از Adminer/Studio → Database tab → Local):

- جدول `wp_posts`: همه پست‌ها، صفحات و حتی revision ها (با ستون `post_type`)
- جدول `wp_term_relationships` + `wp_terms`: دسته‌ها و برچسب‌ها
- جدول `wp_users`: کاربران

🔑 «همه‌چیز post است» – فلسفه وردپرس. حتی پیوست‌ها (attachment) و صفحه‌های ثابت، `post_type` های متفاوت از یک جدول‌ان. در فصل ۱۰ خودت `post_type` جدید می‌سازی (مثلاً «محصول») و همین ساختار برایش ساخته می‌شه.

✓ جمع‌بندی فصل

- Post = محتوای زمان‌دار با دسته/برچسب؛ Page = ثابت – در API جدا هستند
- ادیتور بلوکی: / = جستجوی بلوک؛ بلوک = کامپوننت؛ تنظیمات در Sidebar
- Featured Image همیشه بگذار – سوخت لیست‌های Next.js
- Media: Alt Text اجباری؛ عکس سنگین آپلود نکن
- زیر hood: همه‌چیز در `wp_posts` با `post_type`

📝 تمرین فصل ۴

۱. ۵ پست وبلاگی (با موضوع دلخواه: مثلاً معرفی ابزارهای دولوپری) با ۲-۳ دسته، برچسب، عکس شاخص و حداقل ۴ نوع بلوک مختلف بنویس – این داده‌ها سوخت پروژه Headless فصل ۱۶ است، با کیفیت بنویس!
۲. دو صفحه «درباره ما» و «تماس با ما» بساز.
۳. در phpMyAdmin جدول `wp_posts` را باز کن و ستون `post_type` را بین – چه مقادیری دیدی؟ (`page`, `post`, `attachment`, `revision` ...)
۴. (کشف) در ادیتور بلوکی، :: کنار هر بلوک را بزن – می‌تونی بلوک را جابجا/تکرار کنی. و `⌘/Ctrl+Z` هم کار می‌کند 😊

▼ جواب تمرین ۳

مقادیر رایج: `post` (نوشته‌ها)، `page` (برگه‌ها)، `attachment` (رسانه‌ها)، `revision` (نسخه‌های خودکار)، `nav_menu_item` (آیتم‌های منو) – و هر CPT که بسازی (فصل ۱۰) اینجا ستون می‌شود.

➔ فصل بعد: تم‌ها – ظاهر سایت، Site Editor و مفهوم تم فرزند.

فصل ۵ – تم‌ها: ظاهر سایت، Site Editor و تم فرزند

🎯 هدف: اکوسیستم تم‌ها رو بشناسی، یک تم نصب و شخصی‌سازی کنی، و مهم‌ترین مفهوم توسعه‌ای: تم فرزند (Child Theme). برای Headless تم کمتر مهمه – ولی فهم تم = فهم قلب وردپرسه، و در پروژه‌های Classic هم نیازت می‌شه.

۵.۱ – تم: قالب نمایش محتوا

همون محتوا (فصل ۴) با تم‌های مختلف، ظاهرهای کاملاً متفاوت می‌گیره. دو نسل تم:

نسل	چیه	شخصی‌سازی
Classic Themes	تم‌های PHP سنتی (...Twenty Twenty-One)	Customizer (پنل قدیمی)
Block Themes ★	تم‌های بلوکی مدرن – کل قالب از بلوک‌ها ساخته شده	Site Editor (ادیتور همان گوتنبرگ برای کل سایت!)

وردپرس 6.9 به سمت Block Themes رفته و تم پیش‌فرض‌های جدید (Twenty Twenty-Five/Six) همگی Block Theme اند.

۵.۲ – نصب و تست تم

Themes → Add New → Appearance → جستجو/نصب رایگان از مخزن وردپرس → **Activate**.

تم‌های خوب رایگان برای شروع: **Twenty Twenty-Five** (پیش‌فرض)، **Astra**، **GeneratePress** (دوومی سبک و محبوب برای Classic).

بعد از Activate، سایت رو رفرش کن – همین! ظاهر عوض شد، محتوا همونه. (همون جداسازی data/view که تو در React با props/children می‌بینی.)

۵.۳ – Site Editor: طراحی با بلوک‌ها

Appearance → Editor (در Block Themes): کل سایت قابل ویرایش با بلوک‌ها:

- **Templates**: قالب‌های صفحه (صفحه اصلی، تک‌پست، آرشیو...) – ساختار صفحه
- **Styles**: رنگ‌ها و تایپوگرافی سراسری (مثل design tokens!)

• **Navigation / Header / Footer**: به صورت بخش‌های template part

تمرین: در Styles یک رنگ اصلی و فونت عوض کن → Save → سایت رو ببین.

💡 برای تو: Visual Studio Code = Site Editor برای HTML/CSS به زبان بلوکی. و Templates دقیقاً همون layout های Next.js ان (`app/(shop)/layout.tsx` فکر کن) – فقط با بلوک.

۵.۴ – Customizer (برای تم‌های کلاسیک)

Appearance → Customize: پنل شخصی‌سازی تم‌های قدیمی (لوگو، رنگ‌ها، منوها). اگر تم Classic نصب کردی اینو می‌بینی؛ برای Block Themes منسوخه. فقط بدون وجود داره.

۵.۵ – منوها (Navigation)

Appearance → Menus (یا در Site Editor، بلوک Navigation): منوی بالای سایت رو با صفحات/دسته‌ها بساز. در Headless از API خونده می‌شه (`wp/v2/menu-items` یا GraphQL) – پس اینجا منو رو درست بساز که داده‌اش تمیز باشه.

۵.۶ – Child Theme: قانون طلایی توسعه ⭐

هیچ‌وقت فایل‌های تم اصلی رو مستقیم ویرایش نکن – با اولین آپدیت تم، همه تغییرات می‌پره!
راه‌حل: **تم فرزند** – تمی که «از تم مادر ارث می‌بره» و فقط چیزهای تغییر یافته رو override می‌کنه:

```
wp-content/themes/
├── twentytwentyfive/
│   └── twentytwentyfive-child/
│       ├── style.css
│       └── functions.php
```

تم مادر (دست نمی‌زنیم – آپدیت می‌شود) ←
تم فرزند (کد تو) ←
هدر اجباری + استایل‌ها ←
تو PHP کدهای ←

تم فرزند فقط یک هدر کامنت لازم داره: `style.css`

```
/*
Theme Name: Twenty Twenty-Five Child
Template: twentytwentyfive ← ⭐ رابطه فرزندى
*/
```

فعلاً فقط مفهومش رو بگیر: **تغییرات تو همیشه در لایه‌ای جدا از هسته/تم اصلی** – همون فلسفه‌ای که در کد هم داری (قبل از override کردن library، fork کن!).

🔦 ساخت سریع با WP-CLI (فصل ۳): `wp scaffold child-theme twentytwentyfive-child --parent=twentytwentyfive`

۵.۷ – تم‌های premium و Page Builder ها (آگاهی)

در بازار ایران: تم‌های premium (مثل Enfold، Flatsome) و **Page Builder** ها (Elementor، WPBakery) – ابزارهای درگ‌اند‌دراپ صفحه‌سازی. بدون که: – Elementor محبوب‌ترین و بسیاری از پروژه‌های ایرانی باهاش ساخته می‌شن - ولی برای Headless به درد نمی‌خورن (خروجی‌شون HTML تمپلیت است نه داده ساخت‌یافته) – Headless مسیر خودش رو داره: CPT + ACF + API (فصل‌های ۱۰ به بعد)

✅ جمع‌بندی فصل

- دو نسل تم: Classic (Customizer) و **Block (Site Editor)** – آینده با Block
- Site Editor = ویرایش کل سایت با بلوک (Templates + Styles)
- منوها را تمیز بساز – در Headless از API می‌آیند
- **تم فرزند = لایه تغییرات جدا از تم اصلی** – هرگز فایل تم اصلی را ویرایش نکن
- Page Builder ها (Elementor و...) برای Headless به درد نمی‌خورند

📝 تمرین فصل ۵

1. دو تم نصب و فعال کن و فرق ظاهر را ببین؛ بعد Twenty Twenty-Five را برگردان.
2. در Site Editor: رنگ اصلی سایت و فونت را عوض کن و Save – سایت را ببین.
3. منوی بالای سایت را با صفحات فصل ۴ (درباره ما، تماس) و یک دسته‌بندی بساز.
4. با WP-CLI یک تم فرزند بساز و فعالش کن (`wp theme activate ...` + `wp scaffold child-theme ...`) – سایت همچنان کار می‌کند؟
5. (کشف) در Site Editor یک Template را باز کن (مثلاً Single) و ساختار بلوکی‌اش را ببین – این همان «layout» است.

```
# در Open Site Shell:  
wp scaffold child-theme tt5-child --parent=twentytwentyfive  
wp theme activate tt5-child  
wp theme list --status=active
```

اگر پوشه تم مادر اسم دیگری دارد، `--parent` را با همان نام پوشه بده.

➔ **فصل بعد:** صفحه‌سازی عملی – هدر، فوتر، منو، مگامنو و Elementor؛ یک سایت شرکتی کامل می‌سازیم.

فصل ۶ – صفحه‌سازی عملی: هدر، فوتر، منو، مگامنو و Elementor

🎯 هدف: از «تنظیمات» به «طراحی» می‌رویم. یک سایت شرکتی واقعی می‌سازیم: هدر و فوتر حرفه‌ای، منو و مگامنو، صفحه اصلی با باکس‌های آماری و قیمت، اسلایدر و فرم – هم با Site Editor بلوکی و هم با Elementor. این فصل یعنی: بتوانی ظاهر کاری مشتری را خودت از صفر بسازی.

۶.۱ – دو مسیر صفحه‌سازی: Site Editor در برابر Elementor

همان بحث فصل ۵، ولی این بار عملی. دو ابزار، دو فلسفه:

Elementor	Site Editor (بلوکی)	
پلاگین صفحه‌ساز درگ‌اند‌درپ	ادیتور بلوکی برای کل سایت – بخشی از خود وردپرس	چیه
Section/Column/Widget با تنظیمات زنده	بلوک + استایل‌های سراسری	سبک کار
سنگین‌تر (پلاگین + فایل‌های CSS/JS زیاد)	سبک (هسته وردپرس)	وزن
رایگان + نسخه Pro پولی	رایگان (نسخه Pro تم‌ها جدا)	هزینه
محبوب بازار ایران و تم‌های premium	★ مسیر رسمی وردپرس	آینده
خروجی HTML سنگین – برای Headless غیرقابل استفاده (فصل ۱۰ به بعد)	بلاک‌ها → HTML در <code>content.rendered</code>	در Headless

🌟 **قاعده تصمیم**: مشتری ایرانی که «خودش همه‌چیز را بکشد و رها کند» می‌خواهد → Elementor. پروژه‌ای که تو فرانت آن را با Next.js می‌سازی → این فصل را با Site Editor بخوان و بخش Elementor را فقط برای شناخت بازار. هر دو را اینجا تمرین می‌کنیم.

💡 اگر تم‌های premium ایرانی (مثل قالب‌های فروشگاه‌های رایج بازار) دیدی، بدان اکثراً با WPBakery (ویژوال کامپوزر) یا Elementor ساخته شده‌اند – ردیف/ستون/ویجت همان الگوی این فصل است.

۶.۲ – پروژه این فصل: سایت شرکتی «برق آسا»

تمرین‌های فصل روی یک نمونه واقعی است: سایت شرکت خدمات برق صنعتی با ۵ صفحه:

خانه | درباره ما | خدمات | نمونه کارها | تماس با ما

صفحات را در **Add New** → **Pages** بساز (فصل ۴) – محتوای هر صفحه را پایین‌تر طراحی می‌کنیم.

۶.۳ – هدر و فوتر: چهره سایت

Header (Header) = نوار بالای همه صفحات: لوگو + منو + دکمه CTA («تماس بگیرید»). **فوتر (Footer)** = پایین همه صفحات: آدرس/تلفن، لینک‌های سریع، شبکه‌های اجتماعی، کپی‌رایت.

با **Site Editor (تم بلوکی)**: Editor → Appearance → روی **Header** و **Footer** (Template Parts) کلیک کن:

- لوگو: بلوک **Site Logo** (از Settings → General لوگو آپلود کن)
- منو: بلوک **Navigation** (بند ۶.۴)
- شماره تلفن/ساعت کاری: بلوک Paragraph با آیکون (فصل ۸: ستون‌ها)
- فوتر: بلوک Paragraph + **Site Title** + بلوک **Social Icons**

با **Elementor**: پلاگین **Elementor + Elementor Header & Footer Builder** را نصب کن (فصل ۷: نصب پلاگین) → **Header/Footer** → Theme Builder → Templates جدید → با ویجت‌ها طراحی **Condition: «Entire Site»**.

⚠️ قانون طراحی هدر: حداکثر ۷ آیتم منو، لوگو چپ (در RTL راست)، دکمه تماس برجسته با رنگ اصلی سایت. هدر شلوغ = کاربر گم.

۶.۴ – منو و مگامنو

منوی ساده: Appearance → Menus (تم کلاسیک) یا مستقیم بلوک Navigation (تم بلوکی):

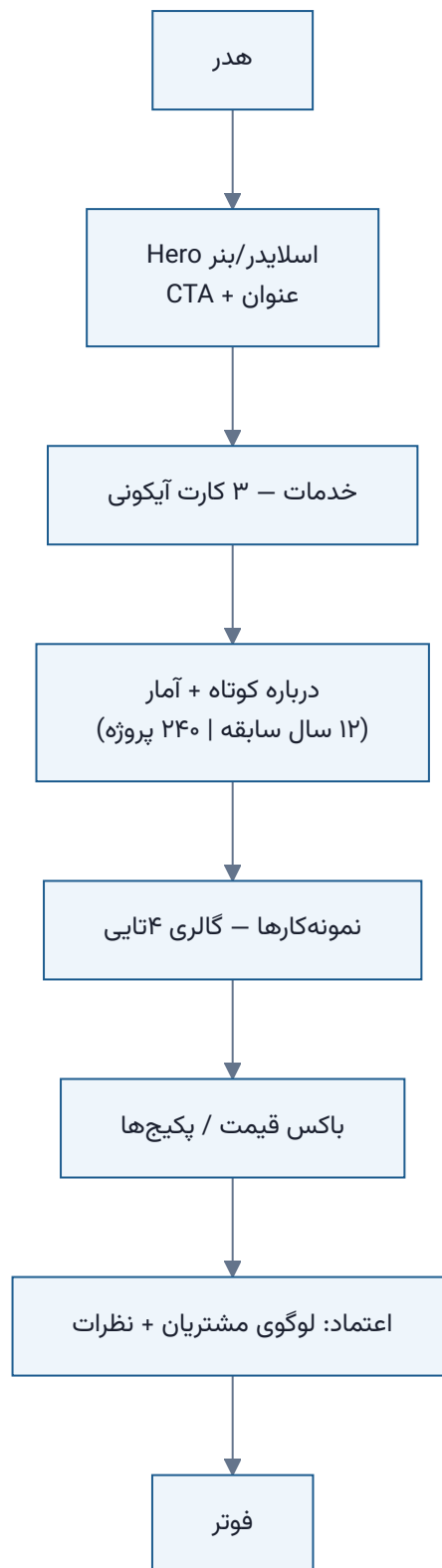
1. منوی «Main Menu» بساز
 2. صفحات ۶.۲ + دسته‌بندی بلاگ را اضافه کن
 3. زیرمنو: آیتم را drag کن زیر آیتم دیگر (کلاس submenu در ذهن تو)
 4. در تم‌های بلوکی جای نمایش منو = Header؛ در کلاسیک: Location → Primary
- مگامنو (Mega Menu)** = زیرمنوی پهن چندستونه (مثل منوی «خدمات» در سایت‌های بزرگ که ستون‌بندی شده باز می‌شود):

- در تم‌های premium و تم‌های ووکامرسی معمولاً قابلیت مگامنو داخلی است (تنظیمات تم)

- در Site Editor: به جای منوی معمولی، یک Template Part جدا بساز که فقط در hover منو نمایش داده شود (الگوی رایج: منوی ساده + صفحه لندینگ جدا برای هر دسته)
- با Elementor Pro: ویجت Mega Menu آماده دارد
- ★ قانون محتوایی: مگامنو را با صفحات/لینک‌های واقعی پر کن نه «به‌زودی» – منوی خالی یعنی کاربر خالی

۶.۵ – صفحه اصلی: Section ها، ردیف و ستون

صفحه اصلی سایت شرکتی، الگوی استاندارد بازار:



با بلوک‌ها: بلوک **Columns** برای کارت‌ها، **Cover** برای Hero (عکس پس‌زمینه + متن رویش)، **Group** برای باکس‌ها - padding و margin را در Sidebar هر بلوک تنظیم کن (dimensions = همان box model که بلدی).

با **Elementor**: ساختار **Widget** → **Column** → **Section**: - ردیف (Section) = یک سطر افقی؛ ستون‌ها داخلش؛ ویجت‌ها داخل ستون (Heading, Image, Button, Icon Box, ...) - هر عنصر سه تب تنظیم دارد: **Content** (محتوا) /

Style (رنگ، فونت) / **Advanced** (فاصله، سایه، مارجین) - **Responsive**: آیکون موبایل کنار هر تنظیم – جدا برای دسکتاپ/تبلت/موبایل ★ (مثلاً در موبایل فونت H1 را کوچک کن)

💡 مفاهیم padding/margin و فرقشان برای تو دوغ است؛ نکته وردپرسی: در Elementor از پدینگ Section برای «ارتفاع‌دهی» استفاده می‌شود (مثلاً 80px بالا/پایین) – الگوی همه قالب‌های بازار.

۶.۶ – جعبه ابزار باکس‌های پرکاربرد

هرچه در سایت‌های ایرانی می‌بینی، این‌هاست:

باکس	با بلوک	با Elementor
اسلایدر	بلوک Slides (تم‌های بلوکی) یا پلاگین Slides	ویجت Image Carousel / Slides
باکس قیمت	Group + Columns + Button	ویجت Price Table (در Pro) یا Column با استایل
باکس جستجو	بلوک Search	ویجت Search
ویدیو	بلوک Video (آپلودی) / Embed (آپارات/یوتیوب)	ویجت Video
موزیک/پادکست	بلوک Audio	ویجت Audio
آیکون‌ها	بلوک Social Icons	ویجت Icon / Icon Box
شمارنده آمار	– (بلوک آماده ندارد)	ویجت Counter
نقشه	Embed نقشه	ویجت Google Maps (فصل ۸: نشان/بلد)

افکت‌ها: **سایه** (Box Shadow در Style/Advanced)، **hover** (تب Hover در Elementor؛ در بلوک‌ها فقط استایل ثابت) و **پس‌زمینه** (رنگ/عکس/گرادین در هر Section).

⚠️ کم‌زور کن با سایه و انیمیشن – «طراحی خفن» بازار ایران یعنی: سفید/فضای خالی/یک رنگ اصلی/عکس باکیفیت. اسلایدر ۵ اسلایدی با ۷ انیمیشن = طراحی ۱۳۹۵!

۶.۷ – آیکون سایت (Favicon) و لوگو

آیکون کوچک کنار تب مرورگر (لوگوی کوچک!):

۱. عکس مربع ۵۱۲×۵۱۲ بساز (PNG با پس‌زمینه شفاف)

۲. **Site Identity** → **(:)** → **Styles** → **Editor** → **Appearance** (یا Customizer در تم کلاسیک)

3. **Site Icon** → آپلود – وردپرس همه سایزها را خودش می‌سازد (در API هم `site_icon` می‌آید)

4. همان‌جا **Site Logo** را هم ست کن (هدر از آن می‌خواند)

۶.۸ – فرم تماس و سایدبار

فرم تماس: پلاگین **Contact Form 7** یا **Fluent Forms** (فصل ۷) → شورت‌کد/بلوک فرم را در صفحه «تماس با ما» بگذار
→ ایمیل‌های فرم را با SMTP وصل کن (فصل ۱۳ – بدون SMTP ایمیل‌ها گم می‌شوند ⚠️). فیلدهای استاندارد: نام،
تلفن، موضوع، پیام + تیک رضایت حریم خصوصی (صفحه policy فصل ۹).
سایدبار (Sidebar): در تم‌های کلاسیک: Widgets → Appearance (ابزارک‌ها) – جستجو، پست‌های اخیر، دسته‌ها. در
تم‌های بلوکی سایدبار هم Template Part است. در طراحی مدرن (و Headless) معمولاً بدون سایدبار هستیم – فقط
این مفهوم را بشناس چون قالب‌های قدیمی ایرانی پر از آن است.

۶.۹ – کاهش حجم تصاویر: بدترین دشمن سرعت

سایت‌های صفحه‌سازی شده معمولاً از عکس‌های سنگین می‌میرند (فصل ۴: Media). قواعد:

- قبل از آپلود: عکس‌ها را فشرده کن – **squoosh.app** (رایگان، مرورگری، ساخت گوگل) یا **tinypng.com**
- اندازه واقعی نمایش: کارت ۴۰۰px عرض لازم دارد، نه عکس ۴۰۰۰ پیکسلی آیفون!
- فرمت: **WebP** برای همه عکس‌ها (سایز ~۳۰٪ JPEG)؛ PNG فقط برای لوگو/شفاف
- پلاگین خودکار: **Smush** یا **ShortPixel** – همه آپلودی‌ها را فشرده و WebP می‌کند (فصل ۷)

🚀
لود ۸ ثانیه در ۴ → hero.jpg 4.2MB
بعد: hero.webp 180KB لود ۰.۵ ثانیه

۶.۱۰ – تصمیم نهایی: Elementor یا بلوک یا Headless؟

سناریوی مشتری	راه تو
«سایت شرکتی ساده، خودم می‌خواهم ادیت کنم، بودجه کم»	تم بلوکی + Site Editor (یا Elementor اگر تم premium خواست)
«فروشگاه می‌خواهم»	ووکامرس + تم فروشگاه‌های (فصل ۱۲)
«سایت جدی/برند قوی/پرفورمنس مهم است»	🌟 Headless : وردپرس فقط پتل، فرانت Next.js – و همه‌چیز این فصل در فرانت با کد ساخته می‌شود (فصل ۱۶)

🔑 نکته فروش (فصل ۱۹): Elementor سرعت تحویل بالا می‌دهد ولی چون هر کسی بلدش است، قیمتش پایین است. Headless تخصص کمیاب است – قیمتش ۳-۵ برابر است. هر دو را بلد باش، بر اساس بودجه انتخاب کن.

جمع‌بندی فصل ✓

- هدر/فوتر = Template Part در Site Editor یا Theme Builder در Elementor
- منو با Navigation/Menus؛ مگامنو = زیرمنوی چندستونه (تنظیمات تم یا Elementor Pro)
- صفحه اصلی = الگوی Hero → خدمات → آمار → نمونه‌کار → قیمت → اعتماد
- Elementor: Section/Column/Widget با سه تب Content/Style/Advanced + تنظیم Responsive جدا
- Site Icon در Favicon؛ فرم تماس = پلاگین + SMTP
- عکس فشرده/WebP قبل از آپلود – سرعت = سئو = پول

تمرین فصل ۶ (پروژه: سایت «برق‌آسا») 📄

۱. ۵ صفحه پروژه را بساز و یک منوی «Main Menu» با زیرمنو (خدمات → سه زیرسرویس) بساز و در هدر بگذار.
۲. در Site Editor هدر و فوتر کامل بساز: لوگو + منو + دکمه تماس / فوتر با ۴ ستون (درباره، لینک‌ها، تماس، شبکه‌های اجتماعی).
۳. صفحه خانه را با الگوی ۶.۵ بساز: Cover برای Hero + Columns برای ۳ کارت خدمات + باکس آمار.
۴. آیکون سایت (Favicon) را ست کن – تب مرورگر را ببین.
۵. با Elementor (رایگان) یک صفحه «تماس با ما» جایگزین بساز: فرم + نقشه + اطلاعات – مقایسه کن با روش بلوکی.
۶. سه عکس Hero را در squoosh.app فشرده و به WebP تبدیل کن – حجم قبل/بعد را یادداشت کن و در رسانه آپلود کن.

▼ نکته تمرین ۳

Hero با بلوک Cover: عکس پس‌زمینه + Overlay (کدرنگ) + درونش Heading و Button. تنظیم کلیدی: حداقل ارتفاع (min-height) ۶۰vh در Sidebar → چشم‌نواز می‌شود. برای کارت‌های خدمات: یک Columns سه‌ستونه؛ در هر ستون: Image + Heading + Paragraph + Button.

➡ فصل بعد: پلاگین‌ها – قابلیت‌های اضافه و ضروری‌ها؛ همان‌هایی که در این فصل پیش‌نمایششان را دیدی.

فصل ۷ – پلاگین‌ها: قابلیت‌های اضافه و ضروری‌ها

🎯 هدف: اکوسیستم پلاگین (npm دنیای وردپرس) – چطور نصب/غیرفعال/حذف کنی، چه پلاگین‌هایی برای هر پروژه ضروریه، و زیر hood یک پلاگین چیه. آخر فصل، عادت‌های امنیتی نصب رو هم داری.

۷.۱ – پلاگین = npm package وردپرس

هر قابلیت‌ای که در هسته نیست: فروشگاه (WooCommerce)، فرم تماس، SEO، مولتی‌زبان، کش... همه پلاگین‌اند:

وردپرس	npm	
premium + wordpress.org/plugins بازارهای	npmjs.com	مخزن
Plugins → Add New → Install	<code>npm install x</code>	نصب
دکمه Activate ⚠️	import در کد	فعال‌سازی
Deactivate → Delete	<code>npm uninstall</code>	حذف

⚠️ تفاوت مهم: پلاگین نصب‌شده ≠ فعال. Deactivated فقط خاموشه (داده‌اش می‌مونه)؛ Delete واقعاً پاکش می‌کنه.

سه راه نصب: 1. **Add New** از مخزن رسمی (رایگان) – استاندارد 2. **Upload Plugin** (فایل zip پولی/اختصاصی) 3. WP-CLI: `wp plugin install woocommerce --activate` ⭐ (سریع‌ترین – از دوره CLI بلدی!)

۷.۲ – پلاگین‌های ضروری هر پروژه (چک‌لیست)

دسته	پلاگین پیشنهادی	کار
SEO	Yoast SEO یا Rank Math	متا تگ‌ها، sitemap، اسکیمای – (فصل ۱۷: خروجیش در Next.js مصرف می‌شه)
امنیت	Wordfence یا Solid Security	فایروال، اسکن، محدود کردن لاگین
بکاپ	UpdraftPlus	بکاپ خودکار زمان‌بندی‌شده (فایل + دیتابیس)
کش/سرعت	LiteSpeed Cache یا W3 Total Cache	کش صفحات (در Headless نقشش کمتر می‌شه)
فرم	Contact Form 7 یا Fluent Forms	فرم تماس (در Headless: فصل ۱۷ راه مدرن)
اسپم	Akismet (پیش‌فرض نصبه)	ضد اسپم کامنت

💡 قانون طلایی پلاگین: کمتر = بهتر. هر پلاگین کد PHP اجرا می‌کنه – پلاگین زیاد یعنی کندی و سطح حمله بیشتر. فقط ضروری‌ها. (مثل بوت‌استرپ + ۲۰ تا UI library روی هم، ولی سنگین‌تر!)

۷.۳ – نصب اولین پلاگین: Yoast SEO

1. Yoast SEO → Install → **Activate** جستجوی Plugins → Add New
2. منوی جدید Yoast SEO ظاهر می‌شه – Wizard اولیه رو رد کن (پیش‌فرض‌ها اوکیه)
3. حالا در صفحه ویرایش هر پست، یک باکس Yoast پایین اضافه شده: عنوان و توضیحات متا + پیش‌نمایش گوگل
4. در فصل ۱۷ این مقادیر متا رو از API می‌خونیم و در Next.js `metadata` می‌ذاریم – پس همین الان برای پست‌های فصل ۴، عنوان سئو بذار!

۷.۴ – زیر hood یک پلاگین (نگاه مهندسی)

هر پلاگین = یک پوشه در `wp-content/plugins/` با حداقل یک فایل PHP که هدر مخصوص داره:

```
<?php
/**
 * Plugin Name: My First Plugin
 * Description: سلام از طرف من!
 * Version: 1.0.0
 * Author: Mahdi
 */
```

همین! فایل PHP با این هدر = پلاگین قابل فعال‌سازی. (در فصل ۱۱ اولین پلاگین واقعی خودت رو می‌نویسی.)

ببین پلاگین‌های نصب‌شده رو: Local → پوشه سایت → `wp-content/plugins/` — هر پوشه، یک پلاگین.

۷.۵ — عادات‌های امنیتی و نگهداری

- پلاگین را فقط از مخزن رسمی یا منبع معتبر نصب کن (پلاگین نال‌شده = اسب تروا!)
- آپدیت‌ها را جدی بگیر — اکثر هک‌های وردپرسی از پلاگین‌های قدیمی (Plugins → Updates)
- پلاگین غیرفعال بلااستفاده = حذفش کن
- قبل از هر تغییر بزرگ: Snapshot در Local (فصل ۳)
- تعداد پلاگین فعال را زیر ۱۰ نگه دار

و WP-CLI برای نگهداری سریع:

```
wp plugin list           # وضعیت همه
wp plugin update --all   # آپدیت همه
wp plugin deactivate akismet # غیرفعال
wp plugin delete akismet  # حذف کامل
```

جمع‌بندی فصل

- پلاگین = قابلیت آماده؛ `Install ≠ Activate`؛ `wp plugin install x --activate`
- ضروری‌ها: (Yoast) SEO، امنیت، بکاپ، کش — و قانون کمتر = بهتر
- زیر hood: پوشه PHP با هدر `Plugin Name`
- امنیت: منبع معتبر، آپدیت منظم، حذف بی‌استفاده‌ها، Snapshot قبل از تغییر

تمرین فصل ۷

1. Yoast SEO را نصب و فعال کن و برای سه پست فصل ۴، عنوان و توضیح سئو بنویس.
2. UpdraftPlus را نصب و یک بکاپ دستی بگیر — فایل‌هاش را پیدا کن.
3. با `wp plugin list` WP-CLI: بزن؛ بعد با CLI یک پلاگین نصب و فعال کن (مثلاً `contact-form-7 --activate`) و در پنل ببینش.
4. پلاگینی را غیرفعال کن و سایت را ببین (چه چیزی غیب شد؟)؛ بعد بکش `up`.
5. (کشف) پوشه `wp-content/plugins/` را در فایل اکسپلورر باز کن — چند پلاگین چند فایل رو ببین (ووکامرس را حتماً ببین اگر نصب کردی — ۳۰۰+ فایل!).

▼ نکته تمرین ۲

UpdraftPlus → Settings → Backup Now. فایل‌ها در `wp-content/updraft/` ذخیره می‌شوند (و می‌توانی به گوگل‌درایو وصلش کنی). برای پروژه‌های واقعی: بکاپ خودکار هفتگی = بیمه شغلی.

➔ **فصل بعد:** سئو عملی — تا سایت در گوگل دیده شود؛ با همان Yoast که نصب کردی.

فصل ۸ – سئو عملی: دیده‌شدن در گوگل

🎯 هدف: سایت خوب ساختن کافی نیست؛ باید پیدا شود. در این فصل سئوی عملی وردپرس را یاد می‌گیری: تنظیم Yoast/Rank Math، ساختار محتوای درست، گوگل سرچ کنسول، ثبت در نقشه‌ها (گوگل/نشان/بلد) و سئوی محلی برای کسب‌وکارهای ایرانی. در آخر، پل سئو به دنیای Headless.

۸.۱ – گوگل چگونه فکر می‌کند؟

سه مرحله برای هر سایت:



سئو = کار کردن روی هر سه لایه. عوامل مهم رتبه (به ترتیب اثر واقعی):

عامل	در وردپرس کجاست
محتوای مفید و متصل (لینک داخلی)	پست‌ها و صفحات (فصل ۴)
عنوان و توضیح متا ★	پلاگین SEO (این فصل)
سرعت سایت	کش/تصویر (فصل ۶ و ۱۳)
موبایل‌فرندلی	تم responsive (فصل ۵ و ۶)
SSL	فصل ۲ و ۱۸
ساختار URL تمیز	Permalink → Post name (فصل ۹ – اگر نرفتی، برو!)

۸.۲ – پلاگین سئو: Yoast یا Rank Math

Yoast را فصل ۷ نصب کردی. جایگزین محبوب: **Rank Math** (رایگان با امکانات بیشتر). هر دو کار را می‌کنند – یکی را انتخاب کن:

در صفحه ویرایش هر پست/صفحه، باکس Yoast پایین‌تر اضافه شده:

1. **Focus Keyphrase** (کلمه کلیدی): مثلاً «تابلو برق صنعتی» – Yoast تحلیل می‌کند که چقدر در محتوا رعایت شده (سبز/نارنجی/قرمز)

2. **SEO Title** (عنوان متا): ★ مهم‌ترین تگ سئو – الگو: **کلمه کلیدی | برند** – زیر ۶۰ کاراکتر (چون گوگل برش می‌زند)

3. **Meta Description** (توضیح متا): ۱۵۰~ کاراکتر، تبلیغی و کلیک‌خور – نه کپی متن پست!

4. **Slug**: کوتاه و با کلمه کلیدی (`/tablo-bergh-sanati/`)

پیش‌نمایش گوگل همان‌جا است – قبل از انتشار ببین در نتایج چطور دیده می‌شوی.

تنظیمات سراسری Yoast (منوی Yoast SEO):

- **General → Features**: همه پیش‌فرض خوب است؛ XML sitemaps روشن ☒
- **Search Appearance**: عنوان/توضیح پیش‌فرض برای هر نوع محتوا + علامت برند
- **شمای سازمانی (Schema)**: در Rank Math/Yoast، نوع کسب‌وکار را ست کن

۸.۳ – sitemap و گوگل سرچ کنسول

Sitemap = فهرست XML همه صفحات سایت برای گوگل: `yoursite.ir/sitemap_index.xml` (Yoast خودش می‌سازد).

★ **Google Search Console (GSC)** – `search.google.com/search-console` – پنل رسمی گوگل برای وبمسترها؛ رایگان و حیاتی:

1. **Add Property** → دامنه یا URL prefix (با DNS یا فایل HTML تأیید کن – همان رکورد TXT فصل ۲!)

2. **Sitemaps** → آدرس sitemap را سابمیت کن

3. **URL Inspection**: آدرس هر صفحه را بزن → «Request Indexing» برای ایندکس سریع (به‌جای انتظار هفته‌ای)

4. بعد از چند روز: گزارش Performance (چه کلماتی آوردت؟) و Coverage (کدام صفحات ارور ۴۰۴ redirect دارند؟)

🔑 **GSC = آنالوژی Lighthouse در مرورگر**: همان چیزی که برای Next.js به آن نگاه می‌کنی، برای وردپرس هم همین است – ارورها را خود گوگل می‌گوید!

۸.۴ – ساختار محتوا: قواعد صفحه‌ای

هر صفحه/پست برای سئوی داخلی:

- **فقط یک H1** (عنوان صفحه) – تیترها با H2/H3 سلسله‌مراتبی (بلوک Heading – فصل ۴)
- **کلمه کلیدی** در: H1، اولین پاراگراف، حداقل یک alt، H2، یک عکس، و slug

- **لینک داخلی:** ★ هر پست به ۲-۳ صفحه دیگر خود سایت لینک بدهد (Yoast همین را یادآوری می‌کند) – لینک داخلی = خون‌رسانی سئو
- **Alt تصویر:** توصیفی و با کلمه کلیدی طبیعی (فصل ۴: در API هم می‌آید)
- محتوای حداقلی: صفحات موروثی «درباره ما» ۳ خطی رتبه نمی‌گیرند – ۴۰۰+ کلمه با محتوای واقعی

۸.۵ – سئوی محلی: گوگل مپ، نشان و بلد IR

برای کسب‌وکارهای محلی (رستوران، خدماتی، فروشگاه فیزیکی) – نیمی از بازار کار تو:

سرویس	کجا	نکته
Google Business Profile	business.google.com	رایگان – ثبت با آدرس دقیق + تلفن + ساعت کاری + عکس؛ روی نقشه گوگل می‌افتد و در جستجوهای محلی («تابلو برق نزدیک من») ظاهر می‌شوید
نشان (neshan.org)	پنل کسب‌وکار	نقشه محبوب ایرانی – اپ‌های داخلی از API آن استفاده می‌کنند
بلد (balad.ir)	پنل در بلد	همان نقش؛ هر دو ثبت کن – هزینه صفر

در سایت هم نقشه بگذار: بلوک Embed با آدرس iframe نقشه، یا ویجت Google Maps در Elementor (فصل ۶). برای سایت‌های Headless: کامپوننت نقشه نشان/لفت‌لت در Next.js (فصل ۱۶ به بعد).

💡 **NAP (Name, Address, Phone):** نام/آدرس/تلفن کسب‌وکار باید دقیقاً یکسان باشد در: سایت، گوگل، نشان، بلد و شبکه‌های اجتماعی – ناسازگاری = کاهش اعتماد گوگل به کسب‌وکار.

۸.۶ – سرعت = سئو: Core Web Vitals

گوگل سرعت را مستقیم در رتبه می‌بیند. ابزار سنجش: **PageSpeed Insights** (pagespeed.web.dev) – آدرس سایت را بده:

- **LCP** (بزرگ‌ترین عنصر دیده‌شده): باید $> ۲.۵s$ – معمولاً عکس Hero است (فصل ۶: فشرده‌سازی!)
- **CLS** (پرش چیدمان): > ۰.۱ – عکس‌ها همیشه width/height داشته باشند
- **INP** (سرعت واکنش): $> ۲۰۰ms$ – پلاگین‌های سنگین را کم کن (فصل ۷: کمتر = بهتر)

پلاگین کش (LiteSpeed Cache – فصل ۱۳) + تصاویر WebP معمولاً نمره موبایل وردپرس را از قرمز به سبز می‌برد. در Headless، این بخش را Next.js برای حل کرده ★ (SSR/ISR + next/image) – یکی از دلایلی که Headless می‌فروشیم.

۸.۷ – سئو در Headless: پل به فصل ۱۷

وقتی فرانت Next.js باشد، مقادیر Yoast همان‌جا مصرف می‌شوند:

```
query {
  post(id: "my-post", idType: SLUG) {
    title
    # پلاگین با WPGraphQL Yoast SEO:
    seo {
      title      # عنوان متای Yoast
      metaDesc   # توضیح متا
      opengraphImage { sourceUrl }
    }
  }
}
```

و در Next.js با `generateMetadata` به تگ‌های `<head>` تبدیل می‌شود (فصل ۱۷ کامل می‌سازیم). یعنی: مشتری همان‌جا در پنل وردپرس سئو می‌نویسد، سایت Next.js همان را نمایش می‌دهد – سئو از پنل، رندر از فرانت مدرن.

✓ جمع‌بندی فصل

- سئو = Crawl + Index + Rank؛ کار روی محتوا، متا، سرعت و اعتبار
- Yoast/Rank Math: کلمه کلیدی + عنوان/توضیح متا در هر پست؛ sitemap خودکار
- GSC را حتماً راه بینداز: سابمیت Request Indexing + sitemap + گزارش‌ها
- قواعد صفحه: یک H1، کلمه کلیدی طبیعی، لینک داخلی، alt تصویر
- سئوی محلی ایران: Google Business + نشان + بلد با NAP یکسان
- سرعت: PageSpeed + کش + WebP – در Headless، Next.js این را می‌تراشد

📝 تمرین فصل ۸

1. برای ۳ پست فصل ۴ و صفحه «خدمات» فصل ۶: کلمه کلیدی + SEO Title + توضیح متا بنویس و چراغ Yoast را سبز کن (همه موارد نه – موارد قرمز کافی است).
2. `wp-course.local/sitemap_index.xml` را باز کن – چند sitemap دارد؟ چه نوع‌هایی؟
3. یک Property در GSC بساز (لوکال را تأیید نمی‌کند، ولی کار را با یک سایت تستی تمرین کن) – رابط GSC را گشتی بزن.
4. صفحه «خدمات» را در PageSpeed Insights بده – سه پیشنهاد اولش را بنویس.
5. یک کسب‌وکار واقعی محلی خودت را (یا یک نمونه فرضی) در Google Business و نشان ثبت کن – فرم NAP را کامل پر کن.
6. در Yoast: Search Appearance → Title/Description پیش‌فرض پست‌ها را با برند خودت تنظیم کن.

▼ نکته تمرین ۲

Yoast برای هر نوع محتوا یک sitemap می‌سازد: `page-sitemap.xml` ، `post-sitemap.xml` ، و برای CPT پروژه‌ها هم `project-sitemap.xml` (چون فصل ۱۰ آن را public کردی) – همه در `sitemap_index` جمع‌اند. این فایل‌ها همان‌هایی هستند که در فصل ۱۷ به GSC سایت Next.js می‌دهی.

➔ **فصل بعد:** کاربران، نقش‌ها و تنظیمات کلیدی – از جمله Permalink که پایه API است.

فصل ۹ – کاربران، نقش‌ها و تنظیمات کلیدی

🎯 هدف: مدیریت کاربران و نقش‌های دسترسی + تنظیماتی که برای پروژه Headless حیاتی‌ان: **Permalinks** (ساختار URL – مستقیم روی API اثر داره!) و تنظیمات خواندن. آخر فصل، یک User Editor برای مشتری هم ساختی.

۹.۱ – نقش‌های کاربری: چه کسی چه اجازه‌ای داره؟

وردپرس ۵ نقش پیش‌فرض داره (از کم‌اختیار به پراختیار):

نقش	کارش	در پروژه واقعی کیه؟
Subscriber	فقط پروفایل خودش (کامنت با اسم خودش)	کاربر عادی سایت
Contributor	می‌نویسه، ولی انتشار دست دیگری است	نویسنده تازه‌کار
Author	پست‌های خودش را منتشر می‌کند	نویسنده
Editor	همه محتوا: همه پست‌ها، صفحات، دسته‌ها، کامنت‌ها	سرمدبیر / مدیر محتوا ★
Administrator	همه‌چیز: پلاگین، تم، کاربران، تنظیمات	تو (در لوکال) / مدیر فنی

قاعده‌ی امنیتی مهم برای پروژه واقعی: مشتری را **Editor** بده، نه **Admin**. چون Admin می‌تونه پلاگین نصب کنه و سایت رو بترکونه. خودت (فنی) Admin می‌مونی.

Users → Add New: یک کاربر `writer` با نقش Author بساز و یک بار باهاش لاگین کن (مرورگر ناشناس) – پنلش رو ببین: پلاگین و تنظیمات غیب شدن!

۹.۲ – Permalinks: ساختار URL ها ★ (برای Headless حیاتی)

Settings → Permalinks – این صفحه تعیین می‌کنه URL پست‌ها چطوری باشه:

گزینه	نمونه URL	نظر
Plain	?p=123	❌ سئوی صفر
Day and name	/عنوان/2026/09/13/	شلوغ
★ Post name	/عنوان-پست/	استاندارد سئو – انتخاب کن!
Custom	دلخواه	پیشرفته

چرا برای تو حیاتی است؟

- 1. Slug** (قسمت آخر URL) = شناسه‌ای که در Next.js برای routing استفاده می‌کند: slug ← `/blog/[slug]` از API میاد ← اینجا تعیین می‌شه که slug چی باشه
- 2.** با تغییر ساختار URL، permalink، های قدیمی می‌شکنن – پس **اول پروژه تعیینش کن، بعد دست نزن**
در ویرایشگر هر پست هم می‌تونی slug رو دستی تمیز کنی (عنوان فارسی → slug خودکار فارسی؛ برای URL تمیزتر می‌تونی لاتین بزنی: `best-tools-2026`).

۹.۳ – تنظیمات خواندن و دیدن (Reading)

Settings → Reading:

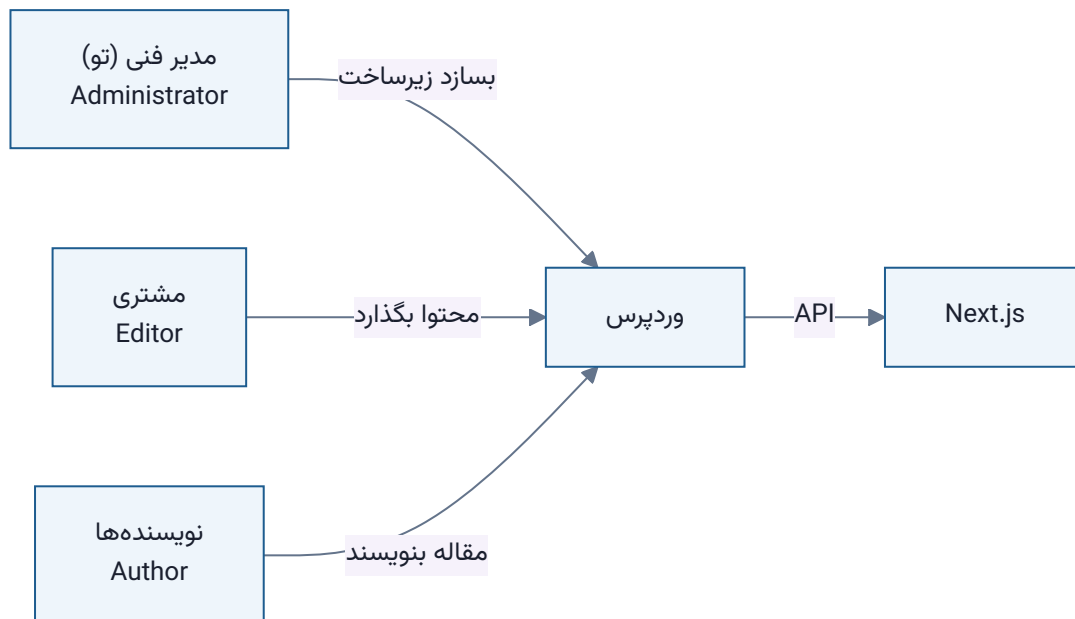
- **Your homepage displays:** «Your latest posts» (پیش‌فرض) یا یک Page ثابت به عنوان خانه
- **Blog pages show at most:** تعداد پست در هر صفحه آرشیو
- **Search engine visibility:** ☒ «Discourage search engines» – ⚠️ این تیک را هرگز در سایت واقعی نزن (سایت از گوگل حذف می‌شود!) فقط برای سایت‌های موقت تست خوبه

۹.۴ – تنظیمات مهم دیگر (گشت سریع)

مسیر	تنظیم مهم	نکته
General	Site Title / Tagline	عنوان و شعار سایت (در <code>name</code> , <code>description</code> API)
General	Timezone	Tehran
Discussion	کامنت‌ها	مدیریت نظرها – در Headless معمولاً کامنت را از سیستم دیگری می‌گیریم (فصل ۱۷)
Media	سایزهای تصویر	اندازه‌های خودکار تولیدی (فصل ۱۷: در <code>next/image</code> مفید)
Privacy	صفحه حریم خصوصی	صفحه policy بساز (برای فرم‌ها لازم می‌شه)

۹.۵ – گردش کار تیمی نمونه (پروژه واقعی)

سناریوی «سایت فروشگاهی با تیم ۳ نفره»:



تو زیرساخت رو می‌سازی (فصل ۱۰: CPT و فیلدها)، مشتری فقط محتوا می‌ریزه، و Next.js نشون می‌ده – مسئولیت‌ها تمیز جدا شدن.

جمع‌بندی فصل ✓

- نقش‌ها: Editor < Administrator < Author < Contributor < Subscriber – مشتری = Editor
- **Permalink روی «Post name»** – و slug تمیز برای routing در Next.js
- Reading: صفحه اصلی پیش‌فرض؛ تیک «Discourage search engines» خطرا!
- General/Timezone/Discussion/Media – سریع تنظیم کن

تمرین فصل ۹ 📝

1. Permalink را روی «Post name» بذار؛ یک پست با slug دستی لاتین بساز (`my-first-post`) و URL اش را ببین.
2. کاربر `writer` با نقش Author بساز و در مرورگر ناشناس باهاش لاگین شو – چه منوهایی را نمی‌بینی؟
3. کاربر `editor-boss` با نقش Editor بساز – چه اختیاراتی نسبت به Author بیشتره؟ (بازش کن و مقایسه کن)
4. تنظیمات General: عنوان سایت را اسم پروژه‌ات کن و Timezone را Tehran.
5. (برای فصل ۱۷) صفحه «حریم خصوصی» را از Settings → Privacy بساز.

▼ نکته تمرین ۳

Editor نسبت به Author: می‌تواند پست‌های دیگران را ویرایش/انتشار کند، صفحات و دسته‌ها را مدیریت کند و کامنت‌ها را مدیریت کند – ولی پلاگین/تم/کاربر (اختیارات Admin) ندارد.

→ **فصل بعد:** مهم‌ترین فصل بخش اول – Custom Post Types و Custom Fields: محتوای ساخت‌یافته‌ای که Headless با آن ساخته می‌شود!

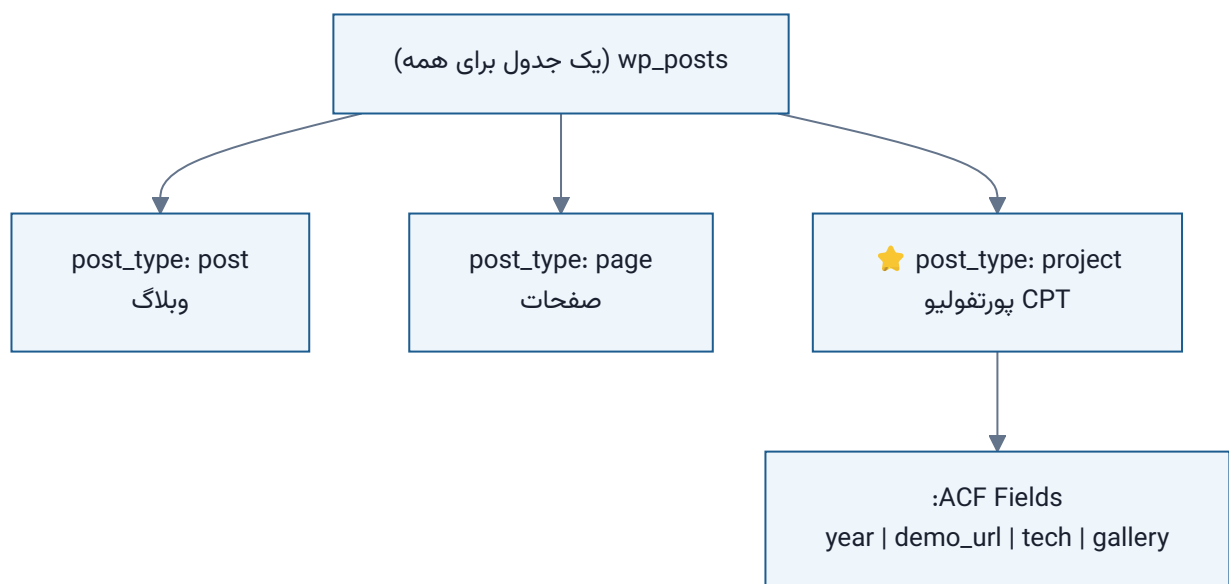
فصل ۱۰ – Custom Fields و Custom Post Types: محتوای ساخت‌یافته ★

🎯 مهم‌ترین فصل بخش اول! پست و صفحه برای «وبلاگ» کافیه، ولی پروژه واقعی «موجودیت» داره: محصول، پروژه پورتفولیو، نمای ملک، دوره آموزشی... یاد می‌گیری CPT بسازی و با ACF فیلدهای ساخت‌یافته بهش بدی – این دقیقاً همون مدل داده‌ای است که در Next.js به صورت typed مصرف می‌کنی.

۱۰.۱ – مسئله: پست برای «محصول» مناسب نیست

سناریو: سایت پورتفولیو می‌سازی؛ مشتری می‌خواه پروژه اضافه کنه با: عنوان، توضیح، عکس، سال انجام، لینک دمو، تکنولوژی‌ها. با پست معمولی؟ همه‌چیز می‌ریزه توی یک textarea آزاد – بدون ساختار، بدون کنترل.

جواب وردپرس: هر نوع محتوا = یک Custom Post Type (CPT) – مثل اینکه `post_type` جدید تعریف کنی. (یادت هست فصل ۴: همه‌چیز در جدول `wp_posts` با ستون `post_type` است؟ حالا داریم به همین ستون، نوع خودمون رو اضافه می‌کنیم!)



۱۰.۲ – ساخت CPT: دو راه

راه ۱: کد (روش حرفه‌ای ★)

در `functions.php` تم فرزند (فصل ۵!) یا در یک پلاگین اختصاصی (فصل ۱۱):

```
add_action('init', function () {
    register_post_type('project', [
        'labels' => [
            'name' => 'پروژه‌ها',
            'singular_name' => 'پروژه',
        ],
        'public' => true, // نمایش داده شود API در سایت و
        'has_archive' => true, // صفحه آرشیو داشته باشد
        'menu_icon' => 'dashicons-portfolio',
        'supports' => ['title', 'editor', 'thumbnail', 'excerpt'],
        'show_in_rest' => true, // ★★ در REST API بدون این ظاهر شود - نمی‌دهدش API
        'rewrite' => ['slug' => 'projects'],
    ]);
});
```

بعد از ذخیره: در پنل، منوی جدید «پروژه‌ها» ظاهر شد! ★ (اگر نیامد: Settings → Permalinks → Save یک بار بزن - قانون رایج رفرش قواعد URL).

راه ۲: پلاگین UI (سریع‌تر برای شروع)

پلاگین Add New CPT (CPT UI) Custom Post Type UI → همین فیلدها را با فرم پر کن. خروجی‌اش همان کد بالا است.

💡 نکته دولوپری: برای پروژه CPT، Headless را با کد بساز (در پلاگین خودت) تا version-able باشد - مثل تفاوت تنظیم دستی و Infrastructure as Code.

۱۰.۳ - Custom Fields با ACF: ساختار دقیق

CPT ساختار نوع محتوا را می‌دهد؛ فیلدهای دقیق (سال، لینک، تکنولوژی‌ها) را Custom Fields می‌دهند. پادشاه این حوزه پلاگین ACF (Advanced Custom Fields) است:

1. نصب (ACF (Plugins → Add New → Advanced Custom Fields → Activate)

2. Add New → Field Groups → ACF: اسم: «اطلاعات پروژه»

3. Location rule: «Post Type is equal to project» - یعنی این فیلدها فقط روی CPT پروژه ظاهر شوند

4. فیلدها را اضافه کن:

Field Label	Field Name (نام)	نوع	نکته
سال انجام	year	Number	
لینک دمو	demo_url	URL	
تکنولوژی‌ها	technologies	Checkbox (React, Next, Node...)	چند انتخابی
گالری	gallery	Gallery	چند عکس

5. حالا پروژه‌ها → **Add New**: فرم اختصاصی دوست! چند پروژه با داده‌های واقعی وارد کن (۳-۴ تا با عکس – سوخت پروژه Next.js!)

🔑 **Field Name** (نام انگلیسی) همان کلیدی است که در API می‌گیری – مثل `prop` های کامپوننت. تمیز و کوتاه بزن (`year` نه «سال انجام پروژه»).

۱۰.۴ – خروجی API را همین حالا ببین! (پیش‌تaste فصل ۱۴)

در مرورگر: `wp-course.local/wp-json/wp/v2/projects` – جیسون پروژه‌هایت را می‌بینی! و فیلدهای ACF با پلاگین **ACF to REST API** هم اضافه می‌شوند (`acf: {...}` در خروجی).

همین یک URL یعنی: وردپرس الان یک **بک‌اند استاندارد JSON** برای محتوای دوست – و در فصل‌های بعد از داخل Next.js می‌خوانیمش.

۱۰.۵ – Taxonomy اختصاصی (اختیاری ولی ارزشمند)

برای پروژه‌ها می‌خواهی «دسته‌بندی مهارت» هم داشته باشی؟ Custom Taxonomy:

```
add_action('init', function () {
  register_taxonomy('tech_stack', 'project', [
    'labels' => ['name' => 'تکنولوژی‌ها', 'singular_name' => 'تکنولوژی'],
    'public' => true,
    'show_in_rest' => true,           // ★ باز هم برای API
    'hierarchical' => false,         // مثل تگ، نه دسته درختی
  ]);
});
```

حالا پروژه‌ها را می‌توانی با فیلتر تکنولوژی آرشیو کنی – در Next.js: `/projects?tech=react`

جمع‌بندی فصل ✓

- محتوای ساخت‌یافته = **CPT** (نوع محتوا) + **ACF** (فیلدهای دقیق)

- CPT با `register_post_type` یا پلاگین CPT UI؛ حتماً `show_in_rest => true`
- ACF: Field Group + Location rule؛ `Field Name` = کلید API
- پروژه‌های نمونه را همین حالا با داده واقعی بگذار – سوخت فصل ۱۶
- Taxonomy اختصاصی = فیلتر/آرشیو ساخت‌یافته

تمرین فصل ۱۰ (پروژه‌محور – تا آخر دوره باهاش کار می‌کنیم!)

1. `project` CPT را با کد (در تم فرزند) بساز – با `show_in_rest` و `slug` اختصاصی.
2. ACF را نصب کن و Field Group «اطلاعات پروژه» را دقیقاً مثل جدول بالا بساز.
3. سه پروژه واقعی (از رزومه‌ات!) با عکس شاخص و فیلدهای کامل وارد کن.
4. `wp-json/wp/v2/projects` را در مرورگر باز کن – خروجی JSON پروژه‌هایت را ببین (عنوان، `slug`، آی‌دی).
5. پلاگین **ACF to REST API** را نصب و فعال کن و همان URL را رفرش کن – بخش `acf` را در JSON پیدا کن
(`year` , `demo_url` , `technologies`).
6. `tech_stack` Taxonomy را اضافه کن و به پروژه‌ها تکنولوژی بچسبان.

چک‌لیست تمرین ۴-۵ ▼

- `/wp-json/wp/v2/projects` → آرایه‌ای از آبجکت‌ها: `id` , `slug` , `title.rendered` , `_links`
- بعد از ACF to REST API: هر آبجکت یک کلید `acf` دارد →
`{"year": "2026", "demo_url": "...", "technologies": ["React", "Next"]}`
- اگر 404 گرفتی: Permalink را دوباره Save کن و `show_in_rest` را چک کن – دو پرتکرارترین گیرهای این مرحله!

➔ **فصل بعد:** کمی PHP برای دولوپر JS – تا کد وردپرسی‌ها را بخوانی و اولین پلاگینت را بنویسی.

فصل ۱۱ – PHP برای دولوپر JS: خواندن کد وردپرس و اولین پلاگین

🎯 هدف: قرار نیست PHP کار بشی – قراره به اندازه‌ای PHP بلد بشی که (۱) کد تم‌ها و پلاگین‌ها رو بخونی، (۲) فهم دقیق Hooks داشته باشی، و (۳) اولین پلاگین کاربردی خودت رو بنویسی. با ذهنیت JS/TS که داری، سریع‌تر از فکر می‌گیری.

۱۱.۱ – PHP در ۵ دقیقه (با مقایسه به JS)

مفهوم	JS	PHP
متغیر	<code>let x = 1;</code>	<code>\$x = 1;</code> ← همیشه با <code>\$</code>
رشته	<code>'a' + b</code>	<code>"a" . \$b</code> ← نقطه = اتصال
آرایه	<code>[1, 2]</code>	<code>[1, 2]</code>
آبجکت/دیکشنری	<code>{a: 1}</code>	<code>['a' => 1]</code> ← =>
تابع	<code>function f(\$a) {}</code>	<code>function f(\$a) {}</code> (آرگومان با <code>\$</code>)
خروجی	<code>console.log</code>	<code>echo / print_r</code>
شرط/حلقه	<code>if/for/foreach</code>	یکسان!
interpolate	<code>`hi \${n}`</code>	<code>"hi \$n"</code> (با دابل‌کوتهیشن)

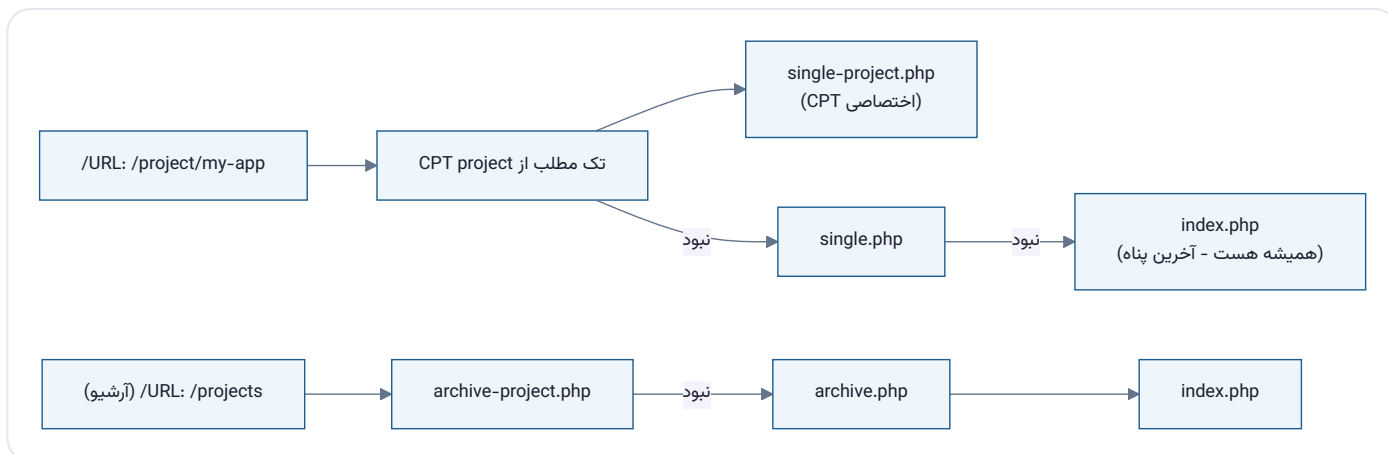
```
$projects = ['shop', 'blog'];
foreach ($projects as $p) {
    echo "پروژه: $p <br>";
}

$user = ['name' => 'Ali', 'role' => 'editor'];
echo $user['name']; // Ali
```

باقی‌اش با context یاد گرفته می‌شه – بذار بریم سراغ جاهایی که واقعاً لازمش داری.

۱۱.۲ – Template Hierarchy: وردپرس کدام فایل را نشان می‌دهد؟

وقتی بازدیدکننده یک URL را باز می‌کند، وردپرس با یک درخت تصمیم مشخص می‌کند از کدام فایل تم استفاده کند:



قاعده: از اختصاصی به عمومی. `single-{posttype}.php` برای تک‌نمایش هر `archive-{posttype}.php`، برای CPT، آرشیوش. (در Block Themes این‌ها template های Site Editor هستند – مفهوم یکی است).

۱۱.۳ – The Loop: قلب نمایش وردپرس

الگوی کلاسیک تم PHP برای پیمایش پست‌ها:

```

<?php if (have_posts()) : ?>
  <?php while (have_posts()) : the_post(); ?>
    <h2><a href="<?php the_permalink(); ?>"><?php the_title(); ?></a></h2>
    <div><?php the_excerpt(); ?></div>
  <?php endwhile; ?>
<?php else : ?>
  <p>چیزی یافت نشد</p>
<?php endif; ?>

```

به زبان تو:

```

if (posts.length) {
  for (const post of posts) {
    console.log(post.title, post.permalink);
  }
} else {
  console.log("چیزی یافت نشد");
}

```

دقیقاً یک `map` روی داده‌هاست – فقط با توابع کمکی وردپرس (`the_title()` و ...). تا اینجا PHP رو که فهمیدی، ۸۰٪ تم‌ها را می‌خوانی.

۱۱.۴ – Hooks: سیستم رویداد وردپرس ★ (مهم‌ترین مفهوم این فصل)

وردپرس در مسیر اجرایش هزاران نقطه قلاب دارد که تو می‌توانی کدت را آویزان کنی – بدون تغییر هسته. دو نوع:

نوع	کی اجرا می‌شود؟	می‌تواند چیزی را عوض کند؟	مثل
Action	«وقتی X اتفاق افتاد، این را هم بکن»	نه (فقط کار انجام می‌دهد)	event listener
Filter	«این مقدار را از من عبور بده»	بله – مقدار را تغییر می‌دهد و برمی‌گرداند	pipe / interceptor

```
// Action: بعد از فعال شدن پلاگین، چیزی انجام بده
add_action('init', function () {
    register_post_type('project', [ /* ... فصل ۱۰ ... */ ]);
});

// Filter: عنوان پست‌ها را هنگام نمایش تغییر بده
add_filter('the_title', function ($title, $id) {
    if (get_post_type($id) === 'project') {
        return $title . ' 🚀';
    }
    return $title; // همیشه مقدار را برگردان!
}, 10, 2); // اولویت + تعداد آرگومان‌ها
```

با همین دو تابع (`add_action` و `add_filter`) کل وردپرس قابل تغییره – فلسفه open/closed: هسته بسته، توسعه باز.

۱۱.۵ – اولین پلاگین واقعی خودت: «تولید پست از قالب»

فصل ۷ دیدی پلاگین = یک PHP با هدر. حالا یکی می‌سازیم که کاری مفید کند: موقع ساخت هر پروژه جدید، به‌طور خودکار مقدار فیلد `year` را سال جاری بگذارد:

در `wp-content/plugins/my-tools/my-tools.php`:

```
<?php
/**
 * Plugin Name: My Headless Tools
 * Description: ابزارهای پروژه Headless من
 * Version: 1.0.0
 */

// وقتی پروژه جدید ذخیره می‌شود، اگر سال خالی بود، سال جاری را بگذار
add_action('save_post_project', function ($post_id) {
    if (get_post_status($post_id) === 'publish') return;
    if (!empty(get_field('year', $post_id))) return; // get_field از ACF
    update_field('year', (int) date('Y'), $post_id); // update_field از ACF
});
```

پلاگین‌ها → Activate → یک پروژه جدید بدون سال بساز → publish → بازش کن: سال خودش پر شده! 🎉

تو الان یک پلاگین واقعی نوشتی که با `save_post_project` (event) کار می‌کند – دقیقاً همون ذهنیت `onClick` / `onChange` تو.

⚠ سه قانون توسعه پلاگین: (۱) هرگز هسته را ویرایش نکن – همه چیز با hook؛ (۲) همه چیز را در پلاگین/تم فرزند؛ (۳) قبل از تغییر مهم: Snapshot.

۱۱.۶ – ابزارهای دیباگ سریع

```
// console.log (مثل خروجی داده با فرمت خوانا):
echo '<pre>'; print_r($data); exit;
// wp-content/debug.log یا به لاگ:
error_log(print_r($data, true));
// wp-config.php فعال سازی دیباگ در:
// define('WP_DEBUG', true); define('WP_DEBUG_LOG', true);
```

(exit) مثل return زود هنگام اسکریپت را می کشد – برای دیدن یک متغیر وسط جریان.)

✓ جمع بندی فصل

- JS = PHP با \$ و . به جای +؛ آرایه انجمنی با ≥
- Template Hierarchy: از اختصاصی (single-project.php) به عمومی (index.php)
- The Loop = با توابع کمکی posts.map(...)
- Action = رویداد، Filter = تغییر مقدار – دو قلاب add_action / add_filter
- اولین پلاگین واقعی: hook روی save_post_project – بدون دست زدن به هسته

📝 تمرین فصل ۱۱

1. فایل single.php تم Twenty Twenty-Five را باز کن (یا پوشه تم را بگرد) و The Loop را در آن پیدا کن – به زبان JS اش بازنویسی کن (روی کاغذ).
2. پلاگین «My Headless Tools» را بساز، فعال کن و سناریوی سال خودکار را تست کن.
3. یک add_filter روی the_content بنویس که آخر همه پست های project جمله «ساخته شده با ❤» را اضافه کند – در سایت (نه API) تست کن.
4. (چالش) یک Action روی wp_insert_post بنویس که error_log کند عنوان هر پست جدید – و فایل debug.log را ببین (WP_DEBUG_LOG فعال کن).

```
add_filter('the_content', function ($content) {
    if (get_post_type() === 'project' && is_singular('project')) {
        return $content . '<p>ساخته شده با ❤️</p>';
    }
    return $content;
});
```

توجه: `the_content` روی فرانت اعمال می‌شود؛ خروجی REST API پیش‌فرض فیلترهای خودش را دارد (در فصل ۱۴ می‌بینی).

➔ **فصل بعد:** 🛒 ووکامرس – فروشگاه کامل روی همین دانش: محصولات، سبد خرید و پرداخت ایرانی.

فصل ۱۲ – ووکامرس: فروشگاه کامل

🎯 هدف: ووکامرس (WooCommerce) = پلاگین فروشگاه‌ساز رایگان دنیا که وردپرس را به یک فروشگاه آنلاین کامل تبدیل می‌کند: محصولات، سبد خرید، تسویه، ارسال و پرداخت. بزرگ‌ترین بازار کار وردپرس ایران است – و نسخه Headless آن (WooGraphQL) هم پشتیبانی می‌شود. آخر فصل یک فروشگاه واقعی با درگاه تست داری.

۱۲.۱ – ووکامرس: فروشگاه‌ساز استاندارد دنیا

وکامرس سابقه دارد: متن‌باز، ساخت Automattic (سازنده خود وردپرس)، موتور ~۲۵٪ از کل فروشگاه‌های اینترنتی دنیا. برای تو یعنی:

- پلاگین است – همان‌طور که فصل ۷ یاد گرفتی: `wp plugin install woocommerce --activate` ★
- خودش از CPT داخلی استفاده می‌کند: `product` (محصول)، `order` (سفارش)، `coupon` (کوپن) – یعنی همه دانش CPT/ACF فصل ۱۰ صادق است
- پس از فعال‌سازی: **Wizard راه‌اندازی** می‌آید – آدرس، صنف، فروش فیزیکی/دیجیتال – پیش‌فرض‌ها را رد کن، بعداً همه‌اش قابل تغییر است

بعد از نصب، منوهای جدید پنل: **WooCommerce** (پیشخوان فروشگاه)، محصولات، تحلیل‌ها، بازاریابی و بخش سفارش‌ها.

۱۲.۲ – تنظیمات پایه برای بازار ایران

Settings → WooCommerce – اول این‌ها را ست کن:

تب	تنظیم	مقدار برای ایران
General	واحد پول	تومان (IRT) – اگر نبود، با پلاگین فارسی‌ساز می‌آید
General	فروش در کشورهای خاص	ایران
Products	واحد وزن/بعد	کیلوگرم / سانتی‌متر
Tax	فعال‌سازی مالیات	بسته به کسب‌وکار (بیشتر مواقع خاموش تا مشورت حسابدار)

💡 پلاگین فارسی‌سازی (مهم برای ایران): پلاگین‌های ایرانی مثل **WP-Parsi** یا ****Jalali**** سه کار کلیدی می‌کنند:

(۱) تاریخ‌های شمسی در کل فروشگاه، (۲) واحد تومان، (۳) اتصال شهرهای ایران برای ارسال. بدون آن، تاریخ میلادی و قیمت‌های دلاری می‌بینی – در بازار ایران حکم خطای فاحش دارد ⚠️

۱۲.۳ – محصولات: چهار نوع

محصولات → Add New – اولین تصمیم: نوع محصول:

نوع	چيست	مثال
Simple (ساده)	یک قیمت، یک موجودی	کابل دووی ۵ متری
Variable (متغیر) ★	چند گزینه با قیمت/موجودی متفاوت – با ویژگی	کابل با طول: ۵/۱۰/۲۰ متر
Virtual	ارسال ندارد	مشاوره آنلاین
Downloadable (دانلودی)	فایل قابل دانلود بعد از خرید	فایل آموزش PDF

داده‌های هر محصول:

- عنوان + توضیح (ادیتور بلوکی خودمان!) + توضیح کوتاه (زیر عنوان محصول در صفحه فروش)
- قیمت عادی / قیمت تخفیفی (Sale) ★
- تصویر محصول (شاخص) + گالری تصاویر (چند عکس)
- دسته‌بندی محصول + برچسب محصول (همان taxonomy – جدا از دسته‌های بلاگ!)
- موجودی (SKU کد کالا، تعداد، اجازه سفارش پس‌اتمام؟)
- ویژگی‌ها (Attributes): رنگ، سایز، طول... – بعد از تعریف، در تب **Variations** برای هر ترکیب قیمت/موجودی جدا بده (متغیر)

🔑 ذهنیت دولوپری: هر محصول = یک post در wp_posts با `post_type=product`؛ قیمت و SKU در جدول `wp_postmeta`؛ ویژگی‌ها taxonomy اختصاصی (`pa_color` ...) – دقیقاً همان معماری فصل ۱۰ که خودت برای «پروژه» ساختی، فقط حرفه‌ای‌تر.

۱۲.۴ – نمایش محصولات در سایت

صفحه فروشگاه و دسته‌ها را ووکامرس خودش می‌سازد (فایل‌های template خودش دارد). برای چیدن محصولات در صفحه دلخواه (مثلاً «پرفروش‌ها» در صفحه خانه):

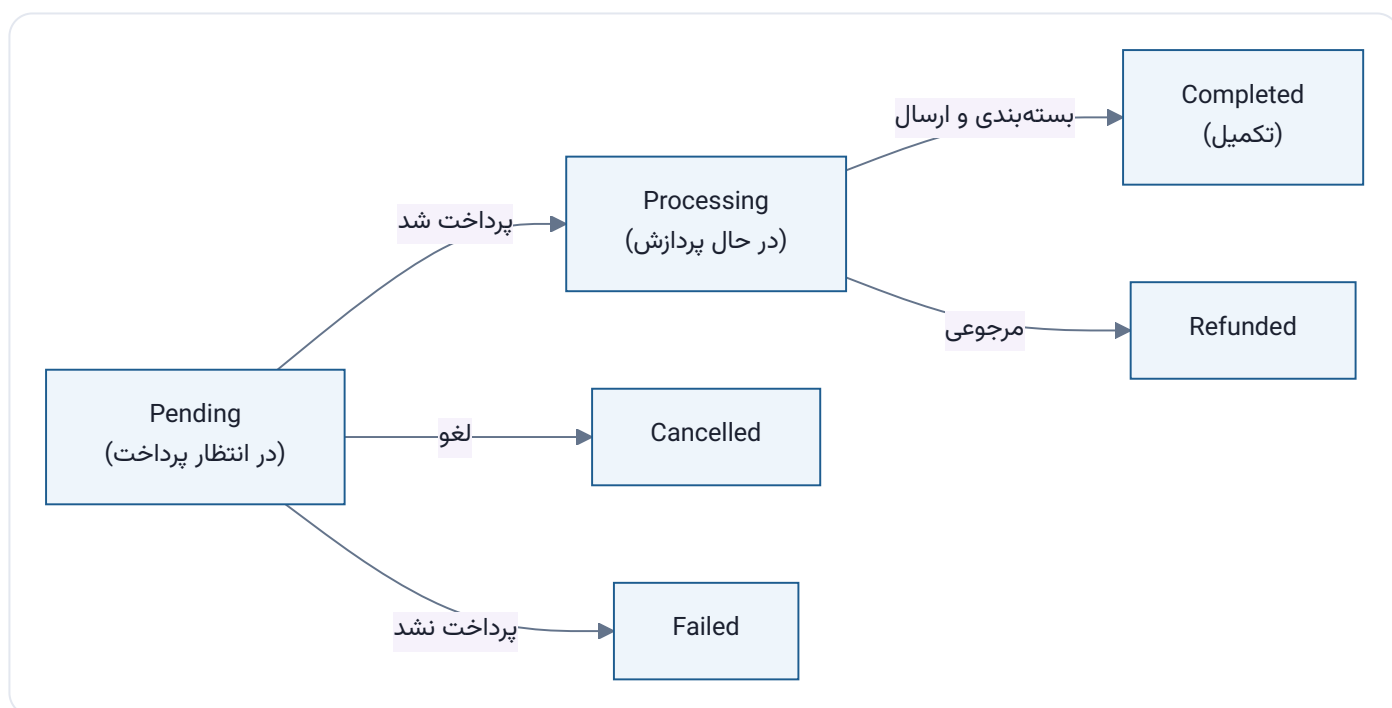
- **بلوک‌های گوتنبرگ:** بلوک «Products by Category» / «On Sale Products» / «Best Selling Products» – با Site Editor جلوه‌های آماده ★
- **شورت‌کد کلاسیک:** `[products limit="4" columns="4" orderby="popularity"]` – در هر صفحه/ویجت کار می‌کند
- در Elementor: ویجت‌های ووکامرس (اگر نصب باشد)
- با کد هم برون‌سپاری می‌شود: `wc_get_products([...])` – ولی معمولاً بلوک‌ها کافی‌اند.

۱۲.۵ – ویرایش گروهی و کلاس مالیاتی

- **Bulk Edit:** در لیست محصولات، چند محصول تیک بزن → Edit → Actions – قیمت/موجودی/وضعیت را گروهی عوض کن (مثلاً «همه ۱۰٪ تخفیف» – صرفه‌جویی چند ساعته!) ★
- **Import/Export:** محصولات را CSV وارد/خارج کن (WooCommerce → Products → All Products → Import) – برای ۵۰۰ محصول اولیه از اکسل مشتری
- **کلاس مالیاتی:** در Settings → Tax – برای فروشگاه‌هایی که نرخ مالیات متفاوت دارند (فعلاً آگاهی کافی است)

۱۲.۶ – سبد خرید و گردش سفارش

گردش استاندارد سفارش (وضعیت‌های ووکامرس):



- مشتری: صفحه محصول → **Add to Cart** → سبد → **Checkout** (تسویه: آدرس + روش ارسال + پرداخت) → سفارش ساخته می‌شود

- مدیر: **WooCommerce → Orders** – سفارش‌ها را می‌بینی، وضعیت را تغییر می‌دهی؛ به مشتری ایمیل/پیامک می‌رود (پیامک: فصل ۱۸ ★)
- صفحات پیش‌فرض (سبد/تسویه/حساب کاربری) را Wizard خودش می‌سازد – در **Settings → Advanced** جابجایی‌اند
- **حساب کاربری:** مشتری‌ها در **My Account** سفارش‌ها و آدرس‌هایشان را می‌بینند (نقش کاربری جدید: Customer – فصل ۹)

۱۲.۷ – ارسال

Shipping Zones → Shipping → Settings:

1. یک Zone بساز: «تهران» → روش: **Flat Rate** (نرخ ثابت، مثلاً ۵۰,۰۰۰ تومان) یا **Free Shipping** (حداقل خرید X)
2. Zone دوم: «سایر شهرها» → نرخ بالاتر
3. **Local Pickup:** تحویل حضوری رایگان
4. هزینه پویا بر اساس وزن: در Flat Rate، فرمول $cost + (weight \times per_kg)$ – کلاس‌های ارسال (Shipping Classes) برای دسته‌های سنگین/سبک

💡 برای پست/تتیاکس واقعی: پلاگین‌های ایرانی (پست پیشتاز/سنجاب) هزینه را از API سرویس پستی می‌گیرند – فصل ۱۸ در لیست افزونه‌های ایرانی می‌بینی.

۱۲.۸ – پرداخت: درگاه‌های ایرانی ★

پرداخت در ووکامرس با **Payment Gateways** است. درگاه‌های پیش‌فرض (چک/حواله/پرداخت در محل) رایگان‌اند؛ برای پرداخت آنلاین واقعی در ایران:

ویژگی	درگاه
محبوب‌ترین برای شروع – ثبت‌نام ساده، پلاگین ووکامرس رسمی، کارمزد کم	زرین پال (zarinpal.com) ★
تسویه سریع، API تمیز	آیدی‌پی (idpay.ir)
نیاز به حساب تجاری بانک + قرارداد – برای فروشگاه‌های جدی	درگاه مستقیم بانکی (ملت/سامان/پارسیان...)
بدون پول واقعی، کل گردش را تمرین کن ★	تست (Sandbox): درگاه تست زرین پال

راه‌اندازی (نمونه زرین پال): حساب زرین پال → Merchant ID را بگیر → پلاگین «پرداخت زرین پال برای ووکامرس» را نصب
 → Merchant ID → ZarinPal → Payments → Settings → WooCommerce + فعال‌سازی. حالا Checkout یک

دکمه پرداخت واقعی دارد که به درگاه بانکی می‌رود و برمی‌گردد و وضعیت سفارش را خودکار به Processing می‌گذارد 🎨

کوپن تخفیف: Marketing → Coupons — کد/درصد/سقف/تاریخ انقضا/محدودیت ایمیل — همان‌هایی که در فصل ۱۹ برای کمپین مشتری می‌سازی.

۱۲.۹ — فروشگاه Headless: پیش‌نگاه WooCommerce

ووکامرس در WPGGraphQL (فصل ۱۵) هم می‌آید — با پلاگین WPGGraphQL for WooCommerce (WooGraphQL):

```
query {
  products(first: 8) {
    nodes {
      id slug name ... on SimpleProduct { price regularPrice stockStatus }
      ... on VariableProduct { variations { nodes { name price } } }
      image { sourceUrl altText }
    }
  }
}
```

یعنی: پنل = ووکامرس برای مشتری؛ فرانت = Next.js فروشگاه با React (چرخ/server-side cart). مسیر پیشرفته فصل ۱۶ به بعد — ولی بدان مسیر رسمی است و در بازار، «فروشگاه Headless» بالاترین قیمت را دارد 💰

✓ جمع‌بندی فصل

- ووکامرس = پلاگین فروشگاه کامل: محصولات/سبد/تسویه/سفارش — با `wp plugin install woocommerce --activate`
- تنظیم ایران: تومان + شمسی‌سازی با پلاگین ایرانی — بدون آن فروشگاه «غیرقابل عرضه» است
- ۴ نوع محصول؛ متغیر + ویژگی + ترکیب‌ها؛ تصویر شاخص + گالری
- گردش سفارش: Pending → Processing → Completed (مدیریت از Orders)
- ارسال با Shipping Zones؛ پرداخت با درگاه ایرانی (زرین‌پال برای شروع) + تست Sandbox
- WooGraphQL = مسیر فروشگاه Headless — پنل ووکامرسی، فرانت Next.js

📝 تمرین فصل ۱۲ (پروژه: فروشگاه ابزار «برق‌آسا»)

1. ووکامرس را نصب/فعال کن و Wizard را بگذار رد بشود؛ تاریخ را شمسی کن (پلاگین فارسی‌ساز نصب کن) و واحد را تومان.
2. ۵ محصول Simple بساز (عکس + گالری + دسته «ابزار برقی» + موجودی) — داده واقعی بگذار، سوخت فصل ۱۶ است.
3. یک محصول Variable بساز: کابل با طول ۵/۱۰/۲۰ متر — قیمت‌های متفاوت و موجودی جدا.
4. یک محصول Downloadable بساز (یک PDF تست) — بعد از «خرید» در My Account قابل دانلود است؟

5. دو Shipping Zone بساز: تهران ۵۰ هزار تومان؛ بقیه ۸۰ هزار؛ + خرید بالای ۲ میلیون = ارسال رایگان.

6. درگاه تست زرین پال را وصل کن و یک سفارش واقعی بزن: سبد → پرداخت تست → برگشت → وضعیت Processing



7. Bulk Edit: به همه محصولات پرفروش ۱۰٪ تخفیف (قیمت Sale گروهی) – در صفحه فروشگاه ببین.

▼ نکته تمرین ۶

اگر درگاه تست راه نیفتاد: اول SSL چک (حتی لوکال؛ در Local گواهی خودش) – درگاه‌ها با http کار نمی‌کنند. بعد Merchant ID را با وضعیت «فعال» چک کن. سفارش‌های تست را از Trash → Orders پاک کن تا گزارش‌ها تمیز بمانند.

➔ فصل بعد: امنیت، بکاپ و نگهداری – سایت فروشگاهی تازه متولد شده؛ حالا باید زنده نگهش داری.

فصل ۱۳ – امنیت، بکاپ و نگهداری: سایت زنده بماند

🎯 **هدف:** ساخت سایت شغل نصفه است؛ نگاهداشتنش نصف دیگر. در این فصل سه ستون نگهداری را یاد می‌گیری: امنیت (که هک نشوی)، بکاپ (که حتی اگر شد، چیزی نپری)، و عملکرد (که سریع بماند). همین فصل، مبنای «قرارداد نگهداری ماهانه» فصل ۱۹ است – یعنی درآمد تکرارشونده 💰

۱۳.۱ – چرا وردپرس هدف شماره یک حمله است؟

بازار ۴۰٪+ یعنی هدف‌گیری ۴۰٪+ – ربات‌های هکر روزانه میلیون‌ها سایت وردپرسی را با همین الگوها تست می‌کنند:

بردار حمله	سهم تقریبی	دفاع
پلاگین/تم قدیمی با حفره ★	اکثریت قاطع	آپدیت منظم + حذف بی‌استفاده‌ها (فصل ۷)
رمز/نام‌کاربری ضعیف (admin / 123456)	خیلی زیاد	رمز قوی + 2FA + محدودسازی لاگین
پلاگین نال‌شده/نامعتبر	رایج در ایران	فقط مخزن رسمی و منبع معتبر (فصل ۷)
هاست ناامن	کمتر ولی قاتل	هاست معتبر + بکاپ آف‌سایت

🔑 **قانون تلخ:** هک شدن تقریباً هیچ‌وقت «هوشمندانه» نیست – صبر و انضباط تو را می‌بُرد، نه مادربوردت را. ۹۰٪
دفاع = آپدیت + رمز + بکاپ.

۱۳.۲ – لایه‌های امنیت پایه (رایگان و فوری)

۱. آپدیت‌ها: هسته/تم/پلاگین – هفتگی چک کن یا Auto-update پلاگین‌های معتبر WP-CLI: `wp core update &&` `(wp plugin update --all)`
۲. کاربر Admin: نام‌کاربری `admin` هرگز (اگر داری، با Add New → Users یک ادمین جدید بساز و `admin` را حذف کن)؛ رمز عبور طولانی < کوتاه پیچیده؛ **Application Password** برای API نه رمز اصلی (فصل ۱۴)
۳. محدودسازی تلاش لاگین: پلاگین **Limit Login Attempts Reloaded** – بعد از چند تلاش اشتباه، IP قفل می‌شود
۴. 2FA (ورود دومرحله‌ای): پلاگین Solid Security یا Wordfence برای اکانت ادمین‌ها ★
۵. نقش‌ها را رعایت کن: مشتری = Editor نه Admin (فصل ۹)
۶. تیک‌های خطرناک: «هرکسی می‌تواند ثبت‌نام کند» را خاموش نگاه‌دار مگر لازم باشی (Settings → General)

۱۳.۳ – پلاگین امنیتی: Wordfence یا Solid Security

یکی را نصب کن (هر دو نه – تداخل!):

قابلیت	چه می‌کند
Firewall	درخواست‌های مخرب را قبل از رسیدن به وردپرس می‌بندد
★ Scan	فایل‌های هسته را با اصل مقایسه می‌کند؛ کد تزریق‌شده را پیدا می‌کند
Login Security	2FA + captcha + لاگ ورودها
Live Traffic	می‌بینی ربات‌ها چگونه تست می‌کنند (ترسناک و آموزشی!)
Country Blocking	بستن پل از کشورهای غیرلازم (احتیاط: API تو هم می‌تواند بلاک شود – برای Headless دقت کن!)

⚠ نکته Headless: اگر فایروال به کلاینت API (سرور Next.js) اجازه ندهد، سایت «خطای اتصال» می‌دهد. Whitelist سرورهای Vercel یا حساسیت فایروال را روی حالت متعادل بگذار – و بعد از تنظیم، حتماً یک fetch از سایت Next.js تست کن.

۱۳.۴ – حالت تعمیر (Maintenance Mode)

وقتی در حال تغییر بزرگ هستی (مهاجرت فصل ۱۸، تغییر قالب)، سایت مخاطب نباید «نیمه‌شکسته» را ببیند:

- پلاگین Maintenance یا قابلیت داخلی ووکامرس – صفحه «به‌زودی برمی‌گردیم» + فقط ادمین‌ها سایت را کامل می‌بینند
- WP-CLI هم می‌شود: `wp maintenance-mode activate / deactivate` ★ (یادت باشد خاموشش کنی!)
- برای سئو: کد وضعیت 503 بده (پلاگین‌ها می‌دهند) – گوگل می‌فهمد «موقتی» است نه «مُرده»

۱۳.۵ – بکاپ: قانون ۱-۲-۳

۳ نسخه، روی ۲ رسانه، ۱ نسخه آف‌سایت (خارج سرور). سرور هک شود/هارد بسوزد/مشتري «اشتباهاً» پاک کند – تو زنده می‌مانی.

راه حرفه‌ای: UpdraftPlus (فصل ۷ نصب کردی):

1. Settings → UpdraftPlus Backups → تب Settings
2. زمان‌بندی: فایل‌ها هفتگی، دیتابیس روزانه (فروشگاه: هر دو روزانه!)
3. ذخیره آف‌سایت: Google Drive (تصدیق با اکانت گوگل – رایگان ۱۵GB) ★ – یا Dropbox
4. تعداد نسخه نگه‌داری: حداقل ۴

بکاپ دستی (فهم زیرساخت):

- فایل‌ها: پوشه کامل سایت (مهم): `wp-content` – تم‌ها/پلاگین/آپلودها – در Local همین پوشه، در هاست File Manager → Compress
- دیتابیس: phpMyAdmin → Export → SQL (یا `wp db export backup.sql` با WP-CLI ★)

```
# ریستور دستی با WP-CLI:  
wp db import backup.sql  
# روی همان مسیر zip بازگردانی فایل‌ها: اکسپورت
```

⚠ Snapshot در Local (فصل ۳) برای آزمایش‌های توسعه است؛ بکاپ UpdraftPlus برای محصول است. هر دو را باهم نگه دار.

۱۳.۶ – بازیابی: سناریوهای واقعی

سناریو	علاج
«صفحه/محتوایی گم شد»	فصل ۴: Revision ها! ویرایشگر → Revision – برگرد به نسخه قبل
سایت سفید شد بعد از نصب پلاگین	WP-CLI: <code>wp plugin deactivate --all</code> → سایت می‌آید؛ بعد یکی‌یکی فعال کن تا مقصر را پیدا کنی
هک/ویروس (ریدایرکت اسپم...)	Wordfence Scan → پاک‌سازی؛ اگر کامل نیست: ریستور بکاپ تمیز + تعویض همه رمزها (پنل/FTP/DB)
دیتابیس خراب	<code>wp db repair</code> یا phpMyAdmin → Repair Table؛ آخرش ریستور
آپدیت ووکامرس فروشگاه را شکست	Snapshot/بکاپ قبلی → ریستور → آپدیت را در Staging تست کن

Staging (آگاهی): نسخه کپی سایت برای تست آپدیت‌ها – Local یک دکمه **Clone** دارد؛ هاست‌های خوب هم Staging یک‌کلیکی می‌دهند. قاعده: آپدیت بزرگ اول Staging، بعد Live ★

۱۳.۷ – سرعت و کش

سایت کند = سئوی ضعیف (فصل ۸) = فروش کمتر. دو لایه اصلی:

- کش صفحات:** پلاگین **LiteSpeed Cache** (اگر هاست لایت‌اسپید است – در ایران اکثراً هست) یا W3 Total Cache – صفحات PHP را به HTML آماده تبدیل می‌کند؛ ده‌ها برابر سریع‌تر
- فشرده‌سازی رسانه:** WebP و Smush (فصل ۶) – عکس، ۷۰٪ وزن یک صفحه وردپرسی است

3. **CDN: Cloudflare رایگان** – DNS را به کلادفلر منتقل کن (فصل ۲: NS!) → کش جهانی + لایه امنیتی رایگان. در ایران

گاهی با اختلال همراه است – برای بازار داخلی، CDN داخلی هاستینگ‌ها هم گزینه است

4. **نگاه دولوپری:** در Headless این کل لایه ساده می‌شود – وردپرس فقط API است و کش واقعی روی Next.js/CDN

★ (فصل ۱۷: ISR)

۱۳.۸ – ایمیل‌های وردپرس: SMTP

وردپرس ایمیل می‌فرستد (ریست رمز، سفارش ووکامرس، فرم‌ها، نوتیف ادمین) – ولی تابع `mail()` روی اکثر هاست‌ها یا

کار نمی‌کند یا در اسپم گم می‌شود ⚠️

راه‌حل استاندارد: پلاگین **WP Mail SMTP** + یک SMTP واقعی:

- **ایمیل هاست خودت** (my@domain.ir از سی‌پنل – ساده‌ترین برای ایران)
- یا سرویس ایمیل (Gmail با App Password، یا سرویس‌های تبلیغاتی ایرانی برای خبرنامه)
- تنظیم: Host/Port/Username/Password → WP Mail SMTP → دکمه **Email Test** – تا این سبز نشود، فرم‌ها و سفارش‌ها بی‌صدا می‌میرند.

۱۳.۹ – چک‌لیست نگهداری ماهانه (این را به مشتری بفروش!)

- [] آپدیت هسته/تم/پلاگین‌ها (+ تست بعد آپدیت)
- [] بکاپ‌ها واقعاً گرفته شده‌اند؟ (UpdraftPlus → Existing Backups – و یک بار Download و باز کردن!)
- [] Wordfence Scan کامل بدون خطر
- [] صفحات اصلی سایت را باز کن: خانه/محصول/فرم/پرداخت تست
- [] PageSpeed بگیر؛ اگر افت کرد: عکس‌های جدید مشتری را فشرده کن!
- [] فضای هاست/تاریخ انقضای دامنه و SSL (از GSC و سی‌پنل)
- [] گزارش کوتاه برای مشتری: بازدید، سفارش‌ها، کارهای انجام‌شده (فصل ۱۹: این گزارش = دلیل تمدید قرارداد)

✓ جمع‌بندی فصل

- حمله‌ها ساده‌اند: آپدیت + رمز قوی + 2FA + محدودسازی لاگین + پلاگین معتبر = ۹۰٪ دفاع
- Wordfence/Solid Security: فایروال + اسکن + 2FA – فقط یکی؛ با فایروال، مسیر API سرور Next.js را نبند
- بکاپ ۱-۲-۳ با **UpdraftPlus + Google Drive**؛ دیتابیس روزانه، فایل هفتگی؛ دستی: `wp db export`
- بازیابی: Revision برای محتوا؛ deactivate--all برای سفیدی سایت؛ Staging برای آپدیت‌های بزرگ
- کش (LiteSpeed) + CDN + WebP = سرعت؛ SMTP = ایمیل‌هایی که واقعاً می‌رسند

تمرین فصل ۱۳

۱. رمز ادمین لوکالت را عوض کن (Users → Profile)؛ کاربر تست با رمز ضعیف بساز و Limit Login Attempts را نصب کن – بعد از چند لاگین اشتباه چه می‌شود؟
۲. Wordfence را نصب و یک Scan کامل بزن – نتیجه را بخوان (لوکال چند هشدار بی‌خطر دارد).
۳. UpdraftPlus: زمان‌بندی دیتابیس روزانه + اتصال Google Drive → یک بکاپ دستی بزن و فایل‌ها را در درایو ببین؛ بعد Download کن و به `wp db import` یا phpMyAdmin ریستور تمرین کن (روی سایت تست!).
۴. یک پلاگین بی‌دردسر نصب کن که سایت را بشکند (یا یک خطای syntax عمدی در پلاگین تمرینی فصل ۱۱ بگذار!) – سایت سفید شد؟ با `wp plugin deactivate --all` برگردان.
۵. WP Mail SMTP با ایمیل محلی Local (Mail Catcher داخلی) تنظیم و تست کن – ایمیل ریست رمز را بگیر.
۶. PageSpeed برای صفحه خانه پروژه‌ات بگیر و سه اقدام بهبود را اجرا کن (عکس WebP معمولاً اولین است).

▼ نکته تمرین ۳

ریستور را روی یک سایت تست Local تمرین کن، نه سایت اصلی: UpdraftPlus → Existing Backups → Restore. فقط فایل‌ها و دیتابیس را برمی‌گرداند به لحظه بکاپ. تمرین یک‌بار ریستور، در بحبان واقعی تفاوت «پنیک» و «حرفه‌ای» است.

→ فصل بعد: WP REST API – ورود رسمی به بخش سوم: وردپرس به عنوان بک‌اند JSON.

فصل ۱۴ – WP REST API: وردپرس به عنوان بک‌اند JSON

🎯 هدف: ورود رسمی به بخش سوم – Headless! وردپرس از قبل یک API کامل REST دارد (wp-json /). یاد می‌گیری endpoint ها را با curl صدا بزنی، فیلتر کنی، و با Application Password احراز هویت کنی. این پایه‌ی فهم WPGraphQL هم می‌شود (فصل ۱۵).

۱۴.۱ – API رایگان که از قبل داری!

هیچ پلاگینی لازم نیست – وردپرس از ۴.۷ به بعد REST API کامل دارد. تستش کن (در مرورگر یا curl):

```
$ curl "http://wp-course.local/wp-json" | head -40
# endpoint نقشه کل API: ریشه

$ curl "http://wp-course.local/wp-json/wp/v2/posts"
# همه پست‌هایت! (همان‌هایی که فصل ۴ نوشتی) JSON آرایه

$ curl "http://wp-course.local/wp-json/wp/v2/projects"
# پورتفولیوی فصل ۱۰ CPT 🚀
```

الگوی endpoint: wp-json/wp/v2/{نوع}

نکته	چه چیزی	endpoint
فیلترهای قدرتمند می‌گیرد	پست‌ها	wp/v2/posts
	صفحات	wp/v2/pages
اگر show_in_rest بود (فصل ۱۰)	CPT تو!	wp/v2/projects
	دسته/برچسب	wp/v2/categories / tags
برای next/image	رسانه‌ها (عکس‌ها!)	wp/v2/media
عمومی فقط عمومی‌ها	کاربران	wp/v2/users
پایین‌تر	فیلتر	wp/v2/pages?slug=about

۱۴.۲ – آناتومی پاسخ (چه چیزی می‌گیری؟)

بریده‌ای از خروجی یک پست:

```
{
  "id": 12,
  "date": "2026-09-13T10:00:00",
  "slug": "my-first-post",
  "status": "publish",
  "link": "http://wp-course.local/my-first-post/",
  "title": { "rendered": "عنوان پست" },
  "content": { "rendered": "<p>متن کامل با HTML...</p>" },
  "excerpt": { "rendered": "<p>خلاصه.</p>" },
  "featured_media": 34,
  "categories": [4],
  "_links": { "self": [...], "wp:featuredmedia": [...]}
}
```

نکته‌های دولوپری:

- ★ همان کلید routing تو در `/blog/[slug]` (Next.js) = `slug`
- `HTML` = `content.rendered` / `title.rendered` آماده (ادیتور بلوکی رندرش کرده) – در Next.js با `dangerouslySetInnerHTML` یا پارسر
- `featured_media` = فقط آی‌دی؛ عکس واقعی یا در `_links` است یا با یک درخواست دیگر (یا در GraphQL – اینجا است که GraphQL می‌درخشه!)
- مقادیر ACF با پلاگین «ACF to REST API» در کلید `acf` می‌آیند (فصل ۱۰ نصب کردی ✓)

۱۴.۳ – فیلترها: کوئری‌های واقعی

REST API وردپرس فیلترهای آماده دارد:

```
# خاص (برای صفحه جزئیات!) slug پست با:
$ curl "http://wp-course.local/wp-json/wp/v2/posts?slug=my-first-post"

# صفحه‌بندی + تعداد:
$ curl ".../wp/v2/posts?per_page=5&page=1"
# pagination! (برای کل تعداد = X-WP-Total نکته: هدر)

# فیلتر دسته، ترتیب، جستجو:
$ curl ".../wp/v2/projects?per_page=12&orderby=date&order=desc&search=react"
```

۱۴.۴ – احراز هویت: Application Passwords

خواندن محتوای منتشرشده عمومی؛ ولی **نوشتن/ویرایش** و دیدن **پیش‌نویس‌ها** نیاز به احراز هویت دارد. راه استاندارد وردپرس مدرن: **Application Password** (رمز جدا برای هر اپ، نه پسورد اصلی):

۱. پنل → Profile → Users → پایین صفحه: **Application Passwords** → اسم: **Add New** → `nextjs-headless`
۲. یک رمز مثل `xxxx xxxx xxxx xxxx xxxx xxxx` می‌دهد – ★ همین یک بار نمایش داده می‌شود؛ ذخیره‌اش کن
۳. استفاده با Basic Auth:

```
$ curl -u "admin:XXXX XXXX XXXX XXXX XXXX" \
"http://wp-course.local/wp-json/wp/v2/posts?status=draft"
# حالا پیش‌نویس‌ها را هم می‌بینی ←
```

این همان جفت کلید/token دوره‌های قبلیت است: مثل PAT گیت‌هاب – با قابلیت لغو مستقل. در Next.js در `.env` می‌گذاری و هرگز به کلاینت نمی‌روی.

۱۴.۵ – نوشتن با API (تست)

```
$ curl -X POST -u "admin:XXXX XXXX..." \
-H "Content-Type: application/json" \
-d '{"title": "پست از API!", "status": "draft"}' \
"http://wp-course.local/wp-json/wp/v2/posts"
```

برو پنل – پیش‌نویس جدید! (در پروژه‌های Headless معمولاً از این برای فرم‌ها استفاده می‌شود – فصل ۱۷)

۱۴.۶ – محدودیت‌های REST (و چرا GraphQL می‌آیم!)

REST وردپرس خوبه ولی سه دردسر داره:

مشکل	مثال	نتیجه
Over-fetch	برای لیست کارتها فقط <code>title</code> و <code>featured_media</code> لازم داری؛ ولی <code>content</code> کامل HTML هر پست هم می‌آید	پهنای باند هدر
N+1 برای روابط	<code>featured_media</code> فقط آی‌دی است؛ عکس واقعی = درخواست جدا برای هر پست	۱۲ پست = ۱۲ درخواست اضافه
embedded پیچیده	راه‌حل <code>_embed</code> هست ولی خروجی را شلوغ و غیرقابل پیش‌بینی می‌کند	DX پایین

GraphQL دقیقاً این سه را حل می‌کند: هرچی خواستی می‌پرسی (نه بیشتر)، روابط در همان کوئری می‌آیند، شکل پاسخ = شکل کوئری. فصل بعد!

جمع‌بندی فصل ✓

- `/wp-json/wp/v2/{نوع}` – بدون هیچ نصبی؛ CPT تو با `show_in_rest` آمده
- `slug` = کلید routing؛ HTML = `content.rendered` آماده؛ ACF در `acf`
- فیلترها: `search` , `orderby` , `per_page` , `?slug=` ؛ هدر `X-WP-Total`

- نوشتن و پیش‌نویس‌ها = **Application Password** (مثل PAT – در env.)
- در سرهای REST → دلیل ورود ما به GraphQL

تمرین فصل ۱۴

1. سه endpoint را در مرورگر باز کن: JSON – posts، pages، projects را مقایسه کن.
2. با curl پست `my-first-post` را با فیلتر slug بگیر و `id` و `featured_media` را استخراج کن (با python یا فقط چشم!).
3. هدر `X-WP-Total` را ببین: `curl -sI ".../wp/v2/posts?per_page=5" | grep -i x-wp` – چند پست داری؟
4. `Application Password` بساز و پیش‌نویس‌ها را با آن ببین؛ بدون رمز چه خطایی می‌گیری؟ (401!)
5. با curl یک پیش‌نویس جدید بساز و در پنل ببینش.
6. فیلدهای `acf` پروژه‌هایت را در خروجی چک کن (پلاگین ACF to REST API فعال باشد).

▼ نکته تمرین ۲

```
curl -s "http://wp-course.local/wp-json/wp/v2/posts?slug=my-first-post" | python -m json.tool  
# [ { "id": 12, "featured_media": 34, ... } ]  
آرایه با حداکثر یک عضو
```

الگوی مهم: اول با slug آیدی بگیر، بعد با آیدی کار کن.

→ **فصل بعد:** WPGraphQL – کوئری دقیق، روابط تو در تو، و پایه‌ی پروژه Next.js!

☆ فصل ۱۵ – WPGraphQL: کوئری دقیق از وردپرس

🎯 هدف: پلاگین WPGraphQL را نصب می‌کنیم تا وردپرس یک سرور GraphQL کامل شود. با GraphQL کوئری می‌نویسی: لیست پست‌ها با عکس، CPT با فیلدهای ACF، منوها – همه در یک درخواست. این ستون فقرات پروژه Next.js فصل بعد است.

۱۵.۱ GraphQL در ۶۰ ثانیه (برای تو که می‌دونی)

```
سرور جواب می‌دهد (همان شکل)
{
  posts {
    nodes {
      title
      featuredImage { node { sourceUrl } }
    }
  }
}
```

```
: تو می‌پرسی (شکل دلخواه):
{
  "posts": {
    "nodes": [
      { "title": "...", },
      { "featuredImage": { ... } }
    ]
  }
}
```

هرچی بخوای می‌گیری، نه بیشتر – و روابط (عکس پست، نویسنده، دسته‌ها) در همان درخواست. دقیقاً سه مشکل REST فصل ۱۴ را حل می‌کند.

۱۵.۲ – نصب و GraphQL

1. Plugins → Add New → **WPGraphQL** → Install → Activate

2. در پنل، منوی جدید **GraphQL → GraphQL IDE** را باز کن – یک playground آماده (مثل Swagger/Postman) برای GraphQL

اولین کوئری‌ات را بزن:

```
{
  posts(first: 3) {
    nodes {
      id
      slug
      title
      date
    }
  }
}
```

Ctrl+Enter → خروجی JSON با دقیقاً همین شکل. نوار سمت راست GraphQL، **دوکومنت کامل schema** را نشان می‌دهد (Docs) – مستندات زنده خود وردپرس!

۱۵.۳ – کوئری لیست: پست با عکس شاخص در یک درخواست

همان چیزی که در REST به N+1 دردرس افتاد:

```
{
  posts(first: 6) {
    pageInfo {
      hasNextPage
      endCursor
    }
    nodes {
      id
      slug
      title
      date
      excerpt
      featuredImage {
        node {
          sourceUrl
          altText
          mediaDetails { width height }
        }
      }
      categories {
        nodes { name slug }
      }
    }
  }
}
```

همه چیز: پست + عکس + دسته ها – یک درخواست. (`pageInfo` / `endCursor` برای cursor-based pagination در Next.js – مثل فصل ۹ دوره Prisma!)

۱۵.۴ – کوئری تک پست (برای صفحه جزئیات)

```
{
  post(id: "my-first-post", idType: SLUG) {
    id
    slug
    title
    date
    content
    author {
      node { name }
    }
    featuredImage { node { sourceUrl altText } }
  }
}
```

`idType: SLUG` یعنی با slug جستجو کن – دقیقاً چیزی که از `params` روتر Next.js می گیری.

۱۵.۵ – CPT تو + فیلدهای ACF در GraphQL ⭐

CPT پروژه (فصل ۱۰) خودکار در schema آمده: `projects`. برای فیلدهای ACF، پلاگین **WPGraphQL for ACF** را نصب/فعال کن (نسخه جدید ACF خودش پشتیبانی دارد – در Field Group هر فیلد «Show in GraphQL» را on کن یا

کل گروه را).

کوئری واقعی پروژه‌ها:

```
{
  projects(first: 10) {
    nodes {
      id
      slug
      title
      date
      featuredImage { node { sourceUrl altText } }
      projectFields {
        year
        demoUrl
        technologies
        gallery {
          nodes { sourceUrl altText }
        }
      }
      techStack {
        nodes { name slug }
      }
    }
  }
}
```

این JSON دقیقاً همون shape ای است که در کامپوننت React می‌خواهی – بدون transform!

۱۵.۶ – منوها و صفحات

```
{
  menus(where: { location: PRIMARY }) {
    nodes {
      name
      menuItems {
        nodes { label url parentId }
      }
    }
  }
  pages(first: 20) {
    nodes { slug title }
  }
}
```

منو و لیست صفحات برای نوار بالای سایت Next.js – در یک درخواست.

نکته	چرا
پیش‌نویس‌ها با auth	WPGraphQL احراز هویت را با Application Password (فصل ۱۴) هم ساپورت می‌کند – برای Preview فصل ۱۷
<code>first: N</code> به جای <code>per_page</code>	cursor-based؛ برای صفحه‌های بی‌نهایت عالی
caching	WPGraphQL با پلاگین‌های کش (مثل WPGraphQL Smart Cache) بهتر می‌شود – برای production
secure endpoint	endpoint شما <code>http://wp-course.local/graphql</code> است – همان را Next.js صدا می‌زند

جمع‌بندی فصل

- **WPGraphQL** = سرور GraphQL روی وردپرس؛ playground = GraphQL + مستندات زنده
- لیست با عکس و دسته در یک کوئری؛ تک‌پست با `idType: SLUG`
- CPT = تو `projects` (root field)؛ ACF بعد از فعال‌سازی GraphQL برای ACF در گروه `projectFields`
- منوها و صفحات هم در schema – کل داده سایت در یک API مدرن

تمرین فصل ۱۵ (خروجی هر کوئری = مصرف مستقیم در فصل ۱۶!)

1. WPGraphQL را نصب/فعال کن و اولین کوئری posts را در GraphQL بزنی.
2. کوئری لیست با `featuredImage` را اجرا کن – `sourceUrl` عکس‌ها را ببین.
3. کوئری تک‌پست با SLUG را برای `my-first-post` بزنی.
4. WPGraphQL for ACF را فعال کن و کوئری `projects` با `projectFields` را بزنی – سال و تکنولوژی‌های پروژه‌ها را ببین! (اگر گروه فیلدها نیامد: در ویرایش Field Group، تب GraphQL → Show in GraphQL را on کن.)
5. کوئری منوها را بزنی؛ اگر منو خالی بود، در پنل منو را بساز و **Location** اش را Primary کن.
6. (کشف) در GraphQL، پنل Docs را باز کن و root فیلد `mediaItems` را پیدا کن – همه عکس‌های سایت در یک لیست!

▼ نکته تمرین ۴

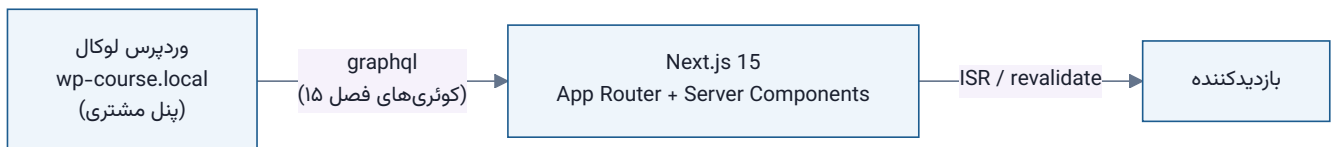
مسیرهای رایج: (الف) ACF نسخه +6: Field Group → Settings → GraphQL → Enable؛ (ب) یا پلاگین «WPGraphQL for ACF». بعد از فعال‌سازی، در GraphQL با Ctrl+Space داخل `projects { ... }` autocomplete فیلدها را می‌بینی — اگر `projectFields` نیامد، schema را رفرش کن (رفرش صفحه).

🚀 فصل بعد: لحظه بزرگ — اتصال Next.js به همین API و ساخت سایت واقعی! ➡

فصل ۱۶ – Next.js + وردپرس: پروژه Headless کامل 🚀

🎯 لحظه‌ای که کل دوره برایش ساخته شد: یک اپ (Next.js (App Router) که محتوایش را از وردپرس لوکال می‌خواند: لیست پست‌ها با عکس، صفحه جزئیات هر پست با slug، و آرشیو پروژه‌های CPT با فیلدهای ACF. فرانت = تو؛ بک‌اند = وردپرس.

۱۶.۱ – معماری نهایی



و یک نکته امنیتی از دوره‌های قبل: فقط سرور Next.js به وردپرس وصل می‌شود (Server Components) – endpoint وردپرس هرگز به مرورگر لو نمی‌رود.

۱۶.۲ – ست‌آپ پروژه

```
$ npx create-next-app@latest wp-headless --typescript --app --src-dir
$ cd wp-headless
$ npm i graphql-request
```

(graphql-request = کلاینت سبک کوئری GraphQL؛ می‌شد با fetch خالی هم رفت – پایین‌تر هر دو را می‌بینی.)

.env.local (از دوره‌های قبل: هرگز در git!):

```
WORDPRESS_API_URL="http://wp-course.local/graphql"
```

⚠️ اگر بعداً روی Vercel دپلوی کردی، wp-course.local فقط روی سیستم تو معناست – باید وردپرس روی یک هاست عمومی باشد (فصل ۱۷). لوکال را کامل کن، بعد دپلوی.

لایه داده: src/lib/wp.ts – همه کوئری‌های وردپرس یک‌جا:

```

import { GraphQLClient } from "graphql-request";

const client = new GraphQLClient(process.env.WORDPRESS_API_URL!);

type Post = {
  id: string;
  slug: string;
  title: string;
  date: string;
  excerpt?: string;
  content?: string;
  featuredImage?: { node?: { sourceUrl?: string; altText?: string } } };

export async function getPosts(first = 6): Promise<Post[]> {
  const data = await client.request<{ posts: { nodes: Post[] } }>(`
    query GetPosts($first: Int!) {
      posts(first: $first) {
        nodes {
          id slug title date excerpt
          featuredImage { node { sourceUrl altText } }
        }
      }
    `, { first });
  return data.posts.nodes;
}

export async function getPostBySlug(slug: string): Promise<Post | null> {
  const data = await client.request<{ post: Post | null }>(`
    query GetPost($slug: ID!) {
      post(id: $slug, idType: SLUG) {
        id slug title date content
        featuredImage { node { sourceUrl altText } }
      }
    `, { slug });
  return data.post;
}

```

💡 تایپ‌ها دستی نوشتیم؛ ابزار GraphQL Code Generator می‌تواند از schema وردپرس تایپ بسازد (یادت که هست – قدم بعدی حرفه‌ای‌ات!).

۱۶.۳ – صفحه لیست پست‌ها

: src/app/blog/page.tsx

```

import Link from "next/link";
import { getPosts } from "@lib/wp";

export const revalidate = 60; // ISR: (پایین‌تر) کش ۶۰ ثانیه

export default async function BlogPage() {
  const posts = await getPosts(12);

  return (
    <main dir="rtl" className="max-w-5xl mx-auto p-6">
      <h1 className="text-3xl font-bold my-8">بلاگ</h1>
      <div className="grid grid-cols-1 sm:grid-cols-3 gap-6">
        {posts.map((post) => (
          <article key={post.id} className="border rounded-xl overflow-hidden hover:shadow-md">
            {post.featuredImage?.node?.sourceUrl && (
              <img src={post.featuredImage.node.sourceUrl}
                alt={post.featuredImage.node.altText ?? ""}
                className="w-full h-40 object-cover" />
            )}
            <div className="p-4">
              <h2 className="font-semibold">{post.title}</h2>
              <time className="text-sm text-gray-500">
                {new Date(post.date).toLocaleDateString("fa-IR")}
              </time>
              <Link href={`/${blog}/${post.slug}`}
                className="text-blue-600 text-sm mt-2 inline-block">
                ادامه مطلب →
              </Link>
            </div>
          </article>
        ))}
      </div>
    </main>
  );
}

```

! <http://localhost:3000/blog> Server Component معمولی – `await` مستقیم، بدون `useEffect`. حمله کن به

۱۶.۴ – صفحه جزئیات با slug (دینامیک روت)

: `src/app/blog/[slug]/page.tsx`

```

import { notFound } from "next/navigation";
import { getPostBySlug, getPosts } from "@lib/wp";

export const revalidate = 60;

// build (Static Generation): پیش‌ساخت صفحات محبوب در
export async function generateStaticParams() {
  const posts = await getPosts(10);
  return posts.map((p) => ({ slug: p.slug }));
}

export default async function PostPage({
  params,
}: {
  params: Promise<{ slug: string }>;
}) {
  const { slug } = await params;
  const post = await getPostBySlug(slug);

  if (!post) notFound();

  return (
    <main dir="rtl" className="max-w-2xl mx-auto p-6">
      <h1 className="text-3xl font-bold my-6">{post.title}</h1>
      {post.featuredImage?.node?.sourceUrl && (
        <img src={post.featuredImage.node.sourceUrl}
          alt={post.featuredImage.node.altText ?? ""}
          className="rounded-xl w-full" />
      )}
      </* content آماده است HTML از وردپرس */>
      <article
        className="prose prose-lg mt-6 max-w-none"
        dangerouslySetInnerHTML={{ __html: post.content ?? "" }}
      />
    </main>
  );
}

```

سه نکته‌ی این فایل که «همه‌چیز» است:

1. `generateStaticParams` = در `build`، برای پست‌های شناخته‌شده صفحه HTML آماده می‌سازد (SSG) – سرعت نور
2. `revalidate = 60` = ISR: بعد از ۶۰ ثانیه، بعدی که آمد، صفحه از نو از وردپرس رندر می‌شود – یعنی مشتری پست جدید می‌گذارد و حداکثر تا یک دقیقه در سایتت ظاهر می‌شود، بدون دیپلوی مجدد!
3. `dangerouslySetInnerHTML` = محتوای HTML گوتنبرگ. امن است چون محتوا از پنل خودت می‌آید (نه کاربران ناشناس) – و اسمش فقط ترسناکه!

۱۶.۵ – آرشیو پروژه‌ها (CPT + ACF)

– با فیلدهای ACF فصل ۱۵: `src/app/projects/page.tsx`

```

type Project = {
  id: string; slug: string; title: string;
  featuredImage?: { node?: { sourceUrl?: string } };
  projectFields?: {
    year?: number;
    demoUrl?: string;
    technologies?: string[];
  };
};

async function getProjects(): Promise<Project[]> {
  const res = await fetch(process.env.WORDPRESS_API_URL!, {
    method: "POST", // GraphQL به POST
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      query: `query { projects(first: 20) {
        nodes { id slug title
          featuredImage { node { sourceUrl } }
          projectFields { year demoUrl technologies }
        } } }`,
    }),
    next: { revalidate: 60 },
  });
  const { data } = await res.json();
  return data.projects.nodes;
}

export default async function ProjectsPage() {
  const projects = await getProjects();
  return (
    <main dir="rtl" className="max-w-5xl mx-auto p-6">
      <h1 className="text-3xl font-bold my-8">پورتفولیو</h1>
      <div className="grid sm:grid-cols-2 gap-6">
        {projects.map((p) => (
          <div key={p.id} className="border rounded-xl p-5">
            <h2 className="font-bold">{p.title}</h2>
            <p className="text-gray-500 text-sm">
              {p.projectFields?.year} {p.projectFields?.technologies?.join(", ")}
            </p>
            {p.projectFields?.demoUrl && (
              <a href={p.projectFields.demoUrl} target="_blank"
                className="text-blue-600 text-sm">دمو {p.projectFields?.demoUrl}</a>
            )}
          </div>
        ))}
      </div>
    </main>
  );
}

```

(اینجا عمده با `fetch` خالی نوشتم تا هر دو روش را دیده باشی – `graphql-request` یا `fetch` خام، سلیقه‌ای است.)

۱۶.۶ – خلاصه چرخه کامل

```

# وردپرس (فصل‌های ۳ تا ۱۵):
docker compose up -d # ببخشید! mean: در Local دکمه Start 🤖

# فرانت:
npm run dev
# /blog ← لیست GraphQL
# /blog/[slug] ← جزئیات
# /projects ← CPT + ACF

```

مشتري در پنل وردپرس پست می‌گذارد → تا ۶۰ ثانیه بعد، سایت Next.js تو به‌روز است. **Headless کامل شد.** 🎉

✓ جمع‌بندی فصل

- `WORDPRESS_API_URL` در `.env.local`؛ لایه داده در `lib/wp.ts` (کوئری‌های فصل ۱۵)
- لیست = `Server Component + grid`؛ جزئیات = `generateStaticParams` + `[slug]` + `notFound`
- `revalidate` = پل زنده بودن محتوا بدون دیپلوی (ISR)
- `content.rendered` با `dangerouslySetInnerHTML` (از پنل خودت = امن)
- `CPT/ACF = typed data` در کامپوننت‌ها

📝 تمرین فصل ۱۶ (پروژه اصلی دوره – حتماً کاملش کن!)

1. پروژه `wp-headless` را بساز، `.env.local` را ست کن و صفحه `/blog` را اجرا کن – پست‌های واقعی‌ات را ببین!
2. صفحه جزئیات `/blog/[slug]` را بساز و از لیست بهش بری.
3. تست ISR: در وردپرس یک پست جدید Publish کن؛ ۶۰ ثانیه صبر، رفرش – هست؟ بعد `revalidate = 10` کن و دوباره تست کن.
4. صفحه `/projects` را با ACF بساز – فیلدهای سال و تکنولوژی را نمایش بده.
5. چالش UI: کارت‌ها را با Tailwind حرفه‌ای کن (سایه، hover، تاریخ فارسی با `toLocaleDateString("fa-IR")`).
6. چالش بعدی: صفحه 404 سفارشی (`not-found.tsx`) و لینک «بازگشت به بلاگ».

▼ نکته تمرین ۳

ISR با `revalidate` کش سرور Next.js را منقضی می‌کند. برای «آنی» شدن (بدون انتظار ۶۰ ثانیه) راه پیشرفته: Webhook از وردپرس به `revalidatePath` در Next.js (Route Handler) – در فصل ۱۷!

➔ **فصل بعد:** حرفه‌ای‌سازی – Preview پیش‌نویس‌ها، فرم‌ها، سئوی Yoast، تصاویر next/image و دیپلوی روی Vercel.

فصل ۱۷ – Headless پیشرفته: Preview، فرم‌ها، SEO، تصاویر و دیپلوی

🎯 هدف: فصل «تبدیل دمو به محصول» – چهار قابلیت که پروژه Headless واقعی را از دمو جدا می‌کند: پیش‌نویس‌بینی برای مشتری (Preview)، فرم‌ها، سئوی کامل با داده Yoast، تصاویر بهینه next/image و دیپلوی روی Vercel.

۱۷.۱ – Preview: مشتری پیش‌نویس را ببیند! ★

مشکل: پست پیش‌نویس از API عمومی نمی‌آید (فصل ۱۴). مشتری می‌خواهد قبل از Publish ببیند چطور شده. راه‌حل: **Preview Mode** – یک Route Handler که با secret وردپرس، درخواست preview را با احراز هویت رندر می‌کند.

گام ۱ – `src/app/api/preview/route.ts` :Route Handler

```
import { draftMode } from "next/headers";
import { redirect } from "next/navigation";
import { NextRequest } from "next/server";

export async function GET(req: NextRequest) {
  const secret = req.nextUrl.searchParams.get("secret");
  const slug = req.nextUrl.searchParams.get("slug") ?? "";

  // چک کن - هرکس با لینک تصادفی وارد نشود .env سیکرت را با
  if (secret !== process.env.PREVIEW_SECRET) {
    return new Response("Invalid secret", { status: 401 });
  }

  (await draftMode()).enable(); // فعال‌سازی Draft Mode در Next.js
  redirect(`/blog/${slug}`);
}
```

گام ۲ – خواندن پیش‌نویس در `lib/wp.ts`: وقتی draft mode روشن است، کوئری با احراز هویت و `asPreview` بزن:

```
export async function getPreviewPost(slug: string, token: string) {
  const client = new GraphQLClient(process.env.WORDPRESS_API_URL!, {
    headers: { Authorization: `Basic ${Buffer.from(
      `${process.env.WP_USER}:${process.env.WP_APP_PASSWORD}`
    ).toString("base64")}` },
  });
  const data = await client.request(`
    query($slug: ID!) {
      post(id: $slug, idType: SLUG, asPreview: true) {
        slug title content date
      }
    }
  `, { slug });
  return data.post;
}
```

(.user:password از Base64 – فصل ۱۴ Application Password همان = WP_APP_PASSWORD)

گام ۳ – استفاده: در صفحه [slug] ، وقتی draftMode().isEnabled است، به جای getPostBySlug ، پیش‌نویس را با توکن بخوان.

گام ۴ – لینک در وردپرس: دکمه پیش‌نمایش وردپرس را طوری شخصی‌سازی کن (یا دستی) به:

<https://your-site.vercel.app/api/preview?secret=XXX&slug=my-draft>

مشتری روی «Preview» می‌زند → نسخه پیش‌نویس را روی سایت واقعی Next.js می‌بیند – بدون publish! (پلاگین‌هایی مثل Faust.js/Headlessbooks همین جریان را خودکار می‌کنند.)

۱۷.۲ – فرم‌ها: تماس با ما و کامنت‌ها

فرم تماس (ساده‌ترین راه درست): فرم POST → Server Action → React به REST وردپرس (فصل ۱۴ یادته) به یک CPT اختصاصی مثل پیام‌ها :

```
"use server";
export async function submitContact(formData: FormData) {
  const res = await fetch(`${process.env.WORDPRESS_API_URL!.replace("/graphql", "")}/wp/v2/messages
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Basic ${Buffer.from(
        `${process.env.WP_USER}:${process.env.WP_APP_PASSWORD}`
      ).toString("base64")}`,
    },
    body: JSON.stringify({
      title: String(formData.get("subject")),
      status: "private", // پیام‌ها فقط در پنل دیده شوند
      acf: { email: formData.get("email"), message: formData.get("message") }, // ACF to REST
    }),
  });
  if (!res.ok) return { error: "ارسال نشد" };
  return { ok: true }; // مشتری در پنل وردپرس، باکس «پیام‌ها» را می‌بیند
}
```

الگوی زیبا: پیام‌ها = یک CPT با ACF؛ مشتری همان‌جا در پنل وردپرس می‌بیندشون. (برای اسپم: honeypot + Google reCAPTCHA سمت کلاینت).

کامنت‌ها: دو راه — (۱) POST به `/wp/v2/comments` (باید `allow_comments` باشد؛ اسپم را Akismet می‌گیرد)، (۲) سرویس آماده مثل Giscus (بر پایه GitHub Discussions — برای بلاگ‌های فنی محبوب).

۱۷.۳ — SEO: داده metadata → Yoast نیکست

Yoast (فصل ۷) مقادیر سئو را در وردپرس می‌گذارد؛ با WPGraphQL for SEO (افزونه یوست برای GraphQL) یا REST `/wp/v2/posts?context=edit` یا پلاگین WPGraphQL Yoast SEO می‌آیند. الگوی مصرف در Next.js:

```
export async function generateMetadata({ params }): Promise<Metadata> {
  const { slug } = await params;
  const post = await getPostBySlug(slug);
  return {
    title: post?.title,
    description: post?.excerpt?.replace(/<[^>+>/g, "").slice(0, 150),
    openGraph: {
      title: post?.title,
      images: post?.featuredImage?.node?.sourceUrl ? [post.featuredImage.node.sourceUrl] : [],
    },
  };
}
```

و دو فایل استاندارد: `sitemap.ts` (همه slug ها از وردپرس) و `robots.ts` — با توابع آماده Next.js. (اسکلت‌شان را با `sitemap.ts` سرچ کن در داکيومنت؛ داده‌اش همین کوئری لیست پست‌هاست).

۱۷.۴ — تصاویر: next/image با رسانه وردپرس

`<img>` ساده کار می‌کند ولی `next/image` بهینه‌تره (lazy load، سایز درست). برای اینکه Next.js عکس‌های دامنه وردپرسی را قبول کند:

```
// next.config.ts
const nextConfig = {
  images: {
    remotePatterns: [
      { protocol: "http", hostname: "wp-course.local" },
      // بعداً production و دامنه وردپرس
    ],
  },
};
```

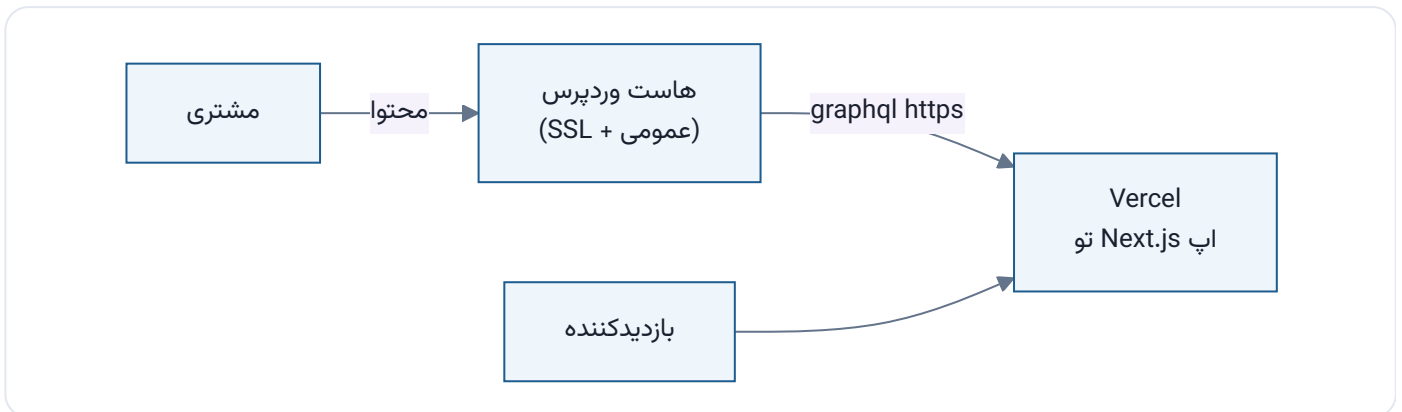
و در کامپوننت:

```
<Image src={p.featuredImage.node.sourceUrl} alt={p.featuredImage.node.altText ?? ""}
width={800} height={450} className="rounded-xl" />
```

(اندازه‌های `mediaDetails` فصل ۱۵ را می‌توانی بدهی تا layout shift صفر شود.)

۱۷.۵ – دپلوی: Vercel + وردپرس عمومی

معماری production:



گام‌ها:

- ۱. وردپرس عمومی:** یک هاست وردپرسی (یا VPS با داکر – دوره داکرت!) – سایت وردپرس فقط برای پنل و API است؛ تمش مهم نیست، پیش‌فرض کافی است
- ۲. تست:** `https://دامنه/wp-json/graphql` و `/graphql` از اینترنت در دسترس باشد
- ۳. Vercel:** ریپو Next.js را import کن `WORDPRESS_API_URL` Environment Variables: (دامنه واقعی) + رازهای preview
- ۴. تحویل بگیر:** هر push به main = دپلوی؛ هر تغییر محتوا در وردپرس = تا `revalidate` ثانیه در سایت **بعدش (اختیاری حرفه‌ای):** Webhook بلافاصله – پلاگین در وردپرس روی `save_post` یک درخواست به `/api/revalidate?secret=...` بزند و تو در Route Handler با `revalidatePath("/blog")` کش را فوری باطل کنی – همان الگوی `$transaction` دوره Prisma: به‌روزرسانی اتمی محتوا. (WPGraphQL Smart Cache هم راه GraphQL-native این کار است.)

۱۷.۶ – چک‌لیست تحویل پروژه Headless

- [] وردپرس عمومی با SSL؛ پنل با رمز قوی؛ کاربر مشتری = Editor
- [] CPT/ACF با `GraphQL enabled – versioned` یا `show_in_rest` در پلاگین خودت
- [] Application Password در env های Vercel (هرگز در git)
- [] ISR/revalidate تنظیم؛ تست «پست جدید تا یک دقیقه می‌آید»
- [] `Yoast → metadata/sitemap/robots`
- [] `next/image + remotePatterns`
- [] فرم تماس + اسپم‌گیر
- [] بکاپ خودکار وردپرس (UpdraftPlus – فصل ۷)

✓ جمع‌بندی فصل

- **Preview** = draft mode + secret + احراز هویت Application Password – مشتری پیش‌نویس را روی سایت واقعی می‌بیند
- فرم‌ها: CPT → REST POST → Server Action → پیام‌ها؛ مشتری در پنل می‌بیند
- SEO: Yoast → `generateMetadata` + sitemap/robots
- تصاویر: `remotePatterns` + next/image + alt از رسانه
- دپلوی: وردپرس عمومی + ISR + Vercel؛ ارتقا: Webhook revalidate فوری

📝 تمرین فصل ۱۷ (فینال پروژه!)

1. Preview Mode را کامل راه بینداز: secret در هر دو طرف + Route Handler + تست با یک پیش‌نویس.
2. فرم تماس بساز (Client Component با useActionState فصل ۱۵ دوره Next) که پیام را به `messages` CPT بفرستد – در پنل وردپرس ببینش.
3. `generateMetadata` را به صفحه جزئیات اضافه کن – منبع صفحه را ببین (عنوان سئو یوست اگر وصل کردی).
4. همه `<img>` ها را به `next/image` مهاجرت کن + `remotePatterns`.
5. (اگر هاست داری ★) وردپرس را عمومی کن و پروژه را روی Vercel دپلوی کن – لینکش را به رزومه‌ات اضافه کن!
6. چالش فینال: Webhook revalidate – پلاگین WP Webhooks یا `save_post` فصل ۱۱ که به Route Handler تویی curl بزند.

▼ اسکلت تمرین ۶

```
// app/api/revalidate/route.ts
import { revalidatePath } from "next/cache";
import { NextRequest } from "next/server";

export async function POST(req: NextRequest) {
  if (req.nextUrl.searchParams.get("secret") !== process.env.PREVIEW_SECRET) {
    return new Response("no", { status: 401 });
  }
  revalidatePath("/blog");
  revalidatePath("/projects");
  return Response.json({ revalidated: true });
}
```

و در پلاگین PHP فصل ۱۱: `wp_remote_post("https://site.vercel.app/api/revalidate?secret=...",`

`save_post` روی `[...]` . حالا publish = به‌روزرسانی فوری! 🚀

➔ **فصل بعد:** مهاجرت به هاست واقعی و ایران‌سازی – از لوکال به اینترنت؛ با انماد، درگاه و پنل SMS.

فصل ۱۸ – مهاجرت به هاست واقعی و ایران‌سازی IR

🎯 هدف: از لوکال به اینترنت. در این فصل: خرید هاست و دامنه، نصب وردپرس روی هاست، مهاجرت سایت لوکالت (با همه محتوایش) به هاست، فعال‌سازی SSL – و بعد، «ایران‌سازی»: انماد، درگاه پرداخت، پنل SMS و نقشه‌های ایرانی. این فصل = لحظه‌ای که پروژه‌ات تحویل‌دانی می‌شود.

۱۸.۱ – چک‌لیست قبل از خروج از لوکال

قبل از مهاجرت، در سایت لوکال این‌ها را تمیز کن:

- [] تیک «Discourage search engines» برداشته شود؟ – برعکس! لوکال تیکش را بگذار (تست) و روی هاست برنذار (فصل ۹) ⚠️
- [] محتوای نمونه/پیش‌فرض (Hello world، صفحه Sample) حذف یا جایگزین شده
- [] Permalink روی Post name (فصل ۹)؛ پست‌ها slug لاتین تمیز دارند
- [] پلاگین‌های اضافی حذف شدند (نه غیرفعال – Delete؛ فصل ۷)
- [] یک بکاپ کامل گرفته شد (فصل ۱۳) – خروج از لوکال بدون بکاپ ممنوع!
- [] رمزهای production قوی‌اند (مخصوصاً اگر از لوکال کپی می‌شود)

۱۸.۲ – خرید هاست و دامنه (بازار ایران)

مسیر استاندارد خرید برای سایت وردپرسی ایرانی:

قدم	چه می‌خری	نکته انتخاب
۱	دامنه	.ir از ایرنیک/ریسلرها (ارزان) یا .com – فصل ۲
۲	هاست وردپرسی اشتراکی	حداقل: SSL، MySQL 8، PHP 8.1+ رایگان (Let's Encrypt)، بکاپ خودکار هفتگی هاست، پشتیبانی پاسخ‌گو
۳	SSL	معمولاً رایگان با هاست – بررسی کن

💡 هاست‌های ایرانی (لوکاس، میزبان‌فا، نویان، پارس‌پک و ده‌ها دیگر) پرداخت ریالی + پشتیبانی فارسی + سرور داخل ایران دارند؛ هاست خارجی برای مخاطب جهانی. انتخاب = مخاطب پروژه، بودجه و نیاز به درگاه/انماد (سایت‌های جدی ایرانی: هاست داخلی + اسکیل‌شده).

بعد از خرید: هاستینگ **Nameserver** ها را می‌دهد (`ns1.host.ir` / `ns2.host.ir`) → در پنل دامنه (ایرنیک یا رجیستری) وارد می‌کنی (فصل ۲: رکورد NS) → چند دقیقه تا چند ساعت (Propagation).

۱۸.۳ – نصب وردپرس روی هاست: دو راه

راه ۱: نصب یک‌کلیکی (سریع) (★)

سی‌پنل → Install → WordPress → **Softaculous / WordPress Toolkit** → دامنه را انتخاب + کاربر/رمز ادمین → Done. معمولاً SSL و به‌روزرسانی خودکار هم تنظیم می‌کند.

راه ۲: نصب دستی (بفهم زیرش چه می‌گذرد – یک بار انجامش بده)

۱. دانلود وردپرس از `fa.wordpress.org` (نسخه فارسی)
۲. سی‌پنل → `public_html` → File Manager → آپلود **Extract** → zip
۳. **MySQL Databases**: دیتابیس بساز (مثلاً `user_wp1`) + کاربر دیتابیس + Add User To DB (همه دسترسی‌ها) – همان مفهوم فصل ۲ ⚠ نام دیتابیس/کاربر در سی‌پنل با پیشنهاد اکانت است
۴. مرورگر: `yoursite.ir` → ویزارد نصب وردپرس → اطلاعات دیتابیس مرحله ۳ را می‌پرسد → وارد کن (DB_HOST معمولاً `localhost`)
۵. عنوان سایت + کاربر ادمین + رمز قوی → نصب → ورود به پنل

🔑 پشت پرده ویزارد: وردپرس فایل `wp-config.php` را با ۴ ثابت

`DB_NAME` / `DB_USER` / `DB_PASSWORD` / `DB_HOST` می‌سازد – اگر روزی دستی هم بچینی، همین فایل است (معادل `.env` پروژه‌هایت).

۱۸.۴ – مهاجرت لوکال → هاست: سه راه

راه ۱: پلاگین مهاجرت (استاندارد) (★)

All-in-One WP Migration یا **Duplicator**:

۱. در لوکال: نصب پلاگین → Export → فایل می‌گیرد (کل سایت + دیتابیس + رسانه در یک بسته)
 ۲. روی هاست: وردپرس تمیز نصب کن (۱۸.۳ راه ۱) → همان پلاگین → Import → فایل را بده → جایگزین کامل می‌کند
 ۳. آدرس‌ها را خودش از `wp-course.local` به `yoursite.ir` بازنویسی می‌کند (سرچ‌ریپلیس سریالی دیتابیس) ✓
- (محدودیت: نسخه رایگان All-in-One محدودیت حجم آپلود دارد – برای سایت‌های بزرگ: Duplicator یا راه ۲)

راه ۲: دستی (یک بار انجامش بده تا مهاجرت را «بفهمی»)

1. فایل‌ها: `wp-content` کامل (تم/پلاگین/آپلود) را zip بزن → روی هاست اکسترکت کن (علی‌رغم تمیز نصب‌شده)
2. دیتابیس: در لوکال `wp db export site.sql` → در phpMyAdmin هاست: Import
3. ★ **بازنویسی آدرس‌ها** – دیتابیس پر از `http://wp-course.local` است. با WP-CLI روی هاست (اگر در دسترس) یا `phpMyAdmin → SQL`:

```
-- (ساده؛ برای رسانه‌ها هم کافی است) در phpMyAdmin:
UPDATE wp_options SET option_value = REPLACE(option_value, 'http://wp-course.local', 'https://yoursite.ir');
UPDATE wp_posts SET post_content = REPLACE(post_content, 'http://wp-course.local', 'https://yoursite.ir');
```

- ابزار حرفه‌ای‌تر: پلاگین **Better Search Replace** (همه جدول‌ها، با `serialize-safe`) – یا در `wp search-replace` Local: `replace "http://wp-course.local" "https://yoursite.ir"` (همین دستور در چیت‌شیت فصل ۲۰ هست)
4. `wp-config.php` هاست را با دیتابیس هاست ست کن (اطلاعات ۱۸.۳)

راه ۳: خودکار هاستینگ

خیلی هاستینگ‌های ایرانی سرویس «انتقال رایگان سایت وردپرسی» دارند – فایل بده، بقیه با آنها. (برای پروژه‌های مشتری جدی: خودت با راه ۱ انجامش بده و برای هدیه نباشن ریسک، بکاپ نزد خودت باشد)

۱۸.۵ – بعد از مهاجرت: چک‌لیست حیاتی

- [] **SSL**: سی‌پنل → Run AutoSSL → SSL/TLS Status (فصل ۲) → General → Settings → آدرس سایت `https` شود
- [] **حذف حالت تست**: Discourage search engines → Reading «تیک برداشته» شود ⚠️ (برعکس لوکال!)
- [] Settings → Permalinks → Save (یک بار – رفرش قواعد؛ فصل ۱۰)
- [] هر صفحه اصلی را باز کن: عکس‌ها لود می‌شوند؟ (اگر شکست: سرچ‌ریپلیس رسانه‌ها راه ۲-گام ۳)
- [] فرم تماس ایمیل می‌فرستد؟ (SMTP با ایمیل دامنه – فصل ۱۳)
- [] درگاه تستی یک سفارش واقعی بزن (فروشگاه)
- [] در GSC (فصل ۸) دامنه جدید را ثبت و sitemap سابمیت کن
- [] بکاپ UpdraftPlus روی هاست + Google Drive دوباره تنظیم و تست شود
- [] **چک API (برای پروژه‌های Headless)**: `https://yoursite.ir/wp-json/graphql` و از اینترنت باز شود – و `.env` پروژه Next.js به دامنه جدید (فصل ۱۷)

سایت تحویل مشتری ایرانی = این چهار تا را هم بچسبان:

۱. انماد (enamad.ir) – نماد اعتماد الکترونیکی

مجوز رسمی مرکز تجارت الکترونیکی برای «فروش آنلاین معتبر». برای چه؟ درگاه‌های پرداخت مستقیم بانکی بدون انماد داده نمی‌شوند؛ کاربر ایرانی هم به سایت بدون لوگوی انماد اعتماد نمی‌کند. مسیر: ثبت‌نام → بارگذاری مدارک → صدور → قرار دادن اسکریپت نماد در فوتر سایت (بلوک Custom HTML یا ویجت Elementor – فصل ۶). درجه‌ها: یک ستاره/دو ستاره/... – برای شروع یک ستاره کافی است.

۲. درگاه پرداخت مستقیم

زرین‌پال و آیدی‌پی (فصل ۱۲) واسطه‌اند و سریع‌اند؛ برای حجم بالا/جدی: درگاه مستقیم بانکی (شاپرک) = حساب تجاری بانک + قرارداد + انماد. پلاگین ووکامرس همه بانک‌ها موجود است. (برای سایت‌های non-shop که «پرداخت» دارند – شهریه دوره، رزرو – درگاه واسطه ساده‌ترین است.)

۳. پنل SMS ★ – ستون گمشده اکثر سایت‌های ایرانی

ایمیل در ایران کم‌خوان می‌شود؛ پیامک کانال اصلی اطلاع‌رسانی است (کد تأیید، سفارش جدید، وضعیت ارسال):

۱. پنل SMS بخر: کاوه‌نگار، ملی‌پیامک، فراز و... (خط خدماتی/اشتراکی – هزینه خط + بسته پیامک)
۲. پلاگین اتصال به ووکامرس را نصب کن (همه پنل‌ها پلاگین وردپرسی دارند)
۳. رویدادها را وصل کن: سفارش جدید → SMS به مدیر؛ تغییر وضعیت سفارش → SMS به مشتری؛ فراموشی رمز → کد با پیامک

۴. نقشه و آدرس ایرانی

- فرم آدرس: شهر/استان ایران در ووکامرس با پلاگین فارسی‌ساز (فصل ۱۲) ست می‌شود
- نقشه تماس: نشان یا بلد به جای گوگل مپ (فصل ۸) – embed یا iframe
- تقویم شمسی در همه جا (پلاگین فارسی‌ساز فصل ۱۲)

۱۸.۷ – هزینه واقعی راه‌اندازی (برآورد بازار ۱۴۰۴)

آیتم	هزینه سالانه/ماهانه (تقریبی)
دامنه <code>.ir</code>	۱۰۰-۲۰۰ هزار تومان/سال
هاست وردپرسی اشتراکی متوسط	۲-۵ میلیون تومان/سال
SSL	رایگان (Let's Encrypt)
تم premium / افزونه‌ها (در صورت نیاز)	۱-۱۰ میلیون تومان/سال
پنل SMS + بسته	۲۰۰-۵۰۰ هزار تومان + مصرفی
کل راه‌اندازی مشتری‌محور	مبنای قیمت‌گذاری تو – فصل ۱۹ ★

(اعداد تقریبی‌اند و بازار عوض می‌شود – قبل از پیش‌فاکتور، قیمت روز را چک کن.)

۱۸.۸ – تحویل به مشتری

- **کاربر مشتری: Editor** نه Admin (فصل ۹) – رمز قوی جدا؛ تو Admin می‌مانی
- یک جلسه آموزش پنل (نیم ساعت: محتوا/رسانه/سفارش‌ها) + ویدیو ضبط‌شده از همان جلسه ★
- فایل تحویل: اطلاعات دامنه/هاست/پنل‌ها در یک doc + اطلاعات Application Password API (اگر Headless)
- **قرارداد نگهداری** جدا از ساخت (فصل ۱۹): آپدیت/بکاپ/مانیتورینگ ماهانه
- سایت زنده = شروع رابطه؛ مرگ سایت بعد از تحویل = آبروی تو!

✓ جمع‌بندی فصل

- قبل از مهاجرت: محتوا تمیز + Permalink + **بکاپ کامل**؛ تیک سئوی «تست» را یادت برود روی هاست!
- خرید: دامنه (ایرنیک) + هاست وردپرسی با SSL رایگان → NS را وصل کن
- نصب: Softaculous یک‌کلیکی ★ یا دستی (zip + دیتابیس + `wp-config`)
- مهاجرت: **All-in-One WP Migration / Duplicator** خودکار، یا دستی: فایل + `wp db export` + سرچ‌ریپلیس دامنه
- بعد از مهاجرت: SSL → تیک سئو برداشته → Save Permalinks → تست کامل → GSC → بکاپ → تست API
- ایران‌سازی: **انماد + درگاه + پنل SMS + نشان/بلد** – چهار ستون تحویل بازار ایران

1. (بدون پول!) مسیر خرید را شبیه‌سازی کن: در irnic.ir ثبت‌نام را تا مرحله تأیید هویت پیش برو؛ در پنل یک هاستینگ ایرانی، فاکتور هاست + دامنه را تا قبل از پرداخت بساز و اسکرین‌شات های پیش‌فاکتور را نگه دار (فصل ۱۹: برای پیش‌فاکتور به مشتری استفاده می‌شود).
2. مهاجرت را **داخل لوکال** تمرین کن: یک سایت Local دوم ([wp-live](#)) بساز و سایت اصلی را با All-in-One WP Migration به آن منتقل کن – دامنه‌ها عوض شد و عکس‌ها سالم‌اند؟ ★ این همان تمرین هاست است بدون هزینه.
3. نسخه دستی هم یک بار: export دیتابیس + [wp search-replace](#) – فرق روش‌ها را حس کن.
4. فرم درخواست انماد را باز کن (بدون ارسال) – چه مدارکی می‌خواهد؟ برای مشتری‌ات لیست کن.
5. یک پنل SMS (کاوه‌نگار/ملی‌پیامک) ثبت‌نام تست کن و داکيومنت پلاگین ووکامرسی‌اش را بخوان – رویدادهای قابل اتصال را لیست کن.
6. چک‌لیست ۱۸.۵ را برای یک سایت فرضی پر کن – کدام آیتم‌ها بدون هاست قابل تمرین‌اند؟ (بقیه را در اولین پروژه واقعی اجرا کن!)

▼ نکته تمرین ۲

All-in-One WP Migration: Export → File (حجم را می‌بینی – اگر < حد رایگان بود، Export با افزونه‌ی «chunk» یا Duplicator). Import در سایت دوم خودش سرچ‌ریپلیس می‌کند؛ فقط بعد از مهاجرت: Save → Permalinks و Test API اگر Headless است. اگر عکس‌ها شکستند: ویرایش یک پست → عکس را re-select کن و بعد سرچ‌ریپلیس رسانه‌ها را چک کن.

➔ **فصل بعد:** فریلنسری و درآمد – همه این مهارت‌ها را به «قیمت» تبدیل کن.

فصل ۱۹ – فریلنسری و درآمد: مهارت را به قیمت تبدیل کن 💰

🎯 هدف: تا اینجا ۱۸ فصل مهارت ساختی؛ این فصل ساختن درآمد است. سه محصول خدماتی که با همین دوره می‌فروشی، قیمت‌گذاری، پیدا کردن مشتری (ایرانی و جهانی)، گردش کار پروژه از جلسه اول تا تحویل، و قرارداد نگهداری ماهانه. این فصل ورودی‌های نظری ندارد – همه‌اش اجراست.

۱۹.۱ – بازار: چرا وردپرس هنوز پول‌ساز است؟

- **۴۰٪+ وب دنیا** وردپرسی است و بازار ایران تقریباً تمام کسب‌وکارهای کوچک/متوسطش یا وردپرس دارد یا می‌خواهد
- رقابت اصلی تو: سازنده‌های «سایت در ۲۴ ساعت» با تم آماده – ضعف‌شان: بدون سئو، بدون امنیت، بدون نگهداری، بدون مقیاس
- **مزیت تو:** دولوپری که وردپرس را مثل فصل ۱ تا ۲۰ می‌فهمد (CPT/ACF/API) و Next.js بلد است – ترکیب کمیاب. در بازار جهانی، «Headless WordPress developer» یک تخصص جست‌وجوشده و پردرآمد است

💡 قانون بازار: هرچه تحویل «شخصی‌تر و فنی‌تر»، قیمت بالاتر. تم نصب کنی = قیمت تم؛ فروشگاه واقعی با درگاه/انماد/پیامک بسازی = قیمت پروژه؛ Headless با پرفورمنس تحویل بدهی = قیمت تخصص.

۱۹.۲ – سه محصول خدماتی تو

با دانش همین دوره، سه «بسته» با سه سطح قیمت:

بسته	محتوا	ابزار فصل‌ها	مشتری هدف	قیمت نسبی
۱. سایت شرکتی	۵-۸ صفحه، تم بلوکی یا Elementor، فرم، سئوی پایه، SSL، هاست	۵، ۶، ۸	کسب‌وکار کوچک	💰
۲. فروشگاه ووکامرسی	بسته ۱ + محصولات، درگاه، انماد، SMS، گزارش	۱۲، ۱۳، ۱۸	فروشنده/برند	💰💰
۳. سایت Headless premium	وردپرس به‌عنوان پنل + فرانت + Next.js + SEO/ISR + دپلوی	۱۷-۱۰	استارت‌آپ/برند جدی/پروژه بین‌المللی	💰💰💰

قاعده ارائه: هر سه را داری – بر اساس بودجه و بلوغ مشتری پیشنهاد بده، نه یک جور سرویس برای همه.

۱۹.۳ – قیمت‌گذاری

سه مدل رایج:

1. **پروژه‌ای (Fixed):** رایج‌ترین در ایران. فرمول اولیه: $\text{قیمت} = \text{برآورد ساعت} \times \text{نرخ ساعتی} \times ۱.۳$ (ضریب برای جلسات/اصلاحات/غیرقابل پیش‌بینی)
 2. **ساعتی:** برای کار مبهم/مشاوره/تعمیر سایت‌های دیگران (Hacked site repair) گران‌تر از ساخت از صفر! چون ریسک داری
 3. **نگهداری ماهانه (Retainer) ★:** بسته فصل ۱۳ به صورت ماهانه – آپدیت + بکاپ + مانیتورینگ + چند ساعت تغییر. **درآمد تکرارشونده** – ۵ مشتری $\times X =$ حقوق ماهانه‌ات بدون پروژه جدید
- قواعد چانه‌زنی:

- هرگز قیمت را سر جلسه اول درجا نگو – ۲۴ ساعت «پیش‌فاکتور» بده (مجال محاسبه + اقتدار)
- قیمت پایین‌تر از بازار = مشتری بدتر؛ مراجعان ارزان‌جو وقت تو را می‌خورند
- تخفیف فقط در برابر چیز بده: پرداخت کامل اول کار، یا محتوای آماده توسط مشتری، یا حجم بیشتر

۱۹.۴ – مشتری از کجا؟

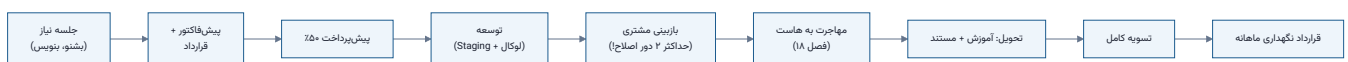
ایران:

- **پونیشا، کارلنسر، ترب:** پروژه‌های وردپرسی روزانه؛ رقابت بالاست – با نمونه‌کار Headless متمایز شو (کسی ندارد!)
- شبکه‌های اجتماعی حرفه‌ای (لینکدین فارسی) + معرفی (پروژه‌های قبلی)
- آژانس‌ها: به عنوان فریلنسر زیرمجموعه کار بگیر – جریان پروژه پایدار، قیمت متوسط
- فروشگاه‌سازهای محلی/غرفه‌داران: نیازمند فروشگاه واقعی (بسته ۲)

جهانی (Headless تو می‌درخشد ★):

- **Upwork / Fiverr:** جستجو برای `headless wordpress next.js` – رقابت کمتر از وردپرس کلاسیک و نرخ ساعتی ۴-۲ برابری
- لینکدین انگلیسی + GitHub با پروژه‌های open source (پلاگین‌های فصل ۱۱! را به عنوان نمونه کار فنی منتشر کن)

۱۹.۵ – گردش کار پروژه (از تماس تا تحویل)



قواعد طلایی هر مرحله:

- **جلسه نیاز:** دامنه پروژه را مکتوب کن – «سایت با ۸ صفحه» نه «یه سایت شیک». محتوا را از مشتری بگیر (عکس‌ها/متن) – تعویق محتوا = تعویق تحویل؛ این را در قرارداد بنویس
- **پیش‌فاکتور:** ریزآیتم‌ها + مدت + پرداخت ۵۰/۵۰ + «محتوا توسط کارفرما» ★
- **توسعه:** روی لوکال/Staging نه هاست اصلی؛ Snapshot و بکاپ‌های فصل ۱۳ همیشه
- **بازبینی:** ★ حداکثر ۲ دور اصلاح در قرارداد – «اصلاح بی‌نهایت» قاتل سود فریلنسر است
- **تحویل:** چک‌لیست فصل ۱۸ (SSL، سئو، بکاپ، درگاه) + آموزش ویدیویی پنل (Admin نه Editor!)

۱۹.۶ – قرارداد: حداقل‌های لازم

چیزی که روز دعا نجات می‌دهد (فرمت پیشنهادی خودت را با مشاوره حقوقی نهایی کن):

1. محدوده کار (قسمت خیریه – هر چیز خارج آن = هزینه جدا)
2. زمان‌بندی با فرض «محتوا تا تاریخ X آماده»
3. پرداخت: ۵۰ شروع / ۵۰ تحویل (پروژه بزرگ: ۳۰/۴۰/۳۰)
4. اصلاحات: ۲ دور، هر دور مشخص
5. مالکیت: دامنه و هاست به نام **مشتری** از روز اول ★ (سایت مشتری به نام تو = گروگان‌گیری برعکس – بدنامی)
6. نگهداری: قرارداد جدا؛ تغییرات خارج نگهداری = پروژه جدید
7. تحویل دادنی‌ها: فایل‌ها/دسترسی‌ها/آموزش

۱۹.۷ – پورتفولیو: ویتترین تو

همین دوره سه نمونه‌کار به تو می‌دهد:

1. **سایت شرکتی «برق‌آسا»** (فصل ۶) – دمو بسته ۱
2. **فروشگاه ابزار** (فصل ۱۲ + فصل ۱۸) – دمو بسته ۲ با درگاه تست
3. **سایت Headless** (فصل ۱۶ + دپلوی فصل ۱۷) ★ – این را آنلاین دپلوی کن و لینک بده؛ برای بازار جهانی، پشتش یک Case Study بنویس (معماری: WP as CMS + WPGraphQL + Next.js ISR + Lighthouse score)

🔑 فرمت هر Case Study: مشکل → راه‌حل → تکنولوژی → نتیجه (سرعت/سئو) → لینک. متنی، کوتاه، به انگلیسی – در لینکدین و GitHub README.

۱۹.۸ – رشد پس از فریلنسری

- **آژانس کوچک:** ۲-۳ نفر (تو فنی، یکی محتوا/پشتیبانی) – ظرفیت پروژه ۳×
- **محصول‌سازی:** قالب/پلاگین ایرانی با پشتیبانی ماهانه فروش؛ یا افزونه‌ای که در پروژه‌ها نوشتی (فصل ۱۱) را عمومی کن

- **آموزش:** دوره/محتوای فارسی Headless WordPress – بازار تقریباً خالی است، همین دوره را دوباره فروشی کن 😊
- مسیر بین‌المللی: ریموت‌کارهای Headless (لینکدین + ریموکرافت) با نرخ دلاری

جمع‌بندی فصل ✓

- سه بسته خدماتی: شرکتی / فروشگاه / **Headless premium** – بر اساس بودجه مشتری پیشنهاد بده
- قیمت = ساعت × نرخ × ۱.۳؛ **نگهداری ماهانه = درآمد تکرارشونده** (فصل ۱۳ می‌فروشی)
- مشتری ایران: پلتفرم‌ها + معرفی؛ جهانی: Headless + لینکدین + GitHub
- گردش کار: نیاز مکتوب → پیش‌فاکتور → ۵۰/۵۰ → توسعه ۲ → Staging **دور اصلاح** → مهاجرت → آموزش → تسویه → نگهداری
- قرارداد: محدوده/پرداخت/مالکیت به نام مشتری/نگهداری جدا
- پورتفولیو از همین دوره: ۳ دمو + Case Study انگلیسی برای Headless

تمرین فصل ۱۹ (خروجی: بسته فروش تو) 📄

1. سه بسته خدماتی خودت را بنویس: برای هر کدام «چه چیزی تحویل می‌شود» در ۵ بند + زمان تحویل + قیمت (برآورد نرخ ساعت بازار را بپرس/چک کن).
2. یک پیش‌فاکتور نمونه برای «فروشگاه ابزار» بساز (ریزآیتم + شرایط پرداخت + بند محتوا و اصلاح) – قالبش را نگه دار، برای هر مشتری کپی می‌کنی.
3. در پونیشا/کارلنسر ۵ آگهی وردپرسی را بخوان: کدام را با دانش فعلی می‌توانی بگیری؟ برای یکی پیشنهاد (پروپوزال) بنویس.
4. در Upwork جستجوی `headless wordpress` – نرخ‌های ساعتی را یادداشت کن (کیفیت کیف پول را تغییر می‌دهد!).
5. Case Study پروژه Headless فصل ۱۶ را به انگلیسی بنویس (مشکل/راه‌حل/نتیجه) و در GitHub README پروژه بگذار.
6. چک‌لیست نگهداری ماهانه فصل ۱۳ را به یک «بسته نگهداری» با ۳ سطح قیمت تبدیل کن.

▼ نکته تمرین ۳

پروپوزال کوتاه و اختصاصی می‌برد: ۳ خط اول = درک مشکل مشتری (نقل از آگهی‌اش)، بعد راه‌حل تو با نمونه‌کار مرتبط (لینک دمو فصل ۱۶)، بعد زمان/قیمت، بعد یک سؤال هوشمندانه. «Dear sir, I am expert in wordpress...» کپی‌پیست‌ها را نزن – همان‌ها را رد می‌کنند.

[→ فصل آخر: چیت‌شیت جامع و گلاساری – مرور سریع کل دوره.](#)

فصل ۲۰ – چیت‌شیت، گلاساری و نقشه راه

هدف: مرجع مرور سریع چهار دنیا: مدیریت وردپرس، صفحه‌سازی و فروشگاه، زیرساخت و مهاجرت، توسعه
.Headless

۲۰.۱ – چیت‌شیت پنل وردپرس

— مسیره‌های پنل —

Posts / Pages	← محتوای زمان‌دار / ثابت (فصل ۴)
Appearance → Themes	← نصب و فعال‌سازی تم
Appearance → Editor	← Site Editor (تم‌های بلوکی) → Header/Footer/Styles
Appearance → Menus	← (فصل ۶ – Elementor/مگامنو: تنظیمات تم) منوها + زیرمنو
Plugins → Add New	← (Install ≠ Activate!) نصب پلاگین
Users → Profile	← (!پایین صفحه) Application Passwords
Settings → General	← (فصل ۶) Timezone، Site Icon، زبان (فارسی!)
Settings → Permalinks	← CPT بعد از ساخت Post name + Save ★
Settings → Reading	← فقط برای تست «Discourage» صفحه اصلی / تعداد پست / تیک
WooCommerce → Settings	← تومان/ارسال/درگاه (فصل ۱۲)
WooCommerce → Orders	← گردش سفارش: Pending → Processing → Completed
Marketing → Coupons	← کوپن تخفیف
Yoast/Rank Math	← (فصل ۸) sitemap + متا در هر پست + SEO Title
UpdraftPlus / Wordfence	← بکاپ زمان‌بندی‌شده / فایروال و اسکن (فصل ۱۳)

— WP-CLI (در Local → Open Site Shell) —

wp core version	
wp plugin list / install x --activate / update --all / delete x	
wp theme list / activate x	
wp scaffold child-theme x-child --parent=x	
wp post list --post-type=project --fields=ID,post_title	
wp db export backup.sql / wp db import backup.sql	# بکاپ و ریستور دستی
wp search-replace "http://old" "https://new"	# (فصل ۱۸) مهاجرت دامنه!
wp maintenance-mode activate / deactivate	# (فصل ۱۳) حالت تعمیر
wp user create boss boss@x.ir --role=editor	# کاربر مشتری

— نجات —

Local Snapshot	← کامیت محیط (قبل از هر تغییر بزرگ)
Revision	← بازگشت محتوا: ویرایشگر → نسخه‌ها (فصل ۴)
Recovery mode	← اگر پلاگینی سایت را شکست، ایمیل لینک ریکاوری می‌آید
wp plugin deactivate --all	← سفیدی سایت: همه پلاگین‌ها را خاموش، یکی‌یکی روشن کن
rename plugins folder	← پوشه پلاگین مقصر را تغییر نام بده، FTP/Explorer: آخرین راه

۲۰.۲ – چیت‌شیت زیرساخت، مهاجرت و فروشگاه

— مهاجرت لوکال → هاست (فصل ۱۸) —

۱. بکاپ کامل (فصل ۱۳) ← بدون این هیچ قدمی
۲. هاست را در پنل دامنه (ایرنیک) وارد کن NS: دامنه
۳. wp-config + دیتابیس + zip: یک‌کلیکی / دستی Softaculous: نصب روی هاست
۴. یا دستی (All-in-One WP Migration (Export → Import): انتقال
دامنه wp-content zip + wp db export + search-replace
۵. تیک سئو برداشته شود → SSL (AutoSSL) ← بعد از مهاجرت
بکاپ دوباره → GSC → تست کامل → Save → Permalinks

— سی‌پنل (فصل ۲ و ۱۸) —

File Manager ← وردپرس zip/Extract فایل‌سیستم هاست؛
MySQL Databases ← DB_HOST = localhost کاربر + اتصال + ساخت DB
phpMyAdmin ← wp_posts و wp_postmeta دیتابیس؛ جدول Export/Import
SSL/TLS Status ← Run AutoSSL (رایگان Let's Encrypt)
Softaculous ← نصب وردپرس یک‌کلیکی

— ایران‌سازی (فصل ۱۲ و ۱۸) —

اولین کار بعد نصب - (و.و. Parsi WP) تومان + شمس‌سازی ← پلاگین فارسی‌ساز
درگاه پرداخت ← زرین‌پال/آیدی‌پی (واسطه)؛ مستقیم بانکی = انماد لازم
درگاه مستقیم + اعتماد؛ اسکریپت نماد در فوتر ← (enamad.ir) انماد
سفارش جدید/تغییر وضعیت → پیامک؛ پلاگین ووکامرس ← (کاوه‌نگار/ملی) SMS پنل
یکسان همه‌جا (فصل ۸) NAP نشان + بلد ← سئوی محلی؛ + Google Business

۲۰.۳ – چیت‌شیت Headless (کوئری‌ها و کد)

— GraphQL (فصل ۱۵) —

```
query GetPosts($first: Int!) {  
  posts(first: $first) {  
    pageInfo { hasNextPage endCursor }  
    nodes {  
      id slug title date excerpt  
      featuredImage { node { sourceUrl altText } }  
      categories { nodes { name slug } }  
    }  
  }  
}  
  
query GetPost($slug: ID!) {  
  post(id: $slug, idType: SLUG) {  
    title content date  
    author { node { name } }  
    featuredImage { node { sourceUrl } }  
  }  
}  
  
query GetProjects {  
  projects(first: 20) {  
    nodes {  
      slug title  
      featuredImage { node { sourceUrl } }  
      projectFields { year demoUrl technologies } # ACF  
      techStack { nodes { name } } # taxonomy  
    }  
  }  
}
```

```
// — الگوهای Next.js (فصل ۱۶-۱۷) —
export const revalidate = 60; // محتوا تا ۶۰ ثانیه تازه - ISR
export async function generateStaticParams() { ... } // SSG برای slug ها
const { slug } = await params; // params یک Promise است!
if (!post) notFound(); // تمیز 404
dangerouslySetInnerHTML={{ __html: post.content }} // HTML (امن = خودت)
(await draftMode()).enable(); // Preview Mode
revalidatePath("/blog"); // وردپرس → به روزرسانی فوری Webhook
```

```
# — REST (فصل ۱۴) سریع —
GET /wp-json/wp/v2/posts?slug=xxx&per_page=10
GET /wp-json/wp/v2/projects # CPT (show_in_rest!)
POST /wp-json/wp/v2/posts -u "user:app-pass" -d '{"title":"...", "status":"draft"}'
در X-WP-Total # تعداد کل (pagination)
```

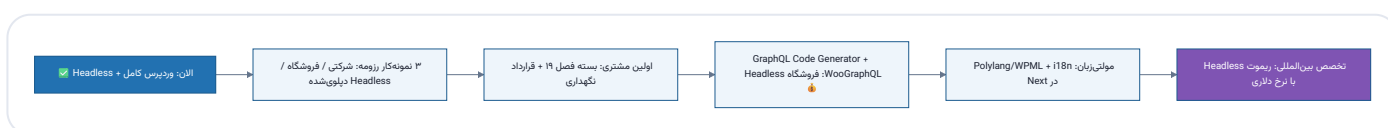
انگلیسی	فارسی	یک جمله
CMS	سامانه مدیریت محتوا	پنلی که مشتری با آن محتوا می‌گذارد
Domain / TLD	دامنه / پسوند	اسم سایت – .ir با ایرنیک، .com جهانی (فصل ۲)
Hosting	هاست	خانه فایل‌ها؛ اشتراکی/VPS – با PHP+MySQL
DNS / Nameserver	دی‌ان‌اس	دفتر تلفن: رکورد A، CNAME، MX، NS (فصل ۲)
SSL / HTTPS	گواهی امنیتی	قفل سبز – رایگان با Let's Encrypt
cPanel / File Manager	سی‌پنل / فایل منیجر	GUI هاست: فایل‌ها، دیتابیس، SSL، نصب وردپرس
Post / Page	نوشته / برگه	محتوای زمان‌دار / ثابت
Gutenberg / Block	گوتنبرگ / بلوک	ادیتور و واحد UI (با React!)
Theme (Block/Classic)	تم بلوکی/کلاسیک	ظاهر سایت – جدید: Site Editor
Child Theme	تم فرزند	لایه تغییرات تو، جدا از تم اصلی
Plugin	افزونه	بسته قابلیت (npm وردپرس)
Page Builder	صفحه‌ساز	Elementor/WPBakery – Section/Column/Widget (فصل ۶)
Mega Menu	مگامنو	زیرمنوی پهن چندستونه برای دسته‌های بزرگ
Favicon / Site Icon	آیکون سایت	لوگوی کوچک کنار تب مرورگر
SEO / Meta Title	سئو / عنوان متا	دیده‌شدن در گوگل؛ مهم‌ترین تگ صفحه (فصل ۸)
Sitemap	نقشه سایت	فهرست XML صفحات برای گوگل (/sitemap_index.xml)
Search Console	سرچ کنسول	پنل گوگل: ایندکس، sitemap، گزارش‌ها
WooCommerce	ووکامرس	فروشگاه‌ساز وردپرس: محصول/سبد/سفارش (فصل ۱۲)
Variable Product	محصول متغیر	چند گزینه (رنگ/سایز) با قیمت و موجودی جدا
Payment Gateway	درگاه پرداخت	واسطه بانکی (زرین‌پال...) یا مستقیم با انماد
Enamad	انماد	نماد اعتماد الکترونیکی – پیش‌نیاز درگاه مستقیم
SMS Panel	پنل پیامک	اطلاع‌رسانی سفارش/تأیید با SMS (فصل ۱۸)

انگلیسی	فارسی	یک جمله
Role	نقش کاربری	Editor = مشتری – Subscriber→Author→Editor→Admin
Slug	نامک	بخش آخر URL – کلید routing تو
CPT (Custom Post Type)	نوع محتوای سفارشی	محصول/پروژه/نمونه کار – ساختار داده
Custom Field / ACF	فیلد سفارشی	فیلدهای ساخت‌یافته (...year، url)
Taxonomy	طبقه‌بندی	دسته/برچسب سفارشی برای فیلتر
WP-CLI	–	خط فرمان وردپرس (db/search-replace/maintenance)
Hook (Action/Filter)	قلاب	رویداد / تغییر مقدار – هسته توسعه PHP
WP REST API	–	JSON – <code>/wp-json/wp/v2/...</code> استاندارد
Application Password	رمز برنامه	توکن برای API (مثل PAT)
WPGraphQL / GraphiQL	–	سرور و پلی‌گراند GraphQL وردپرس
Headless	بدون سر (فرانت جدا)	وردپرس فقط بک‌اند؛ فرانت = Next.js
ISR / revalidate	بازاعتبارسنجی	کش با انقضا – محتوا تازه بدون دپلوی
Draft Mode	حالت پیش‌نویس	دیدن پیش‌نویس در سایت Next.js
Maintenance Mode	حالت تعمیر	صفحه «به‌زودی» هنگام تغییرات (فصل ۱۳)
Staging	استیجینگ	نسخه تست سایت – آپدیت اول آنجا، بعد Live
CDN	شبکه توزیع محتوا	Cloudflare/داخلی – سرعت + لایه امنیتی
SMTP	–	ارسال واقعی ایمیل‌های وردپرس (WP Mail SMTP)
Retainer	نگهداری ماهانه	درآمد تکرارشونده: آپدیت/بکاپ/مانیتورینگ (فصل ۱۹)

۲۰.۵ – گیرهای کلاسیک و علاج

مشکل	علاج
CPT در پنل نیامد	کد را چک کن؛ بعد Save → Permalinks → Settings (رفرش قواعد)
CPT در REST 404 داد	<code>show_in_rest ⇒ true</code> نداشتی (فصل ۱۰)
فیلدهای ACF در API نیست	پلاگین ACF to REST / Show in GraphQL در Field Group
کوئری GraphQL فیلد را نمی‌شناسد	refresh :GraphQL = autocomplete schema در Ctrl+Space
عکس در next/image لود نشد	remotePatterns در next.config (فصل ۱۷)
محتوا آپدیت نمی‌شود	<code>revalidate</code> را چک کن؛ برای فوری: Webhook (فصل ۱۷)
Preview خالی/404	secret دو طرف یکی نیست؛ Application Password فعال؟
سایت سفید شد (کد PHP)	<code>wp plugin deactivate --all</code> ؛ WP_DEBUG روشن کن؛ Snapshot بازگردان
ارور حافظه/سنگینی	پلاگین‌های اضافه را کم کن؛ PHP memory در Local قابل افزایش
<code>params</code> بدون await ارور داد	در Next 15 params یک Promise است: <code>await params</code>
بعد از مهاجرت عکس‌ها/لینک‌ها شکستند	سرچ‌ریپلیس دامنه کامل نمانده – Better Search Replace روی همه جدول‌ها (فصل ۱۸)
بعد از مهاجرت همه‌چیز 404 است	Settings → Permalinks → Save (ری‌نوشت htaccess)
ایمیل‌های سایت نمی‌رسند	<code>mail()</code> هاست خاموش/اسپیم – WP Mail SMTP + تست ایمیل (فصل ۱۳)
درگاه پرداخت سفارش را آپدیت نمی‌کند	SSL فعال نیست؛ IPN/callback یا Merchant ID چک شود؛ لاگ درگاه
ووکامرس تاریخ میلادی/قیمت دلاری	پلاگین فارسی‌ساز نصب/فعال نیست؛ واحد پول در Settings (فصل ۱۲)
محصول متغیر گزینه ندارد	ویژگی (Attribute) را Save کن، بعد تب Variations → Generate ★
هک شد/ریدایرکت اسپم	Wordfence Scan یا ریسکتور بکاپ تمیز + تعویض همه رمزها (فصل ۱۳)

۲۰.۶ – نقشه راه پس از این دوره



سه توصیه پایانی:

1. پروژه رزومه بساز و آنلاینش کن: سایت Headless وردپرس + Next.js با داده واقعی روی Vercel – در بازار ایران این ترکیب کمیاب و پولساز است؛ لینکش را بالای رزومهات بگذار (فصل ۱۹: Case Study انگلیسی).

2. وردپرس را مثل محصول نگه دار: آپدیت‌ها، بکاپ ۱-۲-۳، پلاگین کم، رمز قوی، ۹۰٪ – Wordfence ماجرا همین انضباط است (فصل ۱۳) و همان است که مشتری بابتش ماهانه می‌پردازد.

3. داکيومنت رسمی: www.wpgraphql.com/docs، developer.wordpress.org (REST + Block Editor)، و داکيومنت ووکامرس – هر سه با پایه‌ای که الان داری خوانا شده‌اند.

سفر پنج‌گانه‌ات تا اینجا: WordPress → Docker → CLI → Git/GitHub → PostgreSQL+Prisma → Headless + کامل

تو الان می‌توانی: دیتابیس طراحی کنی، با تیم کار کنی، ترمینال را هدایت کنی، روی سرور دپلوی کنی – و حالا سایت شرکتی و فروشگاه ایرانی تحویل بدهی، CMS را برای مشتری مهاجرت و نگهداری کنی، و با Next.js فرانت مدرن بسازی. پکیج کامل فول‌استک + بازار! 🚀🎉